M01 — Fundamentos de ASP.NET Core MVC

Material teórico

CURSO: Curso de Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

TIPO: TEORÍA

AUTOR: JAIME GALLO

Orientación de lectura y aprendizaje

Este manual es una **referencia de estudio**, no un guion para leer íntegramente en las dos horas de teoría previstas para M01. El recorrido esencial sigue cinco ideas: petición y respuesta HTTP (1.1), separación de responsabilidades MVC (1.2), ubicación y arranque de sus piezas (1.3), selección de acciones y resultados (1.4), y composición de HTML con Razor (1.5).

En clase, cada idea se presenta con **un problema concreto y el fragmento mínimo que lo resuelve**; a continuación se comprueba en la práctica correspondiente. Las comparaciones extensas, diagnósticos alternativos y variantes de código sirven para profundizar después, sin convertirse en requisitos de la jornada. El alumno habrá aprendido cuando pueda predecir qué acción se ejecuta, qué objeto recibe la vista y qué HTML se entrega al navegador; memorizar todas las tablas no es el objetivo.

Índice del módulo

- **1.1. Introducción al desarrollo web con ASP.NET Core**
- **1.2. Arquitectura Model-View-Controller**
- **1.3. Estructura de un proyecto ASP.NET Core MVC**
- **1.4. Controladores, acciones y routing**
- **1.5. Vistas Razor, modelos tipados, layouts y vistas parciales**

1.1. Introducción al desarrollo web con ASP.NET Core

Objetivos de aprendizaje.

Al finalizar este punto, el alumno podrá explicar qué función desempeña ASP.NET Core en una aplicación web, diferenciar una respuesta HTML generada por MVC de una respuesta JSON generada por una API, identificar los elementos esenciales de una petición HTTP y seguir la ejecución desde el navegador hasta una acción MVC y una vista Razor.

Se utilizará una primera aplicación del curso, `CursoWeb.Mvc`, que mostrará una página relacionada con el futuro catálogo de productos. No necesitaremos base de datos, autenticación ni conocimientos nuevos de C#.

MVC y Web API: del contrato de datos a una página HTML

El problema que queremos resolver.

Hasta ahora el alumno sabe construir una Web API capaz de responder a una petición como `GET /api/productos`. Esa API puede entregar una colección JSON con los nombres y precios, pero todavía queda una decisión distinta: **¿quién transforma los datos en una página con título, navegación, lista y estilos?**

Podríamos programar un cliente que solicite JSON y construya el HTML. Sin embargo, para una aplicación web de páginas tradicionales ASP.NET Core MVC ofrece otra posibilidad: el servidor ejecuta una acción, prepara los datos y representa una **vista Razor** antes de devolver el HTML al navegador. Aprenderemos ese segundo recorrido sin abandonar las nociones de HTTP que ya conocemos.

ASP.NET Core es la plataforma web común. Proporciona hospedaje, configuración, inyección de dependencias, middleware y enrutamiento; sobre ella podemos programar API, MVC con vistas, Razor Pages o Blazor. La distinción entre estas alternativas no consiste en utilizar servidores ni protocolos diferentes, sino principalmente en cómo organizamos la atención de la petición y la representación resultante.

Dos respuestas posibles para una misma necesidad.

Imagina que debemos mostrar Cuaderno y Lápiz. Una API de productos podría responder:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

[{"id":1,"nombre":"Cuaderno"},{"id":2,"nombre":"Lápiz"}]
```

En cambio, una página MVC podría responder con el documento que el navegador sabe representar directamente:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

<!DOCTYPE html>
<html lang="es">
<body>
    <h1>Productos</h1>
    <ul>
        <li>Cuaderno</li>
        <li>Lápiz</li>
```

```
    </ul>
</body>
</html>
```

Las dos respuestas son HTTP 200, aunque difieren en el `Content-Type` y en el cuerpo. **JSON expresa datos estructurados; HTML define elementos de una página.** Una API no está obligada a devolver siempre JSON, ni un controlador MVC está obligado a devolver siempre HTML: aquí comparamos los usos que vamos a practicar.

Pregunta	Página MVC con Razor	Web API que ya conocemos
¿Dónde se ejecuta C#?	En el servidor ASP.NET Core	En el servidor ASP.NET Core
¿Qué recibe habitualmente el cliente?	HTML preparado con la vista	Datos, normalmente JSON
¿Quién construye la presentación inicial?	La vista Razor del servidor	El cliente que consume los datos
¿Pueden intervenir EF Core, servicios y rutas?	Sí	Sí
¿Se necesita obligatoriamente una base de datos?	No	No

Esta tabla describe **responsabilidades predominantes**, no prohibiciones técnicas. Una aplicación puede tener páginas MVC y endpoints de API al mismo tiempo. También puede incluir JavaScript en sus páginas. Para comprender el primer flujo del curso no necesitaremos ninguna de esas variantes.

Cómo se materializa la decisión en el proyecto del alumno.

La primera versión de `CursoWeb.Mvc` no muestra todavía la lista de productos. Nos interesa identificar la generación de una página antes de introducir los modelos del punto 1.2. Su controlador contiene una acción sencilla:

```
[HttpGet]
public IActionResult Index()
{
    return View();
}
```

La llamada `View()` no devuelve inmediatamente una cadena HTML ni indica al navegador que abra un archivo `.cshtml`. Crea un resultado de MVC. Cuando se ejecuta ese resultado, el framework localiza la vista asociada a `HomeController.Index`, utiliza `Views/Home/Index.cshtml` y compone la respuesta. El layout compartido añade la estructura exterior, como navegación y cabecera de documento.

En la vista inicial encontramos, entre otras líneas:

```
@{
    ViewData["Title"] = "Primer recorrido MVC";
}

<h1>1.1 | Primer recorrido MVC</h1>
<p>Esta página HTML ha sido generada en el servidor mediante una acción MVC.</p>
```

La expresión `ViewData["Title"]` proporciona un título al layout y el `<h1>` forma parte del cuerpo visible. Estas son dos responsabilidades de presentación distintas. Todavía no vamos a modificar Razor ni a explicar toda su sintaxis: en el punto 1.5 abordaremos las vistas tipadas y la composición con detalle. Por ahora basta con reconocer **qué archivo participa** y **qué recibe el navegador**.

Una decisión que el alumno debe poder explicar.

Si queremos devolver los productos como datos para otra aplicación, un resultado del estilo `Ok(productos)` podría ser apropiado. Si pretendemos construir una página HTML para una persona, `View(productos)` permite entregar los datos a una vista. En este primer ejercicio utilizamos `View()` sin modelo porque solo queremos observar la página y el recorrido HTTP; `Producto` y `View(productos)` se incorporarán en el capítulo 1.2.

Antes de continuar, predice tres situaciones: ¿la llamada `return View()` ejecuta C# dentro del navegador? ¿Qué archivo habría que cambiar para modificar el texto del encabezado? ¿Tendría sentido editar el controlador para cambiar únicamente el contenido de un párrafo? Razona tu respuesta sin modificar todavía el proyecto.

El navegador **no ejecuta** `HomeController` ni la sintaxis Razor. El servidor genera HTML y el cliente representa ese resultado. Si un cambio afecta al texto visible, generalmente buscaremos la vista; si afecta a la selección de datos o a la decisión del resultado HTTP, examinaremos la acción. Esta distinción orientará los pasos siguientes y, sobre todo, evitará modificar archivos al azar.

El siguiente problema es averiguar **cómo llega la petición del navegador al servidor y cómo distinguimos una respuesta HTML de la hoja de estilos que se descarga después**. Para responderlo estudiaremos el intercambio HTTP real antes de profundizar en el pipeline y las rutas.

El recorrido HTTP en el navegador

Qué ocurre al escribir una URL.

Cuando introducimos `http://localhost:5091/` en el navegador, este actúa como **cliente HTTP**. En nuestro laboratorio, `localhost` identifica el propio ordenador, el puerto `5091` selecciona el proceso servidor y `/` es la ruta solicitada. En un sitio público intervendrían además resolución de nombres, red y, normalmente, HTTPS. Estos elementos no alteran la lógica básica de cliente, petición y respuesta.

Una petición HTTP contiene, como mínimo en términos conceptuales:

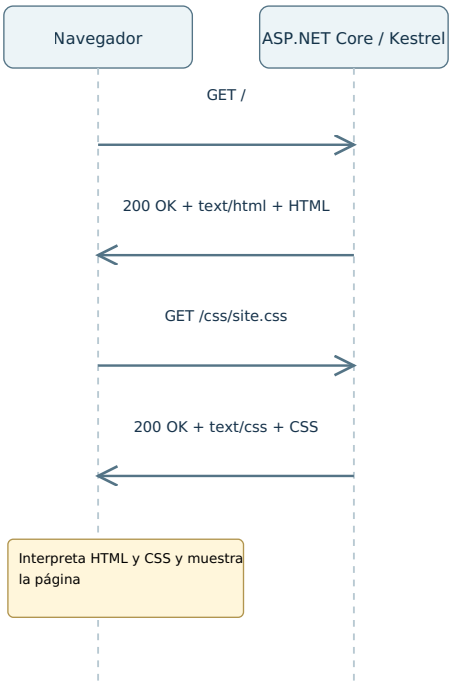
- **Método.** Indica el tipo de operación solicitada. En este punto utilizaremos `GET` para consultar una página. El día 2 utilizaremos `POST` para enviar formularios.
- **Destino.** La URL y el camino o *path* que el servidor examina, por ejemplo `/`, `/Home/Index` o `/css/site.css`.
- **Cabeceras.** Metadatos como los formatos que acepta el cliente. No son el cuerpo de la página.
- **Cuerpo.** Datos opcionales asociados a la petición; normalmente una navegación GET sencilla no necesita uno.

La respuesta también tiene partes diferenciadas:

- **Código de estado.** `200 OK` significa que la solicitud se atendió satisfactoriamente; `404 Not Found` significa que el servidor no encontró un recurso o ruta compatible.
- **Cabeceras de respuesta.** Por ejemplo, `Content-Type` indica el tipo del contenido devuelto.
- **Cuerpo de respuesta.** En nuestro ejemplo, el documento HTML de la vista Razor.

Una carga de página puede producir **varias peticiones HTTP**. El navegador primero recibe el HTML; si este enlaza una hoja de estilo con `<link rel="stylesheet" ...>`, solicita además el archivo CSS. Por eso, cuando abrimos la pestaña *Red/Network* de las herramientas de desarrollo, no suele aparecer una sola fila.

La siguiente figura representa el intercambio principal; la solicitud posterior del CSS se muestra como una segunda petición independiente:



Importante: el navegador **no ejecuta C# ni interpreta código Razor**. Recibe HTML y otros recursos web. Las expresiones Razor se procesan en el servidor al generar la respuesta de esta aplicación MVC. Cuando más adelante estudiemos Blazor, distinguiremos explícitamente su modelo de componentes y su interactividad.

Observar HTTP en lugar de imaginarlo.

La explicación del recorrido anterior se puede comprobar de tres maneras que se complementan:

En el navegador, la página visual muestra el contenido procesado. Al abrir las herramientas de desarrollo (F12) y la pestaña *Red/Network*, una navegación permite observar el documento y sus recursos relacionados. Seleccionando la fila de tipo *document* comprobamos el método GET, la URL y el código 200.

En las cabeceras, la respuesta de un documento HTML contiene `Content-Type: text/html`, posiblemente acompañado de una indicación de codificación. Al pedir el CSS observaríamos en cambio `text/css`. Las cabeceras describen la respuesta y no deben confundirse con los elementos `<h1>` o `<p>`, que forman parte del HTML.

En una herramienta HTTP como `curl`, podemos ver la respuesta sin que intervenga la representación visual del navegador:

```
curl -i http://localhost:5091/
curl -i http://localhost:5091/ruta-inexistente
```

La primera petición debe recibir `200` y una página HTML. La segunda, al no corresponder con una acción de la aplicación, debe recibir `404`. El HTML exacto puede incluir detalles adicionales de framework, pero lo que comprobaremos es el estado HTTP y la marca visible de la vista.

Durante la práctica activaremos dos pequeñas modificaciones y compararemos sus efectos:

- Añadir la cabecera `X-Curso-Paso: 1.1.4` desde la acción MVC **modifica los metadatos HTTP de la respuesta**, no el contenido visual.
- Descomentar un bloque HTML en `Index.cshtml` **modifica el cuerpo HTML**, no necesita añadir una nueva ruta ni cambiar el código HTTP de éxito.

Esta distinción resulta fundamental cuando, más adelante, debamos depurar formularios y llamadas a API: no todos los cambios en el servidor se observan del mismo modo en la pantalla.

Analizar HTTP como un contrato observable, no como una caja negra.

Hasta ahora hemos seguido una petición desde la barra de direcciones hasta una acción MVC. Para comprender cómo depurar aplicaciones web necesitamos observar el intercambio de datos con mayor precisión. HTTP tiene una estructura definida: una solicitud contiene un método, un objetivo o ruta, una versión de protocolo, cabeceras y, cuando corresponde, un cuerpo. La respuesta aporta un código de estado, cabeceras y posiblemente un cuerpo. En el laboratorio inicial nos interesa especialmente una petición GET, que no necesita enviar datos de formulario.

Una representación didáctica de la solicitud es:

```
GET /Home/Index HTTP/1.1
Host: localhost:5091
Accept: text/html
```

La línea inicial indica **qué operación** se solicita y sobre **qué ruta**. La cabecera `Host` identifica el host al que se dirige la petición. `Accept` comunica una preferencia sobre formatos de respuesta, aunque no obliga por sí sola a una aplicación a producir cualquiera de ellos. El ejemplo muestra un mensaje representativo; los navegadores reales incluyen más cabeceras y pueden utilizar HTTP/2 u HTTP/3.

La respuesta de una vista MVC puede conceptualizarse así:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

<!DOCTYPE html>
<html lang="es">
  ...
  <h1>1.1 | Primer recorrido MVC</h1>
</html>
```

Si el servidor devuelve `200`, la solicitud fue atendida satisfactoriamente según ese resultado. `Content-Type` explica cómo debe interpretar el cliente el contenido. En nuestra aplicación, el servidor entrega HTML generado por Razor: el navegador no ejecuta el archivo `Index.cshtml` directamente.

Conviene distinguir tres conceptos que pueden parecer equivalentes pero no lo son: el **código de estado** de la respuesta, el **tipo de contenido** indicado por sus cabeceras y el **contenido concreto** que vemos en la pantalla. El servidor puede devolver 200 con un texto que dice «error» si la acción así lo programa; el texto visible no determina automáticamente el código. Igualmente, un documento con extensión `.css` servido por el middleware estático debe identificarse mediante el tipo de contenido adecuado, no porque el navegador adivine su estructura a partir del nombre.

El papel de los códigos 200, 404 y errores de conexión.

En el punto 1.1 hemos preparado una comprobación de `/ruta-inexistente`. Esta URL no coincide con un endpoint configurado y, por eso, recibimos 404. Sin embargo, un 404 demuestra que **sí hubo comunicación con el servidor**: se ha recibido una respuesta HTTP que declara que no se encontró el recurso solicitado.

Muy diferente es una conexión rechazada cuando el proceso no está ejecutándose o no escucha en el puerto utilizado. En ese caso no se puede afirmar que hubo un «HTTP 404», pues quizá nunca se recibió una respuesta HTTP. Esta distinción será especialmente útil en M05, cuando comparemos los tres modos de ejecución y despliegue.

También hay que distinguir la ruta de un endpoint de la dirección de un archivo fuente. `/Home/Index` no es una orden al sistema operativo para abrir `Views/Home/Index.cshtml`. Es una dirección que el middleware MVC interpreta y relaciona con un controlador y una acción.

Una comprobación cruzada: por qué el navegador y curl muestran cosas distintas.

Hasta este momento hemos visto cómo acceder a `/` desde el navegador y con `curl -i`. Las dos herramientas consultan el servidor, pero no ofrecen exactamente el mismo punto de vista. El navegador **interpreta** el HTML, aplica CSS y permite seguir enlaces; `curl` **muestra** la respuesta de una solicitud sin componer visualmente la página. Esta distinción es útil cuando el aspecto de una página puede inducirnos a un diagnóstico incorrecto.

Supongamos que el navegador muestra correctamente el título, pero no percibimos ningún cambio después de activar el paso 1.1.4. Eso no demuestra que la modificación haya fallado: el paso añade una **cabecera de respuesta** que normalmente no se representa en la página. Para observarla debemos seleccionar el documento principal en F12 → Red o examinar la respuesta HTTP con `curl -i`. Si la cabecera aparece y el HTML no cambia, hemos confirmado justamente el comportamiento previsto.

La situación inversa se presenta en el paso 1.1.5: al activar un bloque de la vista, cambia el cuerpo HTML, mientras el código de estado puede continuar siendo 200 y las cabeceras de la acción permanecer iguales. Para comprobarlo, no basta con buscar un nuevo nombre de cabecera: debemos inspeccionar el cuerpo de la respuesta o la representación visual.

Método de contraste. Antes de modificar el archivo, identifica qué parte esperas que cambie: URL, estado, cabecera, cuerpo o apariencia. Después elige una observación capaz de distinguir ese efecto. La prueba más útil no es la que imprime más información, sino la que permite confirmar o descartar la hipótesis relacionada con la modificación.

Este pequeño método evita una confusión frecuente: pensar que «no veo nada nuevo en pantalla» significa necesariamente que el código ejecutado es el mismo. La pantalla no representa toda la respuesta HTTP; el navegador también procesa información que permanece invisible para la persona usuaria.

Reconstruir una petición para diagnosticar un fallo.

Una técnica sencilla de diagnóstico consiste en seguir un orden fijo antes de modificar el código:

Primero, confirmar el proceso. ¿Hay realmente una aplicación escuchando en el puerto indicado? Si falla la conexión, no tiene sentido inspeccionar el HTML todavía. Revisar la terminal de `dotnet run` proporciona el punto de partida.

Segundo, identificar la URL exacta. `/`, `/Home/Index` y `/css/site.css` no tienen necesariamente el mismo recorrido. La ruta no es solo el nombre mostrado en la barra de direcciones; es parte de la información que MVC y otros middlewares utilizan para decidir el resultado.

Tercero, leer el código de estado. Un 200 de `/` permite continuar al análisis del cuerpo. Un 404 de `/ruta-inexistente` es correcto en esta práctica, pues demuestra que el endpoint no existe. Un 404 de `/css/site.css`, en cambio, señala un fallo para un recurso que el proyecto sí incluye.

Cuarto, observar el tipo de contenido. Si el navegador recibe una página HTML donde esperaba CSS, no debemos concluir que la hoja funciona solo porque la URL devolvió 200. El tipo `text/css` en la respuesta del archivo es una comprobación más precisa.

Quinto, observar el contenido. Una respuesta HTML puede tener código 200, pero no contener el texto que esperamos. La comprobación entonces debe revisar qué vista se ha ejecutado, si se guardaron los cambios y si el proceso utiliza la versión correcta de los archivos.

Sexto, conservar una evidencia mínima. La URL solicitada, el código de estado, una cabecera relevante y una frase del cuerpo suelen bastar para reproducir el hallazgo. No es necesario copiar largas trazas en el material del alumno. Esto también enseña a comunicar incidencias con precisión en una futura práctica profesional.

Un modelo mental de la web orientado a peticiones.

En una aplicación de escritorio tradicional, el desarrollador suele pensar primero en ventanas y eventos de usuario. En una aplicación web resulta especialmente útil empezar pensando en **peticiones**. El usuario introduce una dirección o pulsa un enlace, el cliente solicita un recurso o endpoint y el servidor procesa la solicitud antes de devolver un resultado.

La interfaz puede parecer una única página, pero el navegador suele efectuar múltiples solicitudes. Primero pide el documento HTML; después puede solicitar hojas CSS, imágenes u otros recursos referenciados en ese documento. En nuestro proyecto inicial, `/` devuelve un documento construido mediante MVC y `/css/site.css` entrega un archivo estático. Ambas respuestas proceden del mismo servidor Kestrel, pero su recorrido interno puede ser distinto.

Este hecho explica por qué la inspección de F12 → Red es más informativa que fijarse solo en el resultado visual. La pantalla puede mostrar un encabezado y unos párrafos, mientras el panel de red registra más de una solicitud. El alumno debe ser capaz de identificar cuál corresponde al documento principal y cuál a los recursos que lo complementan.

La solicitud del documento y la solicitud de CSS.

Podemos comparar dos ejemplos conceptuales:

```
GET / HTTP/1.1
Host: localhost:5091
Accept: text/html
```

y

```
GET /css/site.css HTTP/1.1
Host: localhost:5091
Accept: text/css
```

No estamos afirmando que un navegador envíe exactamente esas tres líneas. Son representaciones simplificadas que permiten estudiar el método, la ruta y una posible preferencia de contenido. La comparación muestra algo estable: en el primer caso pedimos `/` y en el segundo pedimos `/css/site.css`.

El servidor debe decidir qué componente atiende cada petición. En nuestra configuración, el documento se obtiene desde una acción MVC y el archivo CSS desde el middleware de recursos estáticos. Por eso una cabecera añadida exclusivamente dentro de la acción Home no tiene por qué aparecer en la respuesta de `site.css`.

Qué aporta el método HTTP.

El método `GET` indica una operación de recuperación de representación de un recurso. En un desarrollo web real existen otros métodos, pero en este primer punto no necesitamos introducir formularios ni operaciones que cambien datos. Las solicitudes de la práctica son GET y el objetivo es aprender a seguirlas de principio a fin.

No debemos asociar GET únicamente a «abrir una página HTML». El mismo método puede solicitar una hoja CSS, un archivo de texto o datos JSON. El tipo de resultado lo decide el servidor y se comunica mediante la respuesta, especialmente a través de su cabecera de contenido.

Leer las herramientas de desarrollo con método.

El panel de red de un navegador contiene mucha información. Para este curso conviene utilizarlo de una forma dirigida, evitando que el alumnado pierda tiempo entre columnas que todavía no necesita interpretar.

Primero abrimos las herramientas de desarrollo y seleccionamos la pestaña **Red / Network**. Después recargamos la página para capturar las solicitudes desde el comienzo. Identificamos el documento principal y abrimos el detalle de esa solicitud.

En la sección de información general observamos la URL y el código de estado. En las cabeceras observamos `Content-Type` y, cuando activamos el paso correspondiente, `X-Curso-Paso`. En el cuerpo de respuesta buscamos el encabezado de nuestra vista y más adelante el bloque `marca-paso-1-1-5`.

Después repetimos el procedimiento para `/css/site.css`. Debemos observar un recurso diferente, con tipo de contenido CSS. La cabecera que se añadió dentro del controlador Home no se espera en esa solicitud, pues corresponde a otro mecanismo de atención.

Qué información no necesitamos memorizar.

El navegador puede incluir múltiples cabeceras y datos de temporización, almacenamiento en caché y seguridad. No es necesario explicarlos todos para comprender el primer recorrido MVC. Nuestro criterio docente es seleccionar un conjunto pequeño de evidencias que responda a preguntas concretas: ¿qué se pidió?, ¿quién lo atendió?, ¿qué respondió el servidor? y ¿qué parte del contenido cambió después de editar el código?

Una vez que el alumno domina esta secuencia, puede emplearla para diagnosticar problemas más avanzados sin que tengamos que dedicar horas a todas las opciones de la herramienta.

Pequeño caso completo de diagnóstico.

Imaginemos que una persona informa: «La aplicación está rota; no se ve como ayer». Esa descripción todavía no permite decidir qué archivo corregir. Podemos convertirla en una secuencia de preguntas útiles.

¿Se puede abrir `/`? Si no se establece conexión, revisamos primero la ejecución de Kestrel y el puerto. Si se recibe 404, investigamos el enrutamiento o la dirección. Si se recibe 200, ya tenemos una evidencia positiva sobre la página inicial.

¿El HTML contiene el encabezado de la práctica? Si falta, revisamos la vista que se está ejecutando, el proyecto lanzado y si los cambios se han guardado. Si aparece, sabemos que la acción y el resultado de vista han producido al menos una parte reconocible de la página.

¿Se recibe el CSS? Si `/css/site.css` responde 404, investigamos su ubicación y `UseStaticFiles`. Si responde 200 con contenido CSS pero el aspecto visual sigue siendo incorrecto, el problema puede depender de las reglas CSS o de su aplicación en el documento.

¿Se observa la cabecera del ejercicio? Si activamos `X-Curso-Paso` y no aparece en el documento HTML, comprobamos que la línea realmente está descomentada en la versión de `HomeController` que ejecuta el servidor. No buscamos esa cabecera en la respuesta CSS.

¿Está el texto añadido en la vista? Si el `section` de 1.1.5 continúa dentro de un comentario Razor, no producirá contenido. Si está activo y el servidor representa la versión correcta, debe aparecer en el HTML.

Esta secuencia enseña a separar causas. A menudo «la página está rota» equivale a varios problemas diferentes, y cada uno exige observar una pieza concreta antes de modificarla.

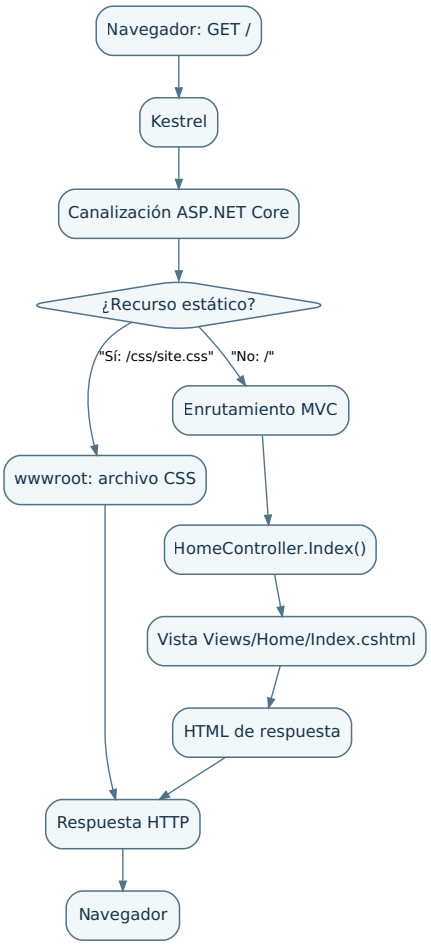
Servidor, Kestrel y pipeline

Servidor, aplicación, framework y navegador: cuatro responsabilidades.

Conviene separar cuatro participantes que a menudo se confunden:

1. **Kestrel**, el servidor web de ASP.NET Core, escucha en una dirección y puerto y recibe conexiones HTTP.
2. **La canalización de ASP.NET Core**, compuesta por middleware y selección de endpoints, procesa la solicitud y decide qué código puede atenderla.
3. **El código de la aplicación MVC**, con controladores y vistas, selecciona la respuesta y genera su contenido. En nuestra primera aplicación, el controlador `HomeController` devuelve `Views/Home/Index.cshtml`.
4. **El navegador** recibe y representa el documento final. Su trabajo visible no implica que conozca las clases del servidor.

Un middleware es un componente que participa en el procesamiento de una petición y puede atenderla directamente o dejar que continúe. En nuestro `Program.cs`, `UseStaticFiles` permite devolver archivos como la hoja de estilos, mientras que `UseRouting` y el mapeo de ruta convencional intervienen en la selección de la acción MVC. No necesitamos dominar todavía cada middleware: basta con reconocer que el servidor no salta mágicamente de la URL a una función cualquiera.



La figura resume el caso del curso. Una aplicación ASP.NET Core más grande puede utilizar autenticación, sesiones, manejo de excepciones u otros middleware; por ahora no los incorporamos.

Entender el servidor, la aplicación y el proceso .NET.

En una explicación informal es frecuente decir que «MVC es el servidor». La idea ayuda a empezar, pero oculta responsabilidades distintas. **Kestrel** es el servidor HTTP integrado con ASP.NET Core que escucha conexiones y participa en su procesamiento. **ASP.NET Core** proporciona el pipeline de middleware, el enrutamiento y los servicios web. **MVC** añade la infraestructura de controladores y vistas. **Nuestro proyecto C#** define qué controlador, vista y recursos incluye la aplicación concreta.

Esos elementos colaboran en el mismo proceso del ejercicio. No hay un proceso distinto para cada controlador ni un servidor nuevo por cada vista. Cuando ejecutamos el proyecto, el host inicializa la aplicación y Kestrel empieza a escuchar en la URL indicada. Las solicitudes entrantes atraviesan el pipeline hasta que un componente las atiende o el procesamiento finaliza.

También conviene separar la configuración del código fuente del modo de arranque. El comando:

```
dotnet run --no-launch-profile --urls http://localhost:5091
```

inicia una aplicación de desarrollo mediante el SDK y solicita que escuche en la dirección propuesta, sin utilizar un perfil de `launchSettings.json`. La parte `http://localhost:5091` incorpora un esquema (`http`), un host (`localhost`) y un puerto (5091). Dos procesos diferentes no pueden, por regla general, escuchar simultáneamente de la misma forma en el mismo extremo de red. Por eso la práctica pide detener el proceso antes de cambiar entre versiones del proyecto.

El alumno debe comprender además que `localhost` se refiere al propio equipo desde el que se ejecuta el cliente. Abrir una dirección `localhost` en otro dispositivo no implica acceder mágicamente al mismo servidor. Esto resulta relevante al ejecutar prácticas en una máquina virtual o en un contenedor, aunque su configuración se pospone a M05.

El ciclo al reconstruir y volver a ejecutar.

Cuando modificamos un archivo C# del controlador, el proyecto debe recompilarse o utilizar un mecanismo de desarrollo que reconstruya los cambios para que el proceso en ejecución los incorpore. Cuando cambiamos una vista Razor, ciertas herramientas de desarrollo pueden mostrar actualizaciones de otra manera, pero no necesitamos basar la práctica en comportamientos variables del entorno. Para una comprobación reproducible seguiremos una secuencia explícita: **detener → guardar → compilar → iniciar → solicitar**.

Esta secuencia evita falsos diagnósticos. Si el alumno ha modificado el archivo del proyecto 1.1 pero continúa ejecutando la DLL del proyecto 1.2, el navegador no mostrará el cambio esperado. La comprobación correcta comienza por identificar el directorio de trabajo y el proyecto que se ha iniciado.

El pipeline y la idea de middleware.

Un middleware es un componente que participa en el procesamiento de solicitudes y respuestas. Algunos hacen trabajo antes de pasar la solicitud al siguiente; otros pueden intervenir al regresar la respuesta; otros pueden atender la solicitud sin seguir avanzando por el pipeline. En esta aplicación hemos incluido dos piezas fáciles de observar:

```
app.UseStaticFiles();
app.UseRouting();
```

`UseStaticFiles` permite servir archivos estáticos reconocidos desde la raíz web, como `wwwroot/css/site.css`. Cuando se solicita ese CSS, no necesitamos ejecutar `HomeController.Index()` para construir su contenido. Por otro lado, la ruta convencional de MVC conecta direcciones como `/Home/Index` con acciones de controlador.

No debemos representar el pipeline como una lista de carpetas ni creer que cada llamada `Use...` es un controlador. Es el **orden de procesamiento de componentes** que la plataforma configura. La ruta de

archivos y la ruta HTTP son conceptos diferentes: un middleware estático resuelve recursos de archivo publicados, mientras que MVC invoca código del controlador que puede renderizar una vista.

Esto explica por qué la cabecera personalizada añadida dentro de `HomeController.Index()` no aparece automáticamente en `/css/site.css`. Esa instrucción pertenece a una acción concreta, no a todas las respuestas producidas por la aplicación. En M01 todavía no introduciremos middleware propio, porque el objetivo es observar la secuencia; comprender sus responsabilidades será suficiente para evitar confusiones.

Una lectura detallada de Program.cs.

`Program.cs` utiliza la forma de arranque simplificada característica de ASP.NET Core moderno:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseStaticFiles();
app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

No debemos leerlo como si cada línea fuera un paso repetido para cada visitante. La mayor parte de estas instrucciones configura el host y la aplicación al arrancar. Una vez que `Run()` está atendiendo solicitudes, los middlewares y endpoints intervienen cuando llegan peticiones.

`CreateBuilder` prepara la configuración del host y el registro de servicios. `AddControllersWithViews` incorpora servicios necesarios para MVC con vistas. `Build` crea la aplicación configurada. `UseStaticFiles` permite responder a solicitudes de recursos estáticos; `UseRouting` participa en el enrutamiento; `MapControllerRoute` registra el patrón convencional; `Run` inicia el proceso de atención.

¿Dónde está la lista de rutas?.

La ruta convencional describe un patrón que puede relacionar diferentes combinaciones de controlador y acción. No necesitamos declarar una llamada independiente en `Program.cs` por cada método de controlador. Tampoco significa que la aplicación publique automáticamente cualquier método privado: los endpoints MVC se construyen con reglas y metadatos propios de los controladores y sus acciones.

En la primera práctica solo seguimos `HomeController.Index`. Más adelante veremos cómo `ProductosController.Index` encaja en el mismo patrón y cómo un atributo permite definir una dirección distinta.

¿Por qué no modificar aún el orden de middleware?.

La práctica no pretende enseñar todas las variantes del pipeline ni construir middleware propio. El orden puede ser relevante para el comportamiento de ASP.NET Core, y alterar varias instrucciones a la vez haría más difícil identificar causas. Conservamos la configuración proporcionada y nos concentramos en una pregunta verificable: **qué resultado devuelve cada URL en este proyecto**.

Rutas, controladores y Razor

Cómo se conecta la URL con nuestro primer controlador.

Observemos los archivos reales de `M01_MVC/PROYECTO/1.1/CursoWeb.Mvc`. El arranque está definido en `Program.cs`:

La configuración que necesitamos ya está en `Program.cs`: `CreateBuilder(args)` prepara la aplicación, `AddControllersWithViews()` registra MVC, `UseStaticFiles()` atiende recursos públicos y `MapControllerRoute(...)` define la ruta convencional. Seguiremos ahora su ejecución en lugar de repetir todo el archivo.

Examinemos cada parte, sin adelantar toda la materia de routing del punto 1.4:

`CreateBuilder(args)` prepara un constructor de aplicación con la configuración y los servicios comunes de ASP.NET Core. Aquí no se crea una vista ni se responde todavía a ninguna petición.

`AddControllersWithViews()` registra los servicios que necesitan los controladores MVC y su infraestructura de vistas. El verbo *Add* se refiere a configurar servicios; no significa que se esté invocando una acción concreta. Los servicios se configuran al arrancar la aplicación.

`Build()` construye la aplicación que utilizará dicha configuración.

`UseStaticFiles()` habilita la atención de archivos estáticos ubicados, en nuestro ejemplo, bajo `wwwroot`. Cuando una página hace referencia a `/css/site.css`, ese recurso no necesita pasar por `HomeController`.

`UseRouting()` participa en la identificación del endpoint que corresponde a una petición. La aplicación registra después una ruta convencional mediante `MapControllerRoute`. El patrón `{controller=Home}/{action=Index}/{id?}` dice que, si una URL no especifica controlador ni acción, se usan `Home` e `Index` como valores predeterminados. `id?` representa un segmento opcional que utilizaremos posteriormente.

`Run()` inicia el host y mantiene la aplicación atendiendo peticiones. Es distinto de la ejecución de una acción: el arranque ocurre una vez por proceso; las peticiones pueden atenderse muchas veces.

Ahora observemos el controlador que ya existe:

```
using Microsoft.AspNetCore.Mvc;

namespace CursoWeb.Mvc.Controllers;

public class HomeController : Controller
{
    [HttpGet]
    public IActionResult Index()
    {
        // Response.Headers["X-Curso-Paso"] = "1.1.4";
        return View();
    }
}
```

La clase deriva de `Controller` para disponer de la infraestructura y de métodos auxiliares de MVC. `[HttpGet]` limita esta acción a solicitudes HTTP GET. La acción `Index` devuelve un `IActionResult`: en este caso, `View()` crea un resultado que posteriormente localizará y representará la vista. **No devuelve una cadena HTML escrita manualmente** ni significa que el navegador pueda ejecutar la vista.

Por convención, `return View();` en `HomeController.Index` selecciona `Views/Home/Index.cshtml`. Esa convención relaciona nombres de controlador, acción y archivo. La estudiaremos con detalle en los puntos 1.3, 1.4 y 1.5.

La vista Razor y el HTML que llega al cliente.

El contenido visible de la vista inicial es:

```
@{
    ViewData["Title"] = "Primer recorrido MVC";
}

<h1>1.1 | Primer recorrido MVC</h1>
<p>Esta página HTML ha sido generada en el servidor mediante una acción MVC.</p>
<p id="resultado-inicial">Respuesta HTTP correcta: aplicación MVC disponible.</p>
```

Una vista Razor permite combinar HTML con instrucciones Razor del lado del servidor. En este primer ejemplo, la expresión `ViewData["Title"]` proporciona un título que utiliza el diseño general de la página. No necesitamos todavía dominar el paso de modelos tipados; se abordará en el punto 1.5.

El archivo `Views/Shared/_Layout.cshtml` dispone el esqueleto común del documento: `<html>`, `<head>`, navegación, área principal y pie. Su instrucción `@RenderBody()` inserta el contenido principal de la vista. El archivo `Views/_ViewStart.cshtml` indica que las vistas utilizarán ese layout. Como resultado, la respuesta HTML completa tiene más elementos de los que aparecen directamente en `Index.cshtml`.

¿Por qué no colocamos todos los elementos en una sola vista? Porque compartir un layout evita repetir cabeceras, navegación y estilos en muchas páginas. Por el momento basta con **reconocer** esta composición; en el punto 1.5 aprenderemos a modificar layouts y crear vistas parciales.

La hoja de estilos `wwwroot/css/site.css` se sirve por separado. Aunque la apariencia cambie con el CSS, la acción del controlador no necesita conocer detalles del navegador.

Controlador y vista: dos puntos distintos donde observar resultados.

Para aislar responsabilidades hemos preparado dos fragmentos comentados en el proyecto. El primero modifica la respuesta desde C#:

```
Response.Headers["X-Curso-Paso"] = "1.1.4";
```

El segundo cambia el HTML Razor:

```
<section id="marca-paso-1-1-5">
    <h2>Contenido añadido durante la práctica</h2>
    <p>El navegador ha recibido una nueva versión de la vista Razor.</p>
</section>
```

El primero no altera necesariamente los elementos visibles de la página: modifica los metadatos de la respuesta. El segundo modifica el contenido que Razor representa. Es posible comprobar ambos por separado. Esta forma de trabajar es pedagógicamente valiosa porque evita atribuir todo cambio a «la página» sin identificar qué parte lo ha generado.

Un navegador puede mostrar exactamente el mismo encabezado visual con o sin la cabecera `X-Curso-Paso`. Una inspección solo visual podría dar la impresión de que el controlador no ha cambiado. Por el contrario, el elemento `marca-paso-1-1-5` aparecerá en el HTML representado después de activar el fragmento Razor, aunque el código de controlador permanezca intacto.

También hay que interpretar correctamente `Response.Headers`. Se trata de un objeto asociado a la respuesta de la petición actual; escribir una cabecera desde una acción no equivale a cambiar las cabeceras de todos los recursos del sitio. No utilizamos aquí cookies, autenticación ni cabeceras de seguridad: los nombres `X-Curso-Paso` y `X-Curso-Reto` son estrictamente marcadores docentes.

El reto como prueba de comprensión.

El reto 1.1 vuelve a separar dos intervenciones: añadir la cabecera `X-Curso-Reto: HTTP-y-HTML` desde el controlador y representar un párrafo `reto-1-1` desde la vista. Si ambos resultados están presentes en la

misma respuesta, no debemos deducir que se generaron de la misma manera. El controlador decidió añadir una cabecera y la vista decidió producir marcado adicional.

La comprobación más útil no es únicamente «aparece el párrafo». Debemos solicitar también el archivo CSS y mostrar que **la cabecera del controlador no aparece allí**. La diferencia confirma qué componente atendió cada tipo de solicitud.

Observar el HTML generado sin confundirlo con Razor.

El archivo `Views/Home/Index.cshtml` contiene marcado HTML y expresiones Razor. Durante el primer recorrido, la acción devuelve `View()` y el motor localiza esa plantilla mediante convenciones. La respuesta que llega al navegador es HTML, no una copia que el cliente deba interpretar como fuente C#.

El archivo principal incluye un encabezado:

```
<h1>1.1 | Primer recorrido MVC</h1>
```

Esta línea no contiene una expresión dinámica, así que el resultado HTML conserva un elemento parecido. Sin embargo, `@{ ViewData["Title"] = "Primer recorrido MVC"; }` es una instrucción Razor que establece datos de presentación y no aparece literalmente en el HTML final.

Esta diferencia puede parecer poco importante en una página sencilla, pero se volverá esencial en 1.2, donde la vista representa productos a partir de una colección. Si el alumno cree que Razor se ejecuta en el navegador, no podrá explicar correctamente por qué la respuesta HTTP contiene los nombres de los productos y no un bucle de C#.

Contenido de una respuesta frente a apariencia.

El HTML define una estructura documental; las hojas CSS controlan aspectos de presentación. Una respuesta HTML correcta no garantiza que la apariencia sea la esperada. Por ejemplo, si el archivo CSS devuelve 404, los títulos y párrafos todavía pueden aparecer, pero faltarán estilos.

Este es otro motivo para inspeccionar tanto el documento principal como sus recursos. En la práctica no se pide diseñar una página compleja: se pide reconocer **qué parte del sistema es responsable de cada tipo de salida**.

Cabeceras personalizadas: datos de control fuera del HTML.

El paso 1.1.4 añade una cabecera a la respuesta:

```
Response.Headers["X-Curso-Paso"] = "1.1.4";
```

La instrucción modifica la colección de cabeceras HTTP de la respuesta de la acción. El navegador puede mostrar ese dato en su panel de red, pero no tiene por qué dibujar automáticamente el texto en la página.

Esto permite distinguir **metadatos de transporte** y **cuerpo de la respuesta**. Una cabecera y un párrafo HTML pueden contener la misma cadena, pero eso no significa que tengan el mismo papel. El cuerpo HTML será interpretado como documento; la cabecera forma parte de la información que acompaña a la respuesta.

Por qué el lugar del código importa.

Si escribimos esta instrucción dentro de `HomeController.Index()`, afecta a las respuestas preparadas al ejecutar esa acción. Si la aplicación atendiera otra URL mediante un archivo estático, esa instrucción no se ejecutaría allí. El reto reproduce la diferencia con una segunda cabecera de diagnóstico y un párrafo Razor.

No introducimos cabeceras de autenticación, controles de caché o seguridad porque exigirían explicar conceptos que no pertenecen a este ejercicio. Los nombres personalizados son marcadores sin significado operativo fuera del laboratorio.

Comprobación, errores y continuidad

Confusiones frecuentes y cómo resolverlas.

«Si abro localhost en el navegador, estoy abriendo un archivo HTML». En nuestro ejemplo no: el navegador solicita la ruta `/` a un proceso web que escucha en un puerto. La vista se genera como respuesta de una acción MVC. Si apagamos el proceso, esa dirección deja de responder.

«El archivo Index.cshtml se descarga y Razor se ejecuta en el ordenador del alumno». No. Razor se evalúa en el servidor. El resultado enviado al navegador es HTML. La extensión `.cshtml` identifica el archivo fuente del proyecto, no el formato de respuesta entregado al usuario.

«View() devuelve directamente una página HTML». Más exactamente, `View()` crea un resultado MVC de vista. La infraestructura ejecuta ese resultado, localiza la vista y genera la respuesta HTML. La distinción explica por qué la acción puede permanecer compacta.

«Una ruta 404 implica que toda la aplicación ha fallado». No necesariamente. La aplicación puede estar funcionando perfectamente y no tener un endpoint compatible con la URL solicitada. En nuestra primera aplicación es normal que `/ruta-inexistente` devuelva 404.

«Si no veo cambios en la pantalla, el código del controlador no se ha ejecutado». No siempre. Una acción puede añadir una cabecera HTTP que solo se aprecia en la respuesta o en las herramientas de desarrollo.

«ASP.NET Core MVC y ASP.NET Core Web API son frameworks totalmente aislados». Comparten la misma plataforma web y numerosos servicios. Usan convenciones y tipos de respuesta con objetivos diferentes.

Relación directa con la práctica.

La práctica `P1.1` reproduce exactamente los hechos descritos aquí: creación de una plantilla MVC, apertura del proyecto inicial del curso, arranque de Kestrel, navegación por `/`, lectura del método y estado de la petición, comparación con una ruta 404, activación de una cabecera y activación de un bloque HTML Razor.

Al finalizar se habrá demostrado un flujo real, no solo memorizado un diagrama. El conocimiento adquirido nos permitirá desarrollar en **1.2** las responsabilidades de Modelo, Vista y Controlador; en **1.3**, la estructura del proyecto; en **1.4**, rutas y acciones; y en **1.5**, vistas Razor más elaboradas.

Primeras pruebas de regresión: cambiar una pieza sin romper otra.

Una modificación puede funcionar localmente y provocar una regresión en otro recurso. Por ejemplo, añadir una cabecera al controlador puede coincidir con un fallo accidental si el alumno modificó `Program.cs` y quitó `UseStaticFiles()`. Una prueba solo sobre `/` no detectaría la ausencia del CSS.

Por eso comprobamos varios resultados aunque el ejercicio parezca pequeño:

Comprobación	Qué confirma	Qué no demuestra
`GET /` devuelve 200	La entrada principal responde	Que se aplicó el CSS correctamente
`GET /Home/Index` devuelve 200	La ruta convencional alternativa funciona	Que todas las rutas MVC funcionan

Comprobación	Qué confirma	Qué no demuestra
`GET /ruta-inexistente` devuelve 404	No se ha publicado ese endpoint	Que toda la configuración de errores de producción es correcta
`GET /css/site.css` devuelve 200 y CSS	El recurso estático básico se sirve	Que el navegador utiliza cada regla CSS
Aparece `X-Curso-Paso`	La acción activada añadió esa cabecera	Que todas las respuestas llevan esa cabecera
Aparece `marca-paso-1-1-5`	Se generó la sección HTML del paso	Que el diseño visual es adecuado

Estas pruebas son pequeñas, pero ilustran un principio general: **verificar el resultado específico de un cambio y el comportamiento previo que no debería alterarse**. El alumno ya conoce el concepto de prueba desde C#, de modo que puede reconocer su aplicación a un sistema web sin estudiar todavía un marco de automatización de navegador.

Al trabajar con un servidor HTTP real, la comprobación cubre más que una llamada aislada al método C#: intervienen el host, el enrutamiento, el resultado de vista y la respuesta. La inspección visual con F12 complementa estas comprobaciones, no las sustituye.

Preguntas de razonamiento con respuestas argumentadas.

¿Por qué `/` y `/Home/Index` muestran la misma página? Porque la ruta convencional utiliza Home e Index como valores predeterminados cuando se omiten los segmentos. No son necesariamente dos copias de HTML: ambas direcciones seleccionan la misma acción dentro de esta configuración.

¿Qué ocurriría si `Index()` devolviera `Content("Hola", "text/plain")` en vez de `View()`? La acción seguiría siendo accesible, pero la respuesta ya no contendría la vista Razor ni su layout. Esta alternativa muestra que el tipo de resultado es una decisión de la acción; no necesitamos activarla ahora, pues haremos la comparación de manera real en 1.4.

¿Por qué la respuesta CSS no incorpora la cabecera escrita en HomeController? Porque el recurso se sirve por el mecanismo de archivos estáticos y no mediante la acción de Home. La instrucción de cabecera está dentro de un método concreto, no en un middleware global.

¿Qué demuestra un 404? Demuestra que se recibió una respuesta HTTP indicando que el recurso o endpoint no fue encontrado. Para diagnosticar la causa hay que identificar la URL y qué componente podría atenderla.

¿Qué ventajas tiene conservar el estado inicial del proyecto con bloques comentados? Podemos ejecutar un estado conocido, activar una única variable y comprobar el cambio atribuible a ella. Si modificamos muchas piezas al mismo tiempo, resulta más difícil razonar sobre qué produjo un resultado.

¿Tiene sentido medir solamente cuánto tarda en arrancar el servidor para afirmar que la práctica está bien? No. La rapidez de arranque no demuestra que se comprendan las responsabilidades, que el alumno pueda seguir una petición o que sepa reproducir un diagnóstico HTTP.

Conexión con lo que aprenderemos después.

La primera práctica se concentra deliberadamente en una única página. Sin embargo, el recorrido que ya conocemos —URL → middleware → acción → resultado → contenido y cabeceras HTTP— seguirá vigente en los demás capítulos. En 1.2 añadiremos un modelo real de producto y aprenderemos a entregarlo a una vista; en 1.3 localizaremos archivos y configuración; en 1.4 añadiremos acciones y códigos de estado; y en 1.5 reutilizaremos marcado Razor.

Cuando trabajemos con Blazor, seguiremos necesitando comprender qué es una petición, qué aporta el servidor y qué renderiza el navegador, aunque la composición de la interfaz y la interactividad tengan otro modelo. El objetivo de esta profundización no es memorizar cabeceras: es construir una **cadena de explicaciones comprobables** y aprender a separar lo observado de nuestras suposiciones.

Tres estrategias incorrectas de depuración.

Estrategia incorrecta 1: modificar el CSS porque una URL devuelve 404. El CSS no determina por sí solo qué endpoints existen. Antes de editar estilos debemos comprobar qué URL se solicitó y si corresponde a un controlador o archivo estático.

Estrategia incorrecta 2: modificar el controlador cuando falta un elemento HTML comentado. Si la acción ya devuelve correctamente una vista, puede que el problema sea simplemente que no se haya activado el bloque Razor del paso 1.1.5. Hay que observar el archivo responsable de la representación.

Estrategia incorrecta 3: interpretar un error de conexión como HTTP 404. Un error de conexión puede significar que no hay un proceso escuchando en la dirección indicada. Un 404 es una respuesta del servidor; distinguirlos evita investigar controladores cuando todavía no se ha iniciado correctamente la aplicación.

La técnica correcta consiste en formular una hipótesis y comprobarla con una observación concreta. El alumnado debe aprender a responder con datos y no con conjeturas: URL, método, estado, tipo de contenido y fragmento de resultado.

Construir una explicación técnica reproducible.

Al terminar la práctica, el alumno debería poder describir la aplicación sin depender de que otra persona conozca su equipo. Una descripción útil incluye la carpeta de proyecto, el comando de ejecución, la URL observada, el resultado esperado y el archivo donde está el cambio.

Por ejemplo: «En `PROYECTO/1.1/CursoWeb.Mvc`, iniciando la aplicación con `dotnet run --no-launch-profile --urls http://localhost:5091`, la dirección `/` devuelve 200 y representa la vista `Home/Index`. Al activar la línea `X-Curso-Paso` en `HomeController`, esa cabecera aparece en la respuesta del documento principal, pero no en la solicitud del CSS».

Esta explicación relaciona código, configuración y observación. También permite que otra persona reproduzca el ejercicio y llegue a la misma observación.

Una afirmación como «lo probé y funciona» es mucho menos útil, porque no deja claro qué funcionalidad se verificó ni cuáles fueron las condiciones de la ejecución.

Cierre razonado de la primera etapa.

El propósito del capítulo no es enseñar todas las posibilidades de ASP.NET Core, sino construir un recorrido completo y fiable sobre el que se apoyarán los demás contenidos. Hemos identificado el servidor HTTP, el pipeline de ASP.NET Core, los controladores MVC, las vistas Razor y los recursos estáticos, y hemos observado sus respuestas.

El valor de trabajar con una única página inicial es que permite introducir un cambio cada vez. Primero añadimos metadatos HTTP y después un fragmento HTML. Esa separación convierte la práctica en un experimento reproducible: sabemos qué archivo modificamos, qué evidencia esperamos y qué partes de la aplicación deberían permanecer estables.

Cuando en 1.2 aparezca un modelo de producto, no tendremos que reaprender qué es una solicitud ni qué significa una vista. La nueva pregunta será qué datos recibe esa vista y cómo los utiliza. El método de trabajo seguirá siendo el mismo: **razonar, activar un bloque, ejecutar, observar y comprobar que el comportamiento previo no se ha roto**.

Lecturas de referencia.

- Microsoft Learn, Información general de ASP.NET Core MVC: https://learn.microsoft.com/es-es/aspnet/core/mvc/overview?view=aspnetcore-8.0
- Microsoft Learn, Tutorial de primera aplicación MVC: https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/?view=aspnetcore-8.0
- Microsoft Learn, Fundamentos de middleware: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/?view=aspnetcore-8.0

1.2. Arquitectura Model-View-Controller

Objetivos.

Al finalizar el capítulo, el alumno debe poder identificar sin ambigüedades las responsabilidades de **Model**, **View** y **Controller**; explicar cómo se relacionan durante una petición HTTP; entregar objetos desde una acción a una vista fuertemente tipada; y razonar por qué separar datos, decisión de la respuesta y presentación facilita modificar y probar una aplicación.

En el punto 1.1 observamos una acción `HomeController.Index()` que devolvía una vista. En el 1.2 incorporamos una pieza que aún faltaba: un **modelo de producto**. Usaremos un catálogo con tres objetos en memoria para evitar que el aprendizaje de MVC quede oculto tras una base de datos. El acceso mediante EF Core llegará en M02.

Modelo, vista y controlador: responsabilidades

Situación de partida: la página responde, pero todavía no representa datos del catálogo.

Al cerrar 1.1 sabemos localizar una acción MVC, seguir una petición `GET /` y distinguir la cabecera HTTP del HTML. Ese conocimiento resuelve el transporte de la página, pero no la siguiente necesidad: mostrar tres productos que puedan cambiar sin editar el marcado de cada línea.

Imagina un catálogo cuyo controlador contiene, además de las decisiones sobre la petición, una cadena HTML con los nombres y precios. Si un producto cambia de nombre, debemos revisar la lógica de la petición y el marcado a la vez. Esa dificultad no se corrige creando más rutas, sino separando **qué información existe**, **qué resultado seleccionamos** y **cómo la representamos**.

Vamos a resolver el mismo problema tres veces de manera conceptual: primero identificaremos la mezcla de responsabilidades, luego presentaremos la clase `Producto` y, por último, recorreremos el camino entre `ProductosController.Index()` y `Views/Productos/Index.cshtml`. En la práctica P1.2 no crearemos una base de datos: el controlador construirá una lista en memoria y la entregará a una vista tipada. Esto permite comprobar el efecto de cada pieza sin introducir EF Core antes de tiempo.

La pregunta que guiará el capítulo es concreta: **si cambiamos Cuaderno por Bloc, ¿qué archivo debe cambiar, cuál debe permanecer igual y por qué?**

El problema de una aplicación que mezcla todas las responsabilidades.

Supongamos que queremos mostrar una página con tres productos. Una solución apresurada consiste en escribir en una misma acción el contenido de los productos y el marcado HTML que aparecerá en pantalla:

```
// Ejemplo conceptual que NO debe copiarse al proyecto.
public IActionResult Index()
{
    string html = "<h1>Productos</h1><p>Cuaderno: 4,50 €</p>";
    return Content(html, "text/html");
}
```

Este código es válido como demostración aislada: la acción puede devolver una respuesta HTTP con contenido HTML. Sin embargo, combina dos decisiones diferentes. Por una parte define qué productos existen; por otra decide cómo pintarlos. Si queremos añadir un producto, cambiar un precio, modificar un elemento `<p>` por una tabla o traducir la presentación, tendremos que editar la misma acción. Conforme crece el código, las responsabilidades se superponen.

¿Y si queremos mostrar esos productos además en una ficha de detalle o exportar un listado? Copiar cadenas HTML empeoraría el problema. ¿Y si queremos comprobar que una regla del dominio es correcta sin abrir un navegador? Si toda la lógica está incrustada en el marcado, la comprobación resulta más difícil.

Este es el problema al que responde la separación de responsabilidades: cada elemento se centra en un tipo de decisión y expone una relación clara con los demás. **Separar no significa crear muchas capas empresariales**: para la dimensión didáctica de nuestro catálogo, tres tipos de archivo son suficientes.

Tres responsabilidades distintas.

El patrón arquitectónico **Modelo–Vista–Controlador (MVC)** organiza una aplicación web en tres familias de responsabilidades. «Familia» es una palabra importante: una aplicación real puede tener múltiples modelos, controladores y vistas; no implica una sola clase de cada tipo.

Modelo — representar los datos y las reglas de negocio.

Un modelo expresa los conceptos que maneja la aplicación. Para nuestro ejemplo, un producto tiene identificador, nombre y precio.

```
namespace CursoWeb.Mvc.Models;

public sealed class Producto
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public decimal Precio { get; set; }
}
```

El objeto `Producto` **no sabe cuál es la URL** donde se mostrará, **no conoce el navegador** ni necesita saber si su representación final será una lista HTML, una tabla o un documento JSON. Es un tipo de C# que recoge información de nuestro dominio. Se ha elegido `decimal` para el precio porque se trata de una magnitud monetaria, y los valores se expresarán en el código con el sufijo `m`.

Que una clase esté en la carpeta `Models` no la convierte automáticamente en una base de datos. En el 1.2 crearemos los objetos en una lista en memoria; se pierden al detener el proceso. En el día 2 reutilizaremos estos conocimientos para persistir productos con EF Core.

En aplicaciones mayores podemos encontrar modelos de dominio, modelos de persistencia y **view models** (tipos adaptados a una vista). No son necesariamente la misma clase. En este ejercicio utilizaremos directamente `Producto` como modelo de la vista porque es pequeño y evita introducir una arquitectura adicional. Más adelante podremos distinguir cuándo convendría proyectar datos a un tipo específico para la presentación.

Vista — representar la información para el usuario.

La vista define cómo se transforma la información recibida en una interfaz. En MVC, suele ser un archivo `.cshtml` que combina HTML y sintaxis Razor.

Una vista de productos puede recibir una lista y recorrerla:

```
@model IReadOnlyList<CursoWeb.Mvc.Models.Producto>?

@if (Model is null)
{
    <p>El controlador no ha enviado una lista.</p>
}
else
{
```

```
    <p>Recibidos: @Model.Count productos.</p>
    <ul>
    @foreach (var producto in Model)
    {
        <li>@producto.Nombre</li>
    }
    </ul>
}
```

Este fragmento muestra una vista **tipada**. La directiva `@model` especifica el tipo de datos que espera. La propiedad `Model` permite acceder a los objetos que el controlador le ha proporcionado. Usamos `IReadOnlyList<Producto>?` porque la vista únicamente necesita consultar los datos y durante una etapa inicial el controlador todavía no envía ningún modelo, por lo que puede ser `null`.

El bloque `@if` decide la **presentación** del caso en que no se ha enviado una colección. `@foreach` recorre los productos para crear elementos `<li>`. No introduce productos nuevos, no consulta una base de datos ni toma decisiones empresariales: transforma datos en HTML.

Esto no significa que en una vista esté prohibida toda expresión de C#. Razor permite expresiones y condicionales adecuados para la presentación. Lo que evitaremos es poner en la vista operaciones de persistencia o lógica de negocio compleja.

Controlador — atender la interacción y coordinar la respuesta.

El controlador recibe el resultado del enrutamiento, obtiene o prepara los datos necesarios y elige qué respuesta entregar. En nuestro ejercicio, recibe `GET /Productos`, construye tres productos fijos y los pasa a la vista:

```
using CursoWeb.Mvc.Models;
using Microsoft.AspNetCore.Mvc;

namespace CursoWeb.Mvc.Controllers;

public class ProductosController : Controller
{
    [HttpGet]
    public IActionResult Index()
    {
        var productos = new List<Producto>
        {
            new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
            new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
            new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
        };

        return View(productos);
    }
}
```

La acción es intencionadamente sencilla: no ejecuta consultas ni reglas de negocio complejas. La lista fija nos permite seguir el recorrido sin ruido técnico. En un sistema de producción normalmente delegaría la recuperación de productos en otro servicio, repositorio o mecanismo de acceso a datos, pero esos elementos no son necesarios para comprender este capítulo.

La línea decisiva es `return View(productos);`. Se crea un resultado de vista asociado al modelo que se quiere representar. MVC localiza la vista correspondiente a `ProductosController.Index` y establece su propiedad `Model` con la lista suministrada. En ese momento, la vista ya no necesita inventarse cuáles son los tres productos: los conoce a través de un contrato tipado.

Mapa de responsabilidades.

Pregunta	Componente principal
¿Qué atributos tiene un producto?	Modelo `Producto`
¿Qué respuesta se prepara para `GET /Productos`?	`ProductosController.Index`
¿Qué productos se entregan a la pantalla en esta demostración?	El controlador prepara la lista; en un sistema mayor consultaría un servicio
¿Qué HTML representa la cantidad y los nombres?	`Views/Productos/Index.cshtml`
¿Qué parte solicita y representa el HTML?	Navegador, que **no** forma parte de las tres piezas del patrón

El controlador puede conocer el modelo y seleccionar la vista. La vista suele recibir datos del modelo. **El modelo no necesita depender del controlador ni de la vista**: esa independencia es una de las ventajas más relevantes de MVC.

Ventajas reales de separar responsabilidades.

Modificación localizada. Si queremos presentar precios con dos decimales, cambiaremos el marcado de la vista. Si queremos variar la lista fija, editaremos el código que prepara los datos. La clase de producto puede seguir representando las mismas propiedades en ambos casos. Esto reduce el número de archivos afectados por una modificación.

Comprobación independiente. Podemos verificar que `Producto` almacena identificador, nombre y precio como cualquier otro objeto C#. Podemos comprobar que un controlador entrega una lista y observar mediante HTTP el HTML de la vista. En nuestro laboratorio también podemos verificar los estados intermedios con peticiones reales a Kestrel.

Reutilización. El modelo `Producto` podrá utilizarse en una lista, una vista de detalle y, cuando llegue el momento, en un formulario. No necesitamos duplicar la definición de identificador y precio en cada página.

Colaboración y evolución. Una persona puede modificar el aspecto de una vista mientras otra amplía un controlador, siempre que ambas respeten el contrato de datos. Esta separación no elimina toda dependencia: si cambia el nombre o tipo de una propiedad requerida, habrá que actualizar el código que la consume.

Costes y límites. MVC añade convenciones y archivos. Para una página estática pequeña puede parecer más trabajo que escribir un único HTML. Un proyecto mal organizado puede seguir siendo complejo aunque utilice carpetas llamadas Models, Views y Controllers. MVC **ayuda** a separar responsabilidades; no garantiza automáticamente que el diseño sea correcto.

En el capítulo 1.4 ampliaremos las acciones, parámetros y tipos de resultado; en 1.5 aprenderemos a construir vistas con un diseño reutilizable. En esta introducción no añadiremos patrones Repository, Clean Architecture ni otras capas empresariales que no aporta el ejercicio.

Errores conceptuales frecuentes.

«El modelo es la base de datos». No. El modelo representa conceptos y datos. Puede estar asociado a persistencia o existir solo en memoria. La clase `Producto` de 1.2 no abre ninguna conexión a una base de datos.

«El controlador debe escribir todo el HTML». No es su responsabilidad principal. Devuelve un resultado de vista y proporciona datos. El archivo `.cshtml` define el marcado de presentación.

«La vista no puede contener ningún C#». Las vistas Razor permiten expresiones, bucles y condicionales de presentación. Lo que evitaremos es añadir lógica empresarial o de acceso a datos dentro de la vista.

«Si la vista recibe `null`, la ruta siempre devuelve 404». No. Una vista bien diseñada puede mostrar un mensaje informativo y responder HTTP 200. Un 404 corresponde a un recurso o endpoint no encontrado, no automáticamente a la ausencia de un objeto.

«El navegador recibe instancias de `Producto`». En esta aplicación MVC, el navegador recibe HTML generado a partir de esas instancias. El objeto C# permanece en el servidor.

«El nombre de la vista es arbitrario». Si se utiliza `return View()` sin especificar un nombre, MVC recurre a convenciones basadas en el controlador y la acción. Por eso nuestro archivo está en `Views/Productos/Index.cshtml`. Más adelante veremos cómo personalizarlo.

«MVC y Razor Pages son lo mismo porque ambos emplean Razor». No. La vista MVC es un elemento de presentación asociado a una acción de controlador. Una Razor Page estructura la gestión de solicitudes alrededor de una página y su PageModel. La comparación detallada será en el capítulo 2.5.

Ventajas y costes al modificar un requisito.

Imaginemos cuatro solicitudes de cambio realizadas por el responsable de una aplicación:

Cambio 1: «El producto de id 1 debe llamarse Bloc». Afecta al valor de los datos que genera nuestro controlador. No requiere modificar las etiquetas HTML. La práctica del reto demuestra precisamente este caso, con la variante de código ya comentada junto al objeto original.

Cambio 2: «Los nombres deben aparecer resaltados». Se trata de una modificación visual. En este punto miraríamos primero `Index.cshtml` y, más adelante, podríamos extraer el marcado reutilizable a una vista parcial. El contrato `Producto` puede seguir igual.

Cambio 3: «Añade un enlace Productos al menú de todas las páginas». Corresponde a una estructura de presentación compartida. El `_Layout.cshtml` permite aplicar ese cambio una vez en el lugar adecuado, sin obligar a cada controlador a incorporar manualmente el enlace.

Cambio 4: «Los cambios de nombre deben conservarse después de reiniciar». No se resuelve solamente cambiando `@Model.Nombre`, porque lo que falta es persistencia. El proyecto actual crea la misma colección fija cada vez; será necesario introducir un mecanismo que almacene datos. Se abordará en M02 para no confundir objetivos.

La separación de responsabilidades **no significa que cada cambio afecte siempre a un solo archivo**. Si ampliamos la clase `Producto` con una nueva propiedad obligatoria, puede ser preciso modificar el código que construye sus instancias y las vistas que utilicen esa propiedad. La ventaja es que sabemos por qué cada archivo necesita cambiar y cuál es el contrato compartido.

Un pequeño ejemplo de evolución sin añadir capas innecesarias.

Si necesitáramos mostrar una propiedad descriptiva adicional, el razonamiento sería:

```
public sealed class Producto
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public decimal Precio { get; set; }
    public string Descripcion { get; set; } = string.Empty;
}
```

Este fragmento es **un ejemplo de lectura conceptual**, no un paso que deba activarse ahora en el proyecto. Primero habría que valorar si la propiedad pertenece realmente al concepto Producto. Después se proporcionarían valores y, finalmente, una vista decidiría si y dónde mostrarla. No hace falta introducir un sistema de servicios para comprender esta secuencia.

En el código ejecutable del curso no añadimos `Descripcion` durante 1.2; la clase suministrada mantiene únicamente las tres propiedades necesarias, y el reto se concentra en variar el nombre de una instancia. Con ello mantenemos un ejercicio acotado y reproducible.

Puente práctico: observar un modelo sin copiar sus valores

Hasta aquí hemos separado el objeto `Producto`, el controlador que lo prepara y la vista que lo representa. La separación resulta realmente útil cuando añadimos una necesidad de presentación **sin volver a modificar los datos**. Pensemos que la página debe destacar el nombre del primer producto, además de mostrar la colección.

Si la acción ya entrega una lista con `View(productos)`, no necesita construir una segunda colección ni generar una frase HTML. El dato está disponible en la vista a través de `Model`. Pero hay una condición que no podemos ignorar: **tener una referencia a una lista no garantiza que haya un primer elemento**. La vista del curso comprueba primero `Model is null`; dentro de la rama que tiene lista, puede examinar `Model.Count` antes de acceder a un índice.

Esta es una diferencia fundamental entre tres situaciones: una referencia nula, una colección vacía y una colección con datos. La primera no ofrece ninguna instancia de lista; la segunda es una lista válida sin elementos; la tercera permite seleccionar posiciones como la inicial. Esas situaciones pueden necesitar presentaciones distintas, y no se resuelven escribiendo valores fijos en Razor.

La aplicación autónoma P1.2 exige usar esa distinción para construir **una frase dependiente del modelo**, no una frase que contenga el nombre Cuaderno como texto constante. Si después se sustituye ese nombre por Bloc en el controlador, tanto el listado como la frase deberían cambiar. Ese resultado será una evidencia más sólida de separación de responsabilidades que una definición memorizada de MVC.

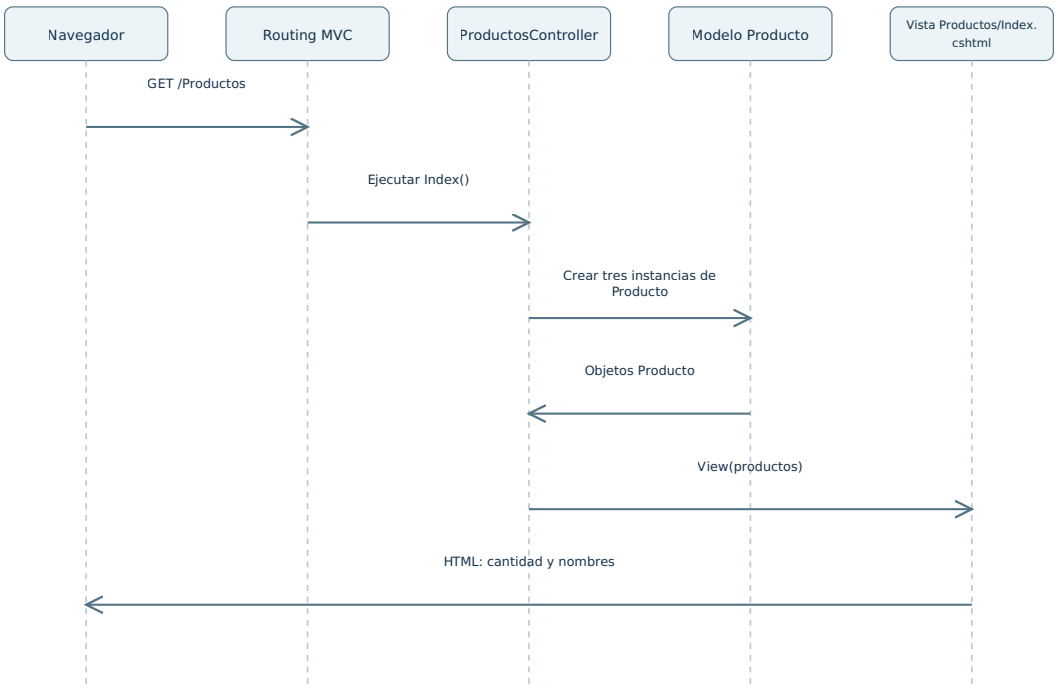
Cómo llegan los datos a la vista

El recorrido completo de una petición.

La separación cobra sentido cuando vemos las tres piezas trabajando juntas. En `Program.cs`, el patrón de ruta convencional registrado es:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Para `GET /Productos`, el segmento «Productos» identifica el controlador y, al omitirse la acción, se utiliza el valor predeterminado «Index». La infraestructura MVC selecciona `ProductosController.Index()`; la acción pasa la lista al resultado de vista; y Razor genera la página que se devuelve al navegador.



Este diagrama representa relaciones **lógicas**: crear objetos del modelo no implica hacer una llamada HTTP a una clase ni que el modelo sea un servicio activo; son instancias C# creadas dentro del mismo proceso. Asimismo, la vista no envía por sí sola un paquete de red al navegador: ASP.NET Core ejecuta el resultado de vista y construye la respuesta HTTP.

El flujo también puede resumirse así:

El diagrama de secuencia anterior permite distinguir **quién inicia la solicitud** de **qué responsabilidad desempeña cada capa**. No representa una red de llamadas HTTP entre clases: la lista, el controlador y la vista se ejecutan dentro de la aplicación ASP.NET Core. El único intercambio HTTP de esa secuencia es el realizado con el navegador. Por ello basta con una única representación del recorrido, sin duplicarlo en varios diagramas equivalentes.

Es importante entender dos diferencias temporales:

1. **En el arranque**, `Program.cs` registra servicios y rutas. No construye la lista para esta vista.
2. **Ante cada petición**, la acción `Index()` se ejecuta y crea una lista nueva en memoria. En este ejemplo no existe un repositorio persistente: cambiar un producto en el código requiere reconstruir o volver a ejecutar el programa para observarlo.

¿Qué ocurre con `/Productos/Index`? La ruta expresa controlador y acción de forma explícita y, por tanto, llega a la misma acción. Esta es una **convención de rutas** que ampliaremos en el capítulo 1.4; en 1.2 solo necesitamos reconocerla y comprobarla.

Qué significa «pasar un modelo a una vista».

Para distinguir entrega de datos y representación, construiremos el proyecto en estados sucesivos.

Estado A: la acción devuelve `View()` sin datos. La vista existe y puede renderizarse, pero su propiedad `Model` es `null`. Nuestra plantilla muestra «El controlador todavía no ha entregado una lista a la vista». La ruta es válida y puede responder HTTP 200: no recibir datos de modelo no implica necesariamente una respuesta 404.

Estado B: la acción devuelve `View(productos)`. La vista recibe tres objetos y muestra «El controlador ha enviado **3** productos». Todavía no hemos activado el bloque HTML de listado. Esto permite comprobar primero el **traspaso de información** entre controlador y vista sin atribuir el resultado a un bucle visual.

Estado C: se activa `@foreach` en la vista. Entonces aparecen Cuaderno, Lápiz y Carpeta. No hemos cambiado el controlador: la vista simplemente utiliza un modelo que ya recibía. Este cambio ilustra con claridad la separación entre suministrar datos y representarlos.

Estado D: habilitamos un enlace Productos en el layout. El navegador puede ir del inicio al listado. No necesitamos modificar la clase `Producto` para cambiar la navegación; tampoco necesitamos crear un nuevo controlador.

Estado	Controlador	Vista	Navegación
A, base	`View()`	Mensaje sin modelo	Solo Inicio
B, datos	`View(productos)`	Cuenta los tres objetos	Solo Inicio
C, listado	`View(productos)`	Cuenta y muestra los nombres	Solo Inicio
D, completo	`View(productos)`	Cuenta y muestra los nombres	Inicio y Productos

La práctica no requiere escribir cuatro aplicaciones distintas. En `PROYECTO/1.2/CursoWeb.Mvc` están preparados los fragmentos que el alumno activa paso a paso, y el alumno compara el efecto de activar cada fragmento desde el navegador.

El contrato de datos como vínculo entre componentes.

La palabra «modelo» se utiliza en desarrollo de software para referirse a cosas diferentes: modelo de dominio, representación de una fila de base de datos, objeto de transferencia, objeto de un formulario o modelo que necesita una vista concreta. En este primer encuentro con MVC utilizamos deliberadamente un único tipo `Producto` para todas las demostraciones, pero es útil entender por qué esta simplificación es válida en el laboratorio y cuándo dejaría de serlo.

Un contrato de datos responde a preguntas precisas: ¿qué propiedades puede consultar el componente consumidor?, ¿cuáles son sus tipos?, ¿cuáles pueden ser nulas?, ¿hay alguna garantía sobre la colección recibida? Nuestra vista del listado declara un tipo que responde a estas preguntas:

```
@model IReadOnlyList<CursoWeb.Mvc.Models.Producto>?
```

La interfaz `IReadOnlyList<Producto>` indica que la vista puede consultar la cantidad de elementos y leerlos por índice, sin necesitar una operación de modificación de la colección. No significa que la lista concreta de C# creada en el controlador sea inmutable. El controlador utiliza `List<Producto>`, que sí permite añadir y eliminar elementos; la interfaz elegida para la vista expresa simplemente las operaciones que esperamos que use. Esta distinción —**la capacidad expuesta por un contrato no tiene por qué incluir todas las capacidades de su implementación**— es un concepto general de diseño de software.

El símbolo `?` detrás del tipo señala que, según las anotaciones de nulabilidad de C#, puede recibirse una referencia nula. La vista contempla ese caso con `if (Model is null)`. Cuando se activa la entrega de la lista, ese caso deja de producirse en el recorrido normal, pero conservar la comprobación nos permite comparar estados sin provocar una excepción.

También importa el tipo de cada propiedad del elemento. `Precio` es `decimal`, no una cadena preformateada con el símbolo «€». El modelo almacena el valor numérico y la vista decide cómo representarlo. Si guardáramos todos los importes como textos, las comparaciones numéricas y la futura

persistencia serían más difíciles de tratar correctamente. Esta separación permite que el valor tenga sentido como dato, independientemente de cómo aparezca en la pantalla.

El valor del contrato no es el valor concreto de los datos.

El tipo `Producto` dice que existe una propiedad `Nombre`, pero no determina qué nombre tiene el primer objeto. Los valores concretos se establecen cuando creamos instancias:

```
var productos = new List<Producto>
{
    new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
    new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
    new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
};
```

La vista solo puede presentar el resultado que recibe. Si el controlador enviara otra lista con productos distintos, el archivo Razor no debería necesitar modificaciones para mostrar los nuevos nombres. Ese es el principio que verificamos con el reto de cambiar Cuaderno por Bloc: el **contrato se mantiene**, pero cambia un **valor de una instancia**.

En un sistema más amplio, podríamos construir un `ProductoListadoViewModel` con únicamente las propiedades que necesita la pantalla. No introducirlo aquí no significa que todo modelo de vista tenga que ser una entidad de dominio; significa que aún no existe una necesidad didáctica que compense añadir tipos adicionales.

Seguir el ciclo de ejecución a nivel de archivo y de método.

Para comprender MVC a fondo no basta con repetir el diagrama «Modelo → Vista → Controlador». De hecho, ese orden puede inducir a error: **una petición HTTP no empieza por ejecutar el modelo**. En nuestro ejemplo, el recorrido observable empieza en el navegador, pasa por la infraestructura ASP.NET Core y llega a una acción concreta.

La ruta convencional registrada por `MapControllerRoute` está preparada para reconocer el controlador y la acción. Al solicitar `/Productos`, el enrutamiento obtiene los valores esperados para `ProductosController.Index`. El controlador recibe el control de ejecución; ahí creamos o seleccionamos los datos y devolvemos un objeto resultado. El motor de vistas representa el archivo Razor correspondiente y utiliza el modelo que ha recibido.

Es importante distinguir **crear un resultado de vista** de **componer inmediatamente todos sus caracteres HTML**. `return View(productos)` expresa qué resultado debe ejecutarse, y la infraestructura MVC se ocupa de representar la vista dentro del proceso de construcción de la respuesta. Para el alumno, esta precisión explica por qué no encontrará el HTML de todas las tarjetas en el método C#.

Una forma de examinar el recorrido sin depuración avanzada.

Podemos detenernos en tres archivos y formular una pregunta diferente en cada uno:

Archivo	Pregunta	Evidencia en el código
`Program.cs`	¿Qué rutas MVC están disponibles?	`MapControllerRoute(...)`
`Controllers/ProductosController.cs`	¿Qué datos se entregan y qué resultado se elige?	`View(productos)`
`Views/Productos/Index.cshtml`	¿Qué HTML se genera a partir de esos datos?	`@model`, `@if` y `@foreach`

Esta lectura tiene una ventaja práctica sobre comenzar abriendo todos los archivos de un proyecto: acota el razonamiento a una petición. Si el HTML muestra una cantidad incorrecta, primero comprobamos qué objetos genera la acción; si la cantidad es correcta pero falta una lista visual, comprobamos la condición o el bucle de la vista. No necesitamos modificar el enrutamiento para resolver un problema de marcado.

No significa que los tres archivos sean independientes en todos los sentidos. El controlador y la vista comparten un contrato tipado, y los nombres de controlador y acción participan en convenciones de búsqueda de vistas. La separación es **de responsabilidad**, no aislamiento absoluto.

La vista como función de presentación.

Resulta útil razonar sobre una vista como una transformación conceptual:

Datos del modelo + plantilla Razor → HTML resultante

No estamos afirmando que Razor sea una función matemática pura o que no intervenga más contexto de la petición; el esquema solo muestra la responsabilidad principal que queremos aprender. Si los datos permanecen iguales pero cambia el archivo Razor, puede cambiar la estructura HTML. Si permanece igual el archivo Razor pero cambian los valores recibidos, puede cambiar el texto representado.

Nuestro ejemplo permite comprobar ambos sentidos de la relación. El paso 1.2.3 modifica la **entrada de datos** de la vista, al pasar de `View()` a `View(productos)`. El paso 1.2.4 modifica la **plantilla de presentación**, al activar el `@foreach`. El reto cambia un **valor del dato**, sustituyendo Cuaderno por Bloc sin tocar la vista.

Esto ofrece un método para pensar en las modificaciones que aparecerán después. Si queremos mostrar un icono junto a cada producto, probablemente cambiaremos la vista. Si queremos seleccionar únicamente productos de cierto tipo, necesitaremos decidir dónde se produce la selección de datos, normalmente antes de entregar el modelo a la vista. Si queremos guardar productos para que sobrevivan al reinicio, necesitamos persistencia, que todavía no existe.

Presentación y negocio no son idénticos.

En un ejemplo pequeño puede parecer razonable poner todas las condiciones dentro de `@if` en Razor. Algunas condiciones realmente son de presentación, como decidir si se muestra un encabezado para una lista vacía. Otras pueden representar políticas de negocio: por ejemplo, decidir qué productos están autorizados para venderse en cierta jurisdicción. La segunda no debería quedar oculta exclusivamente en una plantilla HTML, porque podría necesitarse también desde una API o desde otros procesos.

El límite no es «está prohibido usar C# en la vista», sino **qué responsabilidad pertenece a cada decisión**. En M01 usaremos condiciones simples de presentación y mantendremos los datos fijos fuera del archivo Razor.

Distinguir Model, ViewData y contenido fijo.

Una vista MVC puede obtener información de maneras diferentes. Para entender la estructura del proyecto es importante reconocer qué se utiliza en cada caso, sin concluir que todas las formas son intercambiables.

La directiva `@model` declara el tipo de modelo que la vista espera. Cuando el controlador ejecuta `return View(productos)`, la colección se entrega como el modelo de esa vista. Es el mecanismo principal de nuestro ejemplo porque el listado necesita objetos con propiedades conocidas: `Id`, `Nombre` y `Precio`.

El proyecto también utiliza `ViewData["Title"]` para establecer el título del documento. Se trata de un valor de presentación que el layout puede consultar. No define los productos del catálogo ni sustituye a la colección que recorre `@foreach`. En este caso es suficiente para comunicar una cadena sencilla a la estructura compartida.

Por otra parte, el propio archivo Razor contiene texto fijo, como encabezados y explicaciones docentes. Ese texto aparece aunque no cambien los productos. No todo carácter de HTML procede de una propiedad del modelo; una plantilla combina estructura fija con contenido que depende de los datos recibidos.

Una comprobación aplicada al catálogo.

En `Views/Productos/Index.cshtml` encontrarás un encabezado que describe el punto del curso, un párrafo que muestra `Model.Count` y una lista que recorre `Model`. El encabezado es texto fijo de la plantilla; el contador depende de la colección; los nombres dependen de cada instancia.

Si cambiamos Cuaderno por Bloc en el controlador, el encabezado y la estructura de la lista deberían permanecer iguales. Solo se modifica el nombre representado para ese objeto. Si cambiamos el texto del encabezado en la vista, la colección de datos seguirá teniendo los mismos productos.

Esta distinción permite diagnosticar incidencias con mayor precisión. Un error en un título estático puede resolverse en Razor; un dato equivocado del producto requiere investigar qué valor se preparó para el modelo; un problema de navegación común puede estar en el layout.

Por qué no abusar de valores sueltos.

Sería posible transportar gran cantidad de valores individuales mediante mecanismos no tipados, pero en una vista que presenta una colección estructurada resulta más claro utilizar un modelo fuerte. El compilador y el editor pueden ayudar a comprobar el acceso a propiedades, y el código expresa qué tipo de datos recibe.

No estamos enseñando una prohibición absoluta de `ViewData`. Lo usamos para el título porque esa es una responsabilidad de presentación sencilla. La regla pedagógica es elegir un mecanismo coherente con la información que se necesita representar.

Al completar el punto 1.2, el alumno debe poder diferenciar el contrato `@model`, el valor `Model` recibido por la vista, los valores de título aportados al layout y los textos estáticos escritos en el archivo Razor. Esa distinción será especialmente útil cuando en 1.5 introduzcamos una parcial con su propio contrato de modelo.

Casos de diagnóstico y evidencias

Relación con la práctica P1.2.

La práctica construye un recorrido comprobable en cuatro estados, partiendo del resultado de 1.1: inspeccionar la clase `Producto`, navegar a un controlador que aún no envía datos, activar una lista fija y pasarla mediante `View(productos)`, habilitar el `@foreach` de la vista y, finalmente, mostrar un enlace de navegación. Después se modifica el nombre de un producto para constatar que la vista utiliza los valores del modelo en vez de tener nombres escritos directamente en su HTML.

Es necesario **observar cada estado antes de activar el siguiente**, porque el objetivo principal no es acabar cuanto antes una página sino explicar por qué apareció cada elemento: la cantidad depende de la entrega del modelo, el listado depende de la lógica de presentación y el enlace depende del layout.

Al finalizar, debe poder responderse a la pregunta: *«Cuando cambio el nombre de Cuaderno por Bloc, ¿qué archivo modifico y por qué no tengo que reescribir la vista?»*. La respuesta debe enlazar las responsabilidades de Modelo, Controlador y Vista que acabamos de estudiar.

Comparar tres situaciones que parecen la misma avería.

Los tres estados siguientes pueden producir una pantalla sin filas visibles, pero representan problemas conceptualmente distintos:

Situación A: no se entregó un modelo. La acción utiliza `return View()`, la vista recibe `null` y muestra el mensaje preparado para ese caso. La página puede responder 200. Es un comportamiento correcto para el estado inicial didáctico.

Situación B: se entregó un modelo, pero el bucle está comentado. La acción envía tres objetos, `Model.Count` informa de la cantidad correcta y el navegador no muestra las filas porque todavía no se ha activado su bloque de representación. El problema, si lo tratáramos como una aplicación final, sería de contenido visual de la vista, no de recuperación de datos.

Situación C: se entregó una lista vacía. No es el estado que ejecutamos con nuestros datos fijos, pero podemos razonarlo a partir del contrato: `Model` no sería `null`, `Model.Count` sería cero y un `foreach` sobre esa lista no produciría filas. En una aplicación real convendría presentar un mensaje específico de catálogo vacío. Una colección vacía y la ausencia de colección no son necesariamente lo mismo.

Estos escenarios revelan por qué las comprobaciones deben observar más de un elemento. Comprobar únicamente HTTP 200 no permite saber si la lista está bien representada. Encontrar un nombre escrito literalmente en la vista tampoco demuestra que el controlador haya entregado correctamente el modelo. Una comprobación útil debe vincular datos y presentación.

Los errores de tipado también son distintos.

Si una vista espera un `Producto` individual y recibe una `List<Producto>`, habrá una incompatibilidad de contrato de modelo. No se resuelve añadiendo más HTML. Lo correcto es revisar la declaración `@model` y el argumento pasado a `View(...)` para garantizar que ambos expresen el mismo tipo de información.

El caso contrario también puede ocurrir: una vista preparada para iterar una colección no puede tratar automáticamente un objeto individual como una lista. En 1.4 tendremos ambas vistas y podremos comprobar por qué conviene que cada una declare explícitamente su tipo.

Cómo comprobar la separación mediante evidencias.

Un alumno debería poder defender que MVC se ha utilizado correctamente mostrando **qué cambia y qué no cambia** entre dos ejecuciones. No basta con afirmar que «el código está en tres carpetas».

Después de activar los datos del controlador, el HTML incluye el contador de tres productos, aunque aún no exista una lista de elementos. Eso demuestra la entrega del modelo sin activar todavía el bucle de presentación.

Después de habilitar `@foreach`, aparecen los nombres y precios. El controlador y la lista no han cambiado entre esos dos estados. Por tanto, hemos aislado el efecto de la plantilla.

Después de habilitar el enlace del layout, el navegador dispone de una vía de navegación adicional. No hemos tenido que cambiar los tres objetos ni la acción. De nuevo aislamos la responsabilidad modificada.

Finalmente, al sustituir Cuaderno por Bloc, el contador permanece en tres y el formato HTML de los elementos sigue siendo el mismo. Lo que cambia es el valor de un modelo. La vista continúa consumiendo `producto.Nombre`, que es exactamente lo que debería suceder.

Podemos resumir estas observaciones mediante casos concretos que puedes comprobar en el navegador:

Modificación	Evidencia positiva	Regresión que no debe producirse
Entregar una lista	Mensaje de tres productos	La portada `/` sigue siendo accesible
Activar bucle Razor	Nombres y precios representados	El contador continúa siendo tres

Modificación	Evidencia positiva	Regresión que no debe producirse
Activar enlace de layout	Aparece una navegación adicional	El listado no desaparece
Sustituir un nombre	Aparece Bloc en lugar de Cuaderno	Los otros dos productos y el contador permanecen
No tocar la ruta	`/Productos` sigue respondiendo 200	No se necesita una ruta adicional para cambiar la vista

Al contrastar esos resultados, explica por qué cambia la página en cada estado. Obtener el resultado esperado sin comprender qué responsabilidad cumple cada archivo no es suficiente.

Conexión con los próximos capítulos sin adelantar sus contenidos.

Una vez entendido qué parte del código representa datos, qué parte decide el resultado y qué parte lo presenta, el resto del módulo gana sentido. En 1.3 exploraremos la ubicación de los archivos y la configuración. En 1.4 veremos cómo una acción recibe un identificador y entrega un único `Producto` a una vista de detalle; será una aplicación directa del mismo contrato que ya hemos utilizado. En 1.5 extraeremos la representación repetida de un producto a una parcial Razor, lo que permitirá reutilizar el mismo marcado desde diferentes vistas.

Es importante no confundir el alcance de este primer día con el de una aplicación empresarial terminada. Nuestro catálogo no tiene operaciones de escritura, persistencia, gestión de usuarios ni validación de formularios. El hecho de que funcione y esté organizado con MVC no implica que disponga ya de esas capacidades. En M02 añadiremos las funcionalidades necesarias de manera incremental, reutilizando la separación de responsabilidades que acabamos de establecer.

Referencias técnicas.

- Microsoft Learn, patrón MVC y separación de responsabilidades: https://learn.microsoft.com/aspnet/core/mvc/overview?view=aspnetcore-8.0
- Microsoft Learn, tutorial de controladores MVC: https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/adding-controller?view=aspnetcore-8.0
- Microsoft Learn, sintaxis Razor y vistas tipadas: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/razor?view=aspnetcore-8.0

1.3. Estructura, convenciones y puesta en marcha de un proyecto ASP.NET Core MVC

Objetivos del capítulo.

Este capítulo permitirá al alumno **orientarse dentro de un proyecto MVC real sin depender de memorizar carpetas**. Al terminar podrá identificar el archivo de proyecto, el punto de entrada, los controladores, los modelos, las vistas y los recursos estáticos; relacionar rutas de archivos con rutas URL; entender qué papel desempeña el archivo de configuración; y modificar un recurso web sin confundirlo con una vista Razor.

Partimos de `CursoWeb.Mvc` en su estado completo del punto 1.2: la ruta `/Productos` ya muestra tres productos y dispone de navegación. El ejercicio de 1.3 no vuelve a construir el catálogo, sino que analiza cómo se organiza y se inicia y añade ejemplos mínimos de configuración y archivos estáticos. Reservamos el diseño de rutas más avanzado y las acciones con parámetros para el punto 1.4.

Leer el proyecto y sus archivos

Situación de partida: sabemos qué hace MVC, pero necesitamos encontrar cada responsabilidad.

En P1.2 ya hemos visto que un controlador construye el modelo y una vista representa la lista. El siguiente obstáculo habitual no es conceptual: es localizar de forma fiable el archivo responsable cuando una página no aparece, un estilo no se aplica o una configuración produce un resultado inesperado.

Un proyecto ASP.NET Core MVC contiene código C#, vistas Razor, archivos estáticos y configuración. Pertenecen a la misma aplicación, pero **no recorren el mismo mecanismo**. Una URL de controlador no es una ruta a una carpeta del disco; un archivo bajo `wwwroot` puede publicarse sin crear una acción; y un archivo de configuración no se convierte en público porque lo lea `Program.cs`.

Por eso P1.3 parte del catálogo que ya funciona. Primero leeremos el árbol del proyecto y su arranque; después seguiremos un valor de `appsettings.json` hasta un mensaje de registro; por último compararemos la solicitud de una hoja CSS, de un archivo público y de otro situado fuera de `wwwroot`. Cada observación ayudará a decidir **dónde mirar cuando algo falle**, evitando cambiar el controlador para resolver una incidencia de recursos estáticos.

De un proyecto C# a una aplicación web.

Un proyecto MVC sigue siendo un **proyecto .NET**: tiene un archivo `.csproj`, compila código C# y genera ensamblados. Su particularidad es que incorpora el SDK web de ASP.NET Core, archivos Razor y una aplicación anfitriona capaz de atender HTTP.

En nuestro proyecto encontramos:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

La primera línea selecciona `Microsoft.NET.Sdk.Web`, que aporta la configuración y las referencias del framework web necesarias para compilar una aplicación ASP.NET Core. `TargetFramework` fija el destino

`net8.0`. `Nullable` habilita las anotaciones de nulabilidad, relevantes cuando una vista pueda recibir un modelo nulo; `ImplicitUsings` permite utilizar espacios de nombres comunes sin repetir siempre `using`.

El proyecto de este capítulo no necesita ningún paquete externo adicional para enseñar la estructura. En M02 añadiremos EF Core 8 con SQL Server Express LocalDB cuando sea necesario guardar productos. Adelantar esas dependencias aquí entorpecería el objetivo didáctico.

El archivo `.csproj` **no es la aplicación ejecutándose**: describe cómo construirla. El arranque real está en `Program.cs` y se ejecuta cuando iniciamos el proyecto. La compilación puede producir `bin/` y `obj/`, carpetas generadas que no deben confundirse con los archivos fuente y que se excluyen de Git.

Mapa real de carpetas: identificar responsabilidades por su ubicación.

En la versión correspondiente al punto 1.3, el árbol de mayor interés es el siguiente:

```
CursoWeb.Mvc/
├── CursoWeb.Mvc.csproj
├── Program.cs
├── appsettings.json
├── Properties/
│   └── launchSettings.json
├── Controllers/
│   ├── HomeController.cs
│   └── ProductosController.cs
├── Models/
│   └── Producto.cs
├── Views/
│   ├── _ViewImports.cshtml
│   ├── _ViewStart.cshtml
│   ├── Home/
│   │   └── Index.cshtml
│   ├── Productos/
│   │   └── Index.cshtml
│   └── Shared/
│       └── _Layout.cshtml
├── wwwroot/
│   ├── css/
│   │   ├── site.css
│   │   └── catalogo.css
│   └── recursos/
│       ├── guia-publica.txt
│       └── aviso-laboratorio.txt
└── DOCUMENTACION/
    └── nota-interna.txt
```

Controllers. Agrupa las clases que atienden acciones MVC. Por convención, `ProductosController` es un controlador y su acción `Index` se relaciona con la URL `/Productos` bajo nuestra ruta convencional. El código recibe la solicitud y elige un resultado. Los detalles de acciones y routing se desarrollarán en 1.4.

Models. Alberga, en este ejemplo, el tipo `Producto`. Se mantiene independiente del navegador y de las vistas. La carpeta facilita encontrar los tipos del dominio, pero no significa que cualquier archivo allí almacenado vaya a ser persistido automáticamente.

Views. Contiene los archivos Razor que MVC utiliza para representar respuestas HTML. La convención `Views/Productos/Index.cshtml` vincula la vista a `ProductosController.Index`, mientras que `Views/Home/Index.cshtml` corresponde a la portada del `HomeController`. La ubicación es significativa para la búsqueda por convención que realiza MVC; no se publica directamente un archivo `.cshtml` para que el navegador lo ejecute.

Views/Shared. Reúne elementos comunes de presentación. `_Layout.cshtml` define el documento HTML envolvente y la navegación, y `@RenderBody()` deja espacio para el cuerpo de la vista seleccionada.

Views/_ViewStart.cshtml. Ejecuta las instrucciones Razor de inicialización compartidas por las vistas de ese árbol. En este caso, establece `Layout = "_Layout";` para utilizar el diseño común.

Views/_ViewImports.cshtml. Permite aplicar directivas e importaciones comunes. En nuestro ejemplo contiene `@using CursoWeb.Mvc` y `@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers`; esto prepara el uso de Tag Helpers que veremos con mayor detalle en el día 2. Las importaciones no sustituyen a los controladores ni sirven por sí mismas un recurso HTTP.

wwwroot. Es la **raíz web** predeterminada. Contiene recursos que, si la aplicación dispone del middleware correspondiente y el tipo de archivo está admitido, pueden devolverse directamente mediante solicitudes HTTP. Los archivos CSS y de texto de este capítulo ilustran esa diferencia frente a los archivos C# o Razor del proyecto.

DOCUMENTACION. Es una carpeta propia del ejemplo fuera de `wwwroot`. Su archivo de texto se ha incluido deliberadamente para demostrar que no toda ruta física bajo el proyecto se convierte por defecto en una URL pública. No debe interpretarse como un mecanismo de seguridad para proteger secretos: un repositorio público puede permitir descargar sus archivos directamente desde GitHub.

Properties/launchSettings.json. Define perfiles locales para ejecutar y depurar desde herramientas de desarrollo. No sustituye a `appsettings.json` ni determina por sí solo el puerto de todos los entornos en los que se publique la aplicación. Cuando se usa `dotnet run --no-launch-profile --urls http://localhost:5091`, el argumento de ejecución establece la URL y el perfil local no se aplica.

En proyectos profesionales pueden existir carpetas adicionales, por ejemplo `Services`, `Data`, `DTOs` o `ViewModels`. No son requisitos de MVC para funcionar; responden a necesidades concretas y las introduciremos solo cuando el temario lo necesite.

Dos formas de leer el árbol del proyecto.

El explorador de soluciones muestra una lista de archivos, pero la estructura cobra sentido cuando la analizamos desde dos perspectivas diferentes.

La primera es la **perspectiva de construcción**: preguntamos qué compila, qué archivos forman parte de la aplicación y qué resultados genera el SDK. En ella son centrales `CursoWeb.Mvc.csproj`, las clases C#, las vistas Razor, `Program.cs` y las carpetas temporales `bin` y `obj`. La segunda es la **perspectiva de ejecución HTTP**: preguntamos qué recibe el navegador y por qué. En esta intervienen las rutas registradas, los controladores, el motor de vistas, el layout y los archivos publicados desde `wwwroot`.

Una misma carpeta puede resultar relevante para ambas perspectivas, pero no con la misma función. Las vistas `.cshtml` forman parte del proyecto y son necesarias para representar HTML; no tienen por qué aparecer publicadas como archivos individuales. Los archivos CSS se incorporan al proyecto y, además, pueden ser servidos directamente como recursos estáticos. Los archivos C# se compilan, pero no se devuelven como texto al navegador por la configuración habitual de MVC.

Este doble enfoque evita dos errores frecuentes. El primero consiste en creer que todo archivo visible en el explorador corresponde a una URL. El segundo consiste en creer que todas las respuestas web tienen que provenir de un archivo físico con el mismo nombre que la dirección solicitada.

El resultado de compilar no es el árbol fuente.

Cuando el SDK construye el proyecto, genera ensamblados y artefactos intermedios. `obj` contiene archivos utilizados en fases de compilación y restauración; `bin` contiene resultados de la compilación para una configuración y un framework concretos. Estos directorios pueden regenerarse y, por ello, se excluyen normalmente del repositorio.

En un curso basado en versiones independientes del mismo proyecto, esta diferencia es especialmente importante. Cada carpeta `PROYECTO/1.1`, `1.2`, etc. tiene su propia copia del código fuente para permitir una actividad reproducible. No se debe versionar el resultado `bin/Release/net8.0` de cada una como si

fuera material docente. El alumno aprenderá más ejecutando `dotnet build` sobre la fuente que recibiendo un ensamblado precompilado.

La evolución entre capítulos también explica por qué no pretendemos que el alumno vuelva a escribir todos los archivos en cada punto. Una vez que ha entendido un componente, la siguiente versión conserva la funcionalidad previa y concentra los cambios en el nuevo objetivo didáctico.

El archivo csproj como declaración de capacidades.

La primera línea del archivo de proyecto selecciona `Microsoft.NET.Sdk.Web`. Esta elección no equivale a ejecutar un servidor, sino a declarar que construimos una aplicación web con un conjunto determinado de objetivos, referencias y tareas de compilación.

En el archivo `CursoWeb.Mvc.csproj` aparecen `Sdk="Microsoft.NET.Sdk.Web"` y `<TargetFramework>net8.0</TargetFramework>`. El primero selecciona el SDK web y el segundo declara el framework de destino; ninguno representa por sí mismo una petición HTTP.

`TargetFramework` es el framework de destino del proyecto, no necesariamente la versión exacta de cada dependencia que pudiéramos añadir después. En nuestro ejemplo, el framework se fija en .NET 8 porque ese es el alcance del curso. Los conocimientos previos de C# y Web API permiten al alumnado reconocer que sigue tratándose de un proyecto de .NET, con su proceso de restauración, compilación y ejecución.

`Nullable` ayuda a expresar y diagnosticar riesgos relacionados con referencias nulas. Es pertinente para nuestros tipos `Producto` y para el contrato nullable del listado, aunque aquí no profundizaremos en todas las reglas del compilador. `ImplicitUsings` reduce algunas declaraciones `using` repetitivas en archivos C#. Ninguna de las dos propiedades crea controladores por sí misma: afectan a la compilación del código.

Un error de restauración, un error de compilación de C# y un fallo HTTP de ejecución tienen naturaleza diferente. Si no se puede restaurar el proyecto, el navegador todavía no es una herramienta diagnóstica útil. Si compila pero el endpoint devuelve 404, debemos investigar rutas y acciones, no necesariamente el archivo `.csproj`.

Tres operaciones del SDK que conviene diferenciar.

```
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

`restore` resuelve las referencias de paquetes necesarias para el proyecto. `build` compila los archivos pertinentes y genera resultados. `run` inicia una aplicación y puede realizar tareas de compilación antes de ejecutar, salvo que se indique otra cosa. Por eso en una demostración de aula preferimos mantener claras las tres etapas: cuando falla una de ellas, resulta más fácil identificar qué capa debemos investigar.

No hace falta explicar ahora todos los parámetros del SDK ni construir una infraestructura de CI compleja. Sin embargo, el alumno debe reconocer cuál es el directorio de trabajo y cuál es el proyecto que realmente está lanzando. Ejecutar un comando correcto en la carpeta equivocada puede dar lugar a confusiones difíciles de detectar con una simple inspección del navegador.

Arranque, servicios y configuración

Program.cs: qué ocurre al arrancar y qué ocurre ante una petición.

El código de arranque actual contiene las operaciones esenciales:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseStaticFiles();
app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

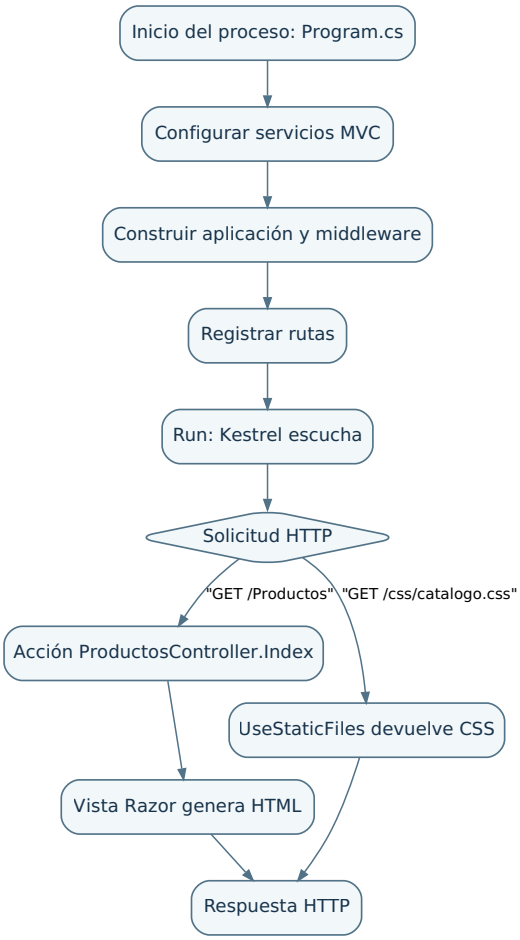
app.Run();
```

La lectura correcta del archivo es **temporal**:

1. `CreateBuilder(args)` prepara configuración, registro de servicios y opciones del host.
2. `AddControllersWithViews()` incorpora los servicios que necesita la infraestructura MVC. No se está mostrando todavía ninguna página.
3. `Build()` construye la aplicación a partir de los servicios configurados.
4. `UseStaticFiles()` incorpora el middleware para atender recursos estáticos reconocidos de `wwwroot`.
5. `UseRouting()` incorpora el middleware de enrutamiento. La ruta convencional registrada mediante `MapControllerRoute` describe los endpoints MVC que la aplicación puede seleccionar.
6. `Run()` inicia el host; a partir de ese momento comienza a atender peticiones en las direcciones configuradas.

Arranque y petición no son lo mismo. Normalmente `Program.cs` se ejecuta una vez al iniciar el proceso, mientras que `ProductosController.Index()` puede ejecutarse tantas veces como un usuario visite `/Productos`. Si se necesitan diez peticiones, no se registran diez veces los servicios. Esta separación ayudará a entender más adelante los ciclos de vida de otros componentes y servicios.

No debemos deducir que `UseRouting()` «dibuja la página» ni que `UseStaticFiles()` sustituye a los controladores. Atienden tipos distintos de trabajo dentro de la plataforma compartida.



El ejemplo utiliza la ruta convencional con controlador y acción predeterminados. Durante 1.3 la identificamos y seguimos; en 1.4 explicaremos sus opciones y su relación con rutas por atributos, parámetros y resultados HTTP.

appsettings.json: configuración distinta del código fuente.

Un proyecto web necesita configuraciones que pueden cambiar según dónde y cómo se ejecute. ASP.NET Core carga valores desde fuentes de configuración, entre ellas archivos JSON, variables de entorno y argumentos de ejecución.

Nuestro `appsettings.json` incluye una sección sencilla de propósito didáctico:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "Curso": {
    "Nombre": "Curso MVC de productos",
    "Edicion": "Laboratorio NET8"
  }
}
```

La sección `Logging` controla los niveles de registro; `AllowedHosts` forma parte de la configuración de hosts y no debe entenderse como un sistema completo de protección de solicitudes. La nueva sección `Curso` es propia de nuestra aplicación y no necesita un proveedor externo.

Después de `Build()`, ASP.NET Core permite leer un valor jerárquico con la notación `Curso:Nombre`:

```
app.Logger.LogInformation(
    "P1.3 configuracion: {Nombre}",
    app.Configuration["Curso:Nombre"]);
```

En el proyecto, esta línea aparece inicialmente **comentada** con la marca `PASO 1.3.3`. Cuando se descomenta y se reinicia, la consola muestra el mensaje `P1.3 configuracion: Curso MVC de productos` como registro informativo. La aplicación ya tiene los servicios de logging del host, por lo que no necesitamos instalar otra biblioteca.

El texto `{Nombre}` de la plantilla de logging es un marcador de valor; permite registrar propiedades de forma estructurada. No debe confundirse con las expresiones Razor de una vista ni con los segmentos del patrón de rutas MVC: son sintaxis parecidas en tecnologías diferentes.

Una observación importante: el argumento `--urls` utilizado al arrancar determina las direcciones del servidor en este laboratorio; no estamos cambiando la sección `Curso` para controlar la URL. Aprenderemos en M05 las diferencias elementales entre ejecutar y publicar.

Seguridad básica. Los archivos de configuración del proyecto pueden contener ajustes, pero **no se deben subir secretos reales** a un repositorio público. Este curso solo usa valores didácticos. El hecho de que un archivo esté fuera de `wwwroot` no impide que un usuario con acceso al repositorio GitHub pueda leerlo.

Los servicios se registran antes de que lleguen peticiones.

En `Program.cs` hemos utilizado `builder.Services.AddControllersWithViews()`. Su posición no es arbitraria: registramos servicios durante la construcción del host, antes de iniciar el proceso de atención de solicitudes. La llamada aporta infraestructura de controladores, vistas y funcionalidades relacionadas que después utilizará MVC.

Es útil separar la frase «registrar servicios» de «crear nuestros tres productos». La primera operación pertenece a la configuración de la aplicación y se realiza al arrancar. La segunda ocurre en nuestro controlador, cuando ejecutamos una acción que crea la lista de datos fijos. No son dos maneras de llamar a lo mismo.

Esta diferencia será importante cuando se incorporen servicios de aplicación en capítulos posteriores. La infraestructura de MVC puede obtener dependencias desde el contenedor integrado de .NET. No necesitamos explicar aquí sus ciclos de vida ni crear interfaces adicionales: basta con reconocer que `AddControllersWithViews` prepara las capacidades de la aplicación antes del procesamiento de peticiones.

La llamada `builder.Build()` marca el paso de configurar el host a construir la instancia de aplicación. A partir de ella utilizamos `app` para preparar el pipeline y los endpoints. Finalmente `app.Run()` inicia la atención de solicitudes. Esta organización no es una peculiaridad estética del archivo: refleja momentos diferentes del ciclo de vida del proceso.

El orden de middleware y la selección de endpoints.

ASP.NET Core procesa solicitudes mediante una secuencia de componentes configurados en el host. En nuestro ejemplo aparecen `UseStaticFiles`, `UseRouting` y `MapControllerRoute`. Es importante observar que **registrar una ruta no significa abrir un archivo**: establece qué endpoints pueden seleccionarse para una petición.

Una forma de comprender el pipeline consiste en seguir dos solicitudes distintas que entran por el mismo servidor.

En `GET /Productos`, el archivo `wwwroot/Productos` no existe. La aplicación puede seleccionar el endpoint del controlador y ejecutar una acción que devuelve un resultado de vista. La respuesta será HTML generado en el servidor.

En `GET /css/catalogo.css`, el archivo estático se encuentra bajo `wwwroot/css`. El middleware de archivos estáticos puede atender la solicitud y devolver el recurso con su tipo de contenido, sin ejecutar `ProductosController.Index()`. El hecho de que ambos recursos se sirvan desde un mismo puerto no significa que sigan el mismo camino interno.

En ambos casos podemos observar con `curl -i` el código HTTP y el tipo de contenido. La elección de `UseStaticFiles` no obliga al navegador a cargar cada archivo por sí solo: para aplicar una hoja CSS, la página HTML debe enlazarla mediante una etiqueta adecuada. Esta es la razón por la que el paso 1.3.4 utiliza un fragmento comentado en el layout.

Qué sucedería si faltara UseStaticFiles.

Si eliminásemos el middleware de archivos estáticos y no hubiera otro mecanismo configurado para servirlos, las solicitudes directas a las rutas de CSS y texto de la raíz web dejarían de funcionar de la manera demostrada. Sin embargo, los archivos permanecerían físicamente en el proyecto. Esto evidencia otra vez que **existencia física de un archivo** y **publicación por HTTP** son conceptos distintos.

No pedimos realizar ese cambio como ejercicio de ejecución en P1.3 porque desestabilizaría otros apartados y obligaría a restaurar configuración compartida. Podemos razonar sobre la consecuencia a partir de la función de `UseStaticFiles`, y la batería de regresión comprueba que los recursos siguen respondiendo tras nuestros cambios.

Leer appsettings.json como una fuente de valores.

En el archivo `appsettings.json` existe una sección didáctica:

```
"Curso": {
  "Nombre": "Curso MVC de productos",
  "Edicion": "Laboratorio NET8"
}
```

No todas las claves de un archivo JSON tienen que pertenecer a una lista fija de propiedades del framework. `Curso` es una sección que hemos creado para nuestra aplicación. El sistema de configuración nos permite consultar valores mediante una clave jerárquica como `Curso:Nombre`.

La notación con dos puntos expresa la jerarquía entre sección y propiedad. Es distinta del acceso a objetos C# con el punto y de las rutas de archivos con la barra. En nuestro caso, la lectura:

```
app.Configuration["Curso:Nombre"]
```

obtiene un valor de tipo cadena. Si quisiéramos expresar una configuración compleja, ASP.NET Core también permite enlazar secciones a clases tipadas, pero no necesitamos introducir ese mecanismo para una única entrada textual.

Configuración y código: qué cambia y qué permanece.

El código del paso 1.3.3 muestra una llamada a logging:

```
app.Logger.LogInformation("P1.3 configuracion: {Nombre}", app.Configuration["Curso:Nombre"]);
```

La cadena de la sección `Curso` aporta el valor, mientras que la plantilla del mensaje define cómo registrarlo. Si cambia el valor de configuración y reiniciamos la aplicación, el código puede seguir siendo el mismo y el mensaje reflejar el valor actualizado. De esta manera comprobamos la utilidad de externalizar un ajuste que no merece ser una constante compilada dentro del controlador.

No debemos atribuir esta operación a una vista Razor. El mensaje aparece en el registro del proceso y no se convierte automáticamente en texto visible en `/Productos`. Para mostrarlo allí tendríamos que decidir explícitamente cómo entregar ese dato a una vista, una actividad que no forma parte del objetivo concreto de 1.3.

Sobre los distintos orígenes de configuración.

ASP.NET Core puede combinar información procedente de archivos JSON, variables de entorno, argumentos y otros proveedores. No existe una única regla universal de prioridad para cualquier configuración personalizada: depende del host y de los proveedores registrados. En las aplicaciones creadas con los valores habituales de `WebApplication.CreateBuilder` hay un orden predeterminado documentado, que puede modificarse.

En este capítulo leemos una sección estable de `appsettings.json` y lanzamos la aplicación con un argumento explícito para la URL. No es necesario memorizar toda la precedencia de proveedores, pero sí evitar la idea errónea de que `appsettings.json` es el único origen posible o de que necesariamente determina el puerto de cualquier despliegue.

El perfil de ejecución local y la configuración de la aplicación.

El proyecto del punto 1.3 contiene un archivo que es fácil confundir con `appsettings.json`: `Properties/launchSettings.json`. No desempeñan el mismo papel. `appsettings.json` contiene valores que la aplicación puede leer mediante su sistema de configuración. En cambio, `launchSettings.json` describe cómo una herramienta de desarrollo inicia localmente el proyecto.

Nuestro perfil original es el siguiente:

```
{
  "profiles": {
    "CursoWeb.Mvc": {
      "commandName": "Project",
      "launchBrowser": true,
      "applicationUrl": "http://localhost:5091",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Qué significa cada entrada. `profiles` agrupa los perfiles disponibles; el nombre `CursoWeb.Mvc` permite seleccionar este perfil por su identificador. `commandName: Project` indica que queremos iniciar el propio proyecto web, no un ejecutable ajeno. `launchBrowser: true` solicita a la herramienta que abra el navegador al arrancar, cuando dicha herramienta admite ese comportamiento. No cambia qué controlador procesa la petición. `applicationUrl` declara la dirección local que usará el perfil y la variable `ASPNETCORE_ENVIRONMENT` establece el nombre del entorno de ejecución para esa sesión.

El nombre `Development` resulta relevante porque ASP.NET Core puede elegir opciones y proveedores de configuración según el entorno. Sin embargo, no equivale a un mecanismo de seguridad ni autoriza el almacenamiento de credenciales en el repositorio. El archivo no debe utilizarse como un almacén de secretos. En este laboratorio la variable explica el contexto del proceso y no modifica nuestra lista fija de productos.

Dos modos de iniciar el mismo código fuente. Desde la carpeta del proyecto podemos optar por un perfil de desarrollo o ignorar los perfiles. Son formas alternativas de iniciar el programa, de modo que deben probarse **por separado**, deteniendo el proceso anterior:

```
dotnet run --launch-profile CursoWeb.Mvc
dotnet run --no-launch-profile --urls http://localhost:5091
```

El primer comando selecciona explícitamente el perfil llamado `CursoWeb.Mvc` y usa su configuración de ejecución. El segundo prescinde de `launchSettings.json` y señala directamente la URL con `--urls`. Precisamente este segundo modo aparece de forma habitual en las prácticas porque reproduce el mismo resultado sin depender de los ajustes del editor o de que el perfil local abra un navegador.

En Visual Studio, seleccionar un perfil de inicio y pulsar **Ejecutar** realiza un trabajo equivalente al de preparar una configuración de ejecución y lanzar el proceso. Es importante que el alumno observe la dirección efectiva mostrada en la salida de Kestrel; no debe dar por hecho que todos los proyectos o entornos escuchan en el mismo puerto. Dos copias de `CursoWeb.Mvc` ejecutadas simultáneamente sobre el 5091 pueden provocar un conflicto de escucha.

Un diagnóstico que evita modificar el archivo equivocado. Imagina que `GET /Productos` devuelve los mismos tres registros, pero el navegador intenta abrir un puerto diferente. En ese caso debemos revisar primero el perfil o los argumentos de arranque. Cambiar `Models/Producto.cs` no resolvería el problema. Por el contrario, si el servidor escucha correctamente y el mensaje `Curso:Nombre` del paso 1.3.3 no coincide con el esperado, es razonable comprobar `appsettings.json` y el código de lectura de configuración.

Esta distinción también permite comprobar la aplicación desde distintas herramientas: la ventana de ejecución de Visual Studio confirma el puerto de Kestrel, mientras que el navegador y `curl` permiten observar la respuesta HTTP de cada URL.

Puente práctico: qué significa publicar una copia de un archivo

La infraestructura `app.UseStaticFiles()` no permite por sí sola que cualquier ruta del disco sea pública. En el caso sencillo de este curso, los archivos estáticos se buscan en la raíz web `wwwroot`. El archivo `wwwroot/recursos/guia-publica.txt` se puede solicitar como `/recursos/guia-publica.txt`; en cambio, un documento que permanezca en `DOCUMENTACION` no se publicará automáticamente con esa ruta.

La diferencia no reside en que un archivo de texto necesite un controlador especial. Reside en **la ubicación física del recurso y en el middleware responsable de responder**. Si hacemos una copia deliberada de un documento de laboratorio a la carpeta pública, una URL que antes devolvía 404 podrá devolver 200. Si solo añadimos un enlace desde el layout pero no copiamos el archivo, el enlace llevará a un recurso inexistente. Y si el archivo ya es público pero no aparece ningún enlace, su URL directa seguirá funcionando.

Este es el problema de la aplicación autónoma P1.3: pasar de un recurso no publicado a uno publicado y distinguir esa decisión de la decisión visual de mostrar un enlace. También nos obliga a introducir un límite importante: que una ruta de archivo sea técnica y fácilmente publicable no convierte en adecuado publicar contraseñas, datos personales o documentación sensible. El laboratorio usa un texto inofensivo preparado para el ejercicio, sin trasladar documentos internos reales.

Recursos públicos y respuestas HTTP

wwwroot: rutas físicas y URLs no son lo mismo.

El archivo `wwwroot/css/catalogo.css` está ubicado físicamente bajo la raíz web del proyecto. Su URL por defecto es `/css/catalogo.css`, sin incluir el nombre de carpeta `wwwroot`. Esto ocurre porque `UseStaticFiles()` expone los recursos desde esa raíz.

En la aplicación preparamos este CSS:

```
/* PASO 1.3.4: recurso estático independiente de los controladores. */
main.container {
    border-left: 5px solid #2d635d;
    padding-left: 1.5rem;
}
```

Mientras el archivo exista bajo `wwwroot`, una solicitud directa a `http://localhost:5091/css/catalogo.css` puede obtenerlo, aunque ninguna vista incluya todavía una referencia a él. La aplicación lo sirve como recurso estático reconocido.

Para que el navegador lo aplique a las páginas MVC, añadimos en `Views/Shared/_Layout.cshtml` una etiqueta HTML:

```
<link rel="stylesheet" href="/css/catalogo.css" />
```

Al activarla, la siguiente navegación solicita el HTML y, a continuación, el CSS adicional. El estilo añade un borde al área principal y permite comprobar visualmente el efecto. El archivo CSS no se transforma en una vista Razor ni hace falta crear un `CatalogoController` que lo devuelva.

La distinción queda reflejada en esta tabla:

Ubicación física	URL de ejemplo	Resultado esperado
`wwwroot/css/site.css`	`/css/site.css`	HTTP 200, CSS
`wwwroot/css/catalogo.css`	`/css/catalogo.css`	HTTP 200, CSS, incluso antes de enlazarlo
`wwwroot/recursos/guia-publica.txt`	`/recursos/guia-publica.txt`	HTTP 200, texto
`wwwroot/recursos/aviso-laboratorio.txt`	`/recursos/aviso-laboratorio.txt`	HTTP 200, texto
`DOCUMENTACION/nota-interna.txt`	`/DOCUMENTACION/nota-interna.txt`	HTTP 404 con esta configuración
`Views/Productos/Index.cshtml`	`/Views/Productos/Index.cshtml`	No es un documento estático servido por `UseStaticFiles`
`Views/Productos/Index.cshtml`	`/Productos`	El HTML se genera al ejecutar la acción MVC

Este comportamiento corresponde a **la configuración concreta de nuestro proyecto**. ASP.NET Core permite modificar las opciones de los proveedores de archivos y habilitar otras rutas estáticas, pero esa ampliación no entra en el alcance de 1.3.

Un archivo fuera de `wwwroot` puede seguir siendo accesible de otra manera si desarrollamos una acción que lo envíe, configuramos otro proveedor de archivos o lo publicamos mediante una infraestructura externa. Por eso la conclusión correcta no es «fuera de wwwroot significa secreto», sino «`UseStaticFiles()` no lo expone por defecto».

Además, retirar un enlace HTML **no despublica** el archivo que ya está en `wwwroot`. Un usuario que conozca o adivine su URL puede solicitarlo directamente. Esta distinción será esencial en el reto práctico.

El reto de distinguir archivos públicos y archivos del proyecto.

La práctica termina con una comprobación sencilla pero reveladora: habilitar en el layout el enlace comentado **Aviso de laboratorio**, que apunta a un archivo ubicado en `wwwroot/recursos`. Después se solicitará por URL otro archivo de texto, `DOCUMENTACION/nota-interna.txt`, situado fuera de `wwwroot`.

El primero debe responder HTTP 200 y el segundo 404 en nuestro proyecto. Más interesante aún, el primero ya era accesible **antes** de habilitar su enlace de navegación. Esta observación muestra que la accesibilidad de un recurso depende de la configuración de publicación, no de que exista un enlace visible en pantalla.

El código y las instrucciones para activar el enlace y observar ambas respuestas están en `P1.3`. El reto no requiere conocimientos de seguridad avanzados ni recursos externos.

Una lectura rigurosa de la raíz web.

El nombre `wwwroot` tiene una función precisa: se utiliza como raíz de archivos web para el middleware estático en la configuración convencional de la aplicación. Una solicitud a `/recursos/guia-publica.txt` no contiene el nombre `wwwroot` en su ruta URL, aunque el archivo físico esté ubicado dentro de esa carpeta.

Podemos expresar la correspondencia del ejemplo:

```
Ruta física:
CursoWeb.Mvc/wwwroot/recursos/guia-publica.txt

Ruta HTTP:
http://localhost:5091/recursos/guia-publica.txt
```

La correspondencia depende de que la aplicación esté configurada para servir estáticos desde su raíz web. No debe interpretarse como una ley del protocolo HTTP ni como una equivalencia entre todas las carpetas del repositorio y rutas públicas.

Esto es particularmente relevante para la seguridad básica. Un archivo colocado bajo `wwwroot` puede quedar accesible por URL aunque nunca aparezca en un menú. No debemos guardar allí documentos privados, credenciales, claves o materiales que no pretendemos publicar. Del mismo modo, guardar una cadena ficticia fuera de `wwwroot` no garantiza su secreto si el repositorio está disponible públicamente en GitHub.

¿Qué enseña el recurso fuera de wwwroot?.

En `DOCUMENTACION/nota-interna.txt` hemos creado un ejemplo de archivo que pertenece al proyecto, pero no se publica como recurso estático en la configuración actual. La URL `/DOCUMENTACION/nota-interna.txt` devuelve 404 porque el middleware no utiliza esa carpeta como raíz web y no existe una acción que la atienda.

Este resultado confirma **una propiedad de la configuración actual**, no una garantía absoluta aplicable a cualquier servidor. Un administrador podría crear otra configuración que sirviera contenido de esa carpeta, o un controlador podría leer y devolver su texto. La seguridad depende de todas las vías de acceso al recurso, no solo de una convención de ubicación.

El reto de 1.3 plantea precisamente esta comparación. Habilitar el enlace a `aviso-laboratorio.txt` modifica la navegación visible, pero no altera el estado de publicación del archivo: ya estaba dentro de `wwwroot` desde el principio.

Tipos de recursos y cabeceras: razonar a partir de HTTP.

Cuando solicitamos un archivo CSS esperamos una respuesta que el navegador pueda tratar como hoja de estilos. Un código 200 es necesario para demostrar la entrega satisfactoria en nuestro ejemplo, pero **no es suficiente**: también es relevante que el tipo de contenido sea `text/css`.

Del mismo modo, un archivo de texto publicado puede producir un cuerpo con líneas sencillas y una cabecera de contenido textual, mientras que una acción MVC que devuelve una vista produce HTML. Un navegador puede mostrar ambos, pero su significado técnico es diferente.

Un pequeño diagnóstico consiste en comparar estas solicitudes:

```
curl -i http://localhost:5091/Productos
curl -i http://localhost:5091/css/catalogo.css
curl -i http://localhost:5091/recursos/guia-publica.txt
curl -i http://localhost:5091/DOCUMENTACION/nota-interna.txt
```

La primera requiere que MVC encuentre el controlador y renderice su vista. Las dos siguientes corresponden a archivos estáticos públicos. La última no encuentra un recurso publicado por la configuración existente.

La comprobación enseña más que aprender de memoria una lista de carpetas. Permite construir una hipótesis sobre qué parte de ASP.NET Core debe procesar una URL y contrastarla con el resultado. En problemas reales, esta habilidad ayuda a distinguir un fallo de rutas, un recurso estático ausente y una respuesta generada correctamente pero sin el formato esperado.

Vistas, layouts y convenciones

Layout, vistas y Tag Helpers: convenciones de presentación.

La página `/Productos` se compone de la vista `Views/Productos/Index.cshtml` y del layout compartido `Views/Shared/_Layout.cshtml`. El archivo `_ViewStart.cshtml` establece ese layout; `_ViewImports.cshtml` define importaciones y habilita Tag Helpers para las vistas.

El layout contiene una zona para las hojas de estilo, navegación y `@RenderBody()`. En nuestro ejercicio activaremos dos enlaces comentados:

```
<link rel="stylesheet" href="/css/catalogo.css" />
<a href="/recursos/guia-publica.txt">Guía pública</a>
```

Se trata de HTML con rutas de recursos estáticos. La vista `Index.cshtml` no necesita saber dónde está la hoja CSS ni incorporar cada enlace de navegación; es una decisión de diseño compartida.

En el día 2 estudiaremos Tag Helpers como `asp-for` y `asp-action`. De momento conviene conocer su lugar de registro, pero no anticipar su sintaxis en formularios. Del mismo modo, las vistas tipadas que ya utilizamos en `1.2` se profundizarán en `1.5`.

Un error frecuente consiste en pensar que una carpeta llamada `Views` se distribuye al navegador como si fuera un directorio de archivos web. No es así en nuestro proyecto: son archivos fuente para generar una respuesta **del lado del servidor**. Las hojas CSS, en cambio, sí se entregan como recursos separados al navegador.

Convenciones, ventajas y límites.

Las convenciones de MVC reducen configuración repetitiva. Los nombres `ProductosController`, `Index` y `Views/Productos/Index.cshtml` bastan en nuestro ejercicio para conectar acción y vista. La carpeta `Shared` permite compartir el layout. `wwwroot` evita escribir una acción para cada CSS.

No hay que convertir esas convenciones en leyes universales. Una aplicación puede personalizar rutas, nombres de vistas y proveedores de contenido estático. Para avanzar con seguridad, primero aprenderemos el funcionamiento convencional; después abordaremos las personalizaciones que el temario sí necesita.

La organización del proyecto también favorece el aprendizaje incremental: añadir un modelo de producto no modifica necesariamente el layout, y cambiar el CSS no requiere modificar la clase del dominio. Esa es una continuación directa del principio de responsabilidades MVC aprendido en el capítulo 1.2.

Qué no haremos todavía: estructurar una solución de múltiples proyectos, configurar contenedores de dependencias personalizados, introducir infraestructura de autenticación, publicar en IIS/Docker, aplicar bundling complejo ni estudiar todos los orígenes de configuración del host. Esas decisiones no ayudan al objetivo limitado de este capítulo.

Convenciones de vistas y archivos compartidos.

Las carpetas `Views/Home` y `Views/Productos` tienen nombres que coinciden con los controladores de la aplicación. No es casualidad: MVC dispone de reglas de búsqueda de vistas que aprovechan esas convenciones. Cuando `HomeController.Index()` devuelve `View()` sin nombre explícito, la infraestructura busca un archivo de vista correspondiente a la acción, por ejemplo `Views/Home/Index.cshtml`.

Esta búsqueda no convierte `Views` en un directorio público de archivos. La ruta de la solicitud sigue siendo `/Home/Index` o `/`, y el documento final se genera en el servidor.

El layout compartido reduce duplicación. Si todas las páginas deben presentar los mismos enlaces, sería un error copiar el HTML del menú en cada vista concreta. Al editar `Views/Shared/_Layout.cshtml`, el cambio se aplica a las páginas que utilizan ese layout.

`_ViewStart.cshtml` establece la selección del layout para las vistas; `_ViewImports.cshtml` facilita directivas comunes. Aunque sus nombres son parecidos, tienen tareas distintas. En P1.5 utilizaremos además una vista parcial compartida que no necesita ejecutarse como una página independiente, lo cual reforzará la diferencia entre archivos fuente de presentación y URLs públicas.

Diagnóstico y preparación de la siguiente práctica

Problemas frecuentes.

«Una vista Razor se abre por su ruta de archivo». En el flujo MVC habitual se solicita una URL asociada a una acción; la infraestructura localiza la vista y genera HTML. La ruta física del archivo no se equipara a una URL pública.

«Para servir un CSS tengo que crear un controlador». No en esta aplicación: `UseStaticFiles()` y `wwwroot` ya cubren esa necesidad.

«Si no hay enlace, el recurso está oculto». No. Los archivos estáticos pueden solicitarse directamente si se conoce la URL y están publicados.

«Todo el contenido del proyecto está disponible en el navegador». No por defecto. El código C#, los archivos de construcción y otros recursos del proyecto no se sirven automáticamente como estáticos por esta configuración.

«Program.cs se ejecuta por cada petición». No. Configura y arranca el proceso una vez; los controladores atienden las peticiones que llegan después.

«El perfil de Visual Studio y appsettings.json son idénticos». No. `launchSettings.json` describe perfiles de lanzamiento local; `appsettings.json` aporta configuración de aplicación y es solo uno de los orígenes posibles.

«El archivo de configuración incluye valores seguros porque no está en wwwroot». Esto no justifica guardar contraseñas en un repositorio. Los recursos y las políticas de despliegue deben distinguirse del control de acceso al código fuente.

Aplicación diagnóstica: localizar antes de editar

Una estrategia de diagnóstico basada en carpetas.

Cuando una página deja de funcionar, la tentación es abrir todos los archivos del proyecto y buscar un error al azar. La estructura MVC permite una secuencia más controlada.

Si `/Productos` devuelve 404, primero comprobamos la configuración de rutas y que exista `ProductosController.Index()`. Si devuelve 200 pero no aparecen nombres, revisamos el modelo entregado y el archivo de vista. Si los nombres aparecen sin estilos, verificamos la referencia CSS del layout, la ubicación física del recurso y su respuesta HTTP. Si la configuración `Curso:Nombre` no aparece en el registro, inspeccionamos `appsettings.json` y la línea de `Program.cs` que la lee.

Esta estrategia vincula **síntoma observable → responsabilidad probable → archivo concreto**. No garantiza que todo fallo real se resuelva siguiendo un único camino, pero evita modificar clases de dominio para corregir un enlace CSS o modificar rutas para solucionar un valor de configuración.

Caso de estudio: se ha copiado el proyecto, pero no aparecen los cambios.

Imaginemos que un alumno abre el proyecto de `PROYECTO/1.2` en una ventana y ejecuta el de `PROYECTO/1.3` desde una terminal. Puede guardar una modificación correcta en el archivo equivocado y concluir que Razor o el compilador no funcionan.

La primera comprobación no debe ser reescribir el código. Debemos identificar el directorio de ejecución y el archivo que se ha modificado. Después detener y volver a lanzar la copia correspondiente. Esta incidencia es especialmente probable en un curso que conserva estados autónomos, de modo que la práctica incorpora explícitamente la ubicación de cada versión.

Si se producen dos intentos de arranque sobre el mismo puerto, el segundo puede fallar porque ya existe un proceso escuchando. No debe diagnosticarse automáticamente como un error de MVC ni del controlador: es una condición del entorno de ejecución.

Preparación para la práctica y el punto 1.4.

La práctica `P1.3` parte de una página `/Productos` ya funcional, sigue el árbol de archivos, activa un registro de configuración, incorpora una hoja CSS, habilita un enlace a un documento estático y resuelve un reto sobre publicación de recursos.

El alumno debe terminar explicando **en qué carpeta se modifica una cosa y por qué**, además de haber comprobado cada resultado. Esta capacidad permite avanzar al punto `1.4`, donde los controladores atenderán nuevas acciones, parámetros y rutas.

Referencias de Microsoft Learn.

- Estructura básica de un proyecto MVC: https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/start-mvc?view=aspnetcore-8.0
- Archivos estáticos en ASP.NET Core: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/static-files?view=aspnetcore-8.0
- Configuración en ASP.NET Core: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/configuration/?view=aspnetcore-8.0
- Vistas Razor y convenciones: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/overview?view=aspnetcore-8.0

1.4. Controladores, acciones, parámetros y routing en ASP.NET Core MVC

Objetivos de aprendizaje.

Al finalizar este punto, el alumno sabrá **seguir una URL hasta su acción MVC**, añadir acciones nuevas a un controlador, distinguir una respuesta HTML de una respuesta de texto o JSON, utilizar parámetros provenientes de la URL y controlar los resultados HTTP de éxito y error. También comprenderá **el enrutamiento convencional frente al enrutamiento por atributos**, aplicando ambos al catálogo de productos.

Se trabaja sobre el proyecto 1.4, continuación funcional del 1.3. El listado de productos y su navegación ya funcionan. Construiremos una acción de estado, un detalle tipado y otra dirección para la misma clase de información. El reto final devolverá una respuesta JSON breve desde un controlador MVC para conectar con los conocimientos previos de Web API.

Acciones, rutas convencionales y rutas por atributos

Situación de partida: el catálogo existe, pero todavía no podemos consultar una ficha concreta.

En el punto 1.3 hemos localizado el controlador y distinguido recursos públicos de rutas MVC. Ahora la necesidad cambia: el usuario quiere abrir la ficha de un producto desde el listado. La dirección `/Productos` por sí sola no expresa **cuál** debe mostrarse; necesitamos asociar una URL con una acción que reciba un identificador y elija un resultado.

Empezaremos por las rutas convencionales que ya utiliza `Program.cs`. A partir de ellas crearemos una acción `Detalle(int id)` que reciba el número de la dirección y seleccione un `Producto` de la lista en memoria. Después estudiaremos tres comportamientos diferentes: un identificador válido con producto existente, un identificador inválido y otro bien formado que no coincide con ningún producto. El servidor no debe dar la misma respuesta en las tres situaciones.

La práctica P1.4 ampliará el controlador de manera incremental: texto sin vista, detalle con modelo tipado, ruta por atributos y enlaces desde el listado. Concluiremos comparando el HTML con el JSON devuelto por el reto. Así se entenderá el enrutamiento como **selección de comportamiento**, no como una colección de URLs para memorizar.

Qué es una acción MVC y cómo se selecciona.

En los ejemplos anteriores, `ProductosController` tenía una acción `Index()` que recibía una petición GET y devolvía `View(productos)`. Cuando abrimos `/Productos`, ASP.NET Core no busca un archivo `Productos.html` en el disco. El sistema de enrutamiento identifica un **endpoint** MVC y ejecuta la acción correspondiente.

Una acción es un método público del controlador que puede preparar datos, decidir qué resultado devolver y recibir valores de la petición. Derivar de `Controller` nos proporciona métodos auxiliares como `View()`, `Content()`, `NotFound()`, `BadRequest()` y `Ok()`. No todas las acciones deben devolver una vista, aunque muchas de las páginas MVC del curso lo harán.

El arranque contiene nuestra ruta convencional:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Su plantilla se descompone en tres segmentos:

- `{controller=Home}`: el controlador es Home cuando no se proporciona otro.
- `{action=Index}`: la acción es Index cuando la URL no indica otra.
- `{id?}`: se admite un tercer segmento opcional, que más adelante enlazaremos al parámetro `id` del método.

Una URL como `/Productos/Detalle/2` permite obtener los valores de ruta: controlador = Productos, acción = Detalle, id = 2. La tabla muestra la lectura que haremos:

URL	Controlador	Acción	Valor de id
`/`	Home	Index	Sin valor
`/Productos`	Productos	Index	Sin valor
`/Productos/Index`	Productos	Index	Sin valor
`/Productos/Estado`	Productos	Estado	Sin valor
`/Productos/Detalle/2`	Productos	Detalle	2
`/Productos/Detalle/99`	Productos	Detalle	99

Una **coincidencia de ruta** no garantiza que la acción devuelva 200. Puede seleccionar la acción correctamente, pero esta determinar que el producto no existe y devolver 404. Por eso distinguiremos entre «no hay endpoint que coincida» y «el endpoint se ejecuta y responde con un resultado de error».

Reutilizar la lista de productos sin base de datos.

El proyecto 1.3 construía la lista directamente dentro de `Index()`. Para añadir una acción `Detalle(id)` sin escribir una segunda lista distinta, la versión 1.4 extrae los mismos datos a un método privado:

```
private static IReadOnlyList<Producto> ObtenerProductosEjemplo() =>
    new List<Producto>
    {
        new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
        new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
        new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
    };

[HttpGet]
public IActionResult Index()
{
    return View(ObtenerProductosEjemplo());
}
```

La lista **no es persistente**, y el método la crea en memoria cada vez que se solicita. Usamos esta refactorización mínima exclusivamente para compartir los mismos datos entre el listado, la ficha y el reto. El método privado **no es una acción pública**; no se publica automáticamente como una URL. En M02 sustituiremos los datos fijos por la persistencia con EF Core.

Enrutamiento convencional frente a rutas por atributos.

Hasta ahora el registro de `MapControllerRoute` determina el patrón general. Cuando necesitamos una dirección que no respeta `/Productos/NombreAccion/id`, podemos definir **una plantilla de ruta sobre una acción**.

Añadiremos:

```
[HttpGet("/catalogo/ficha/{id:int}")]
public IActionResult Ficha(int id)
{
    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
    {
        return NotFound();
    }

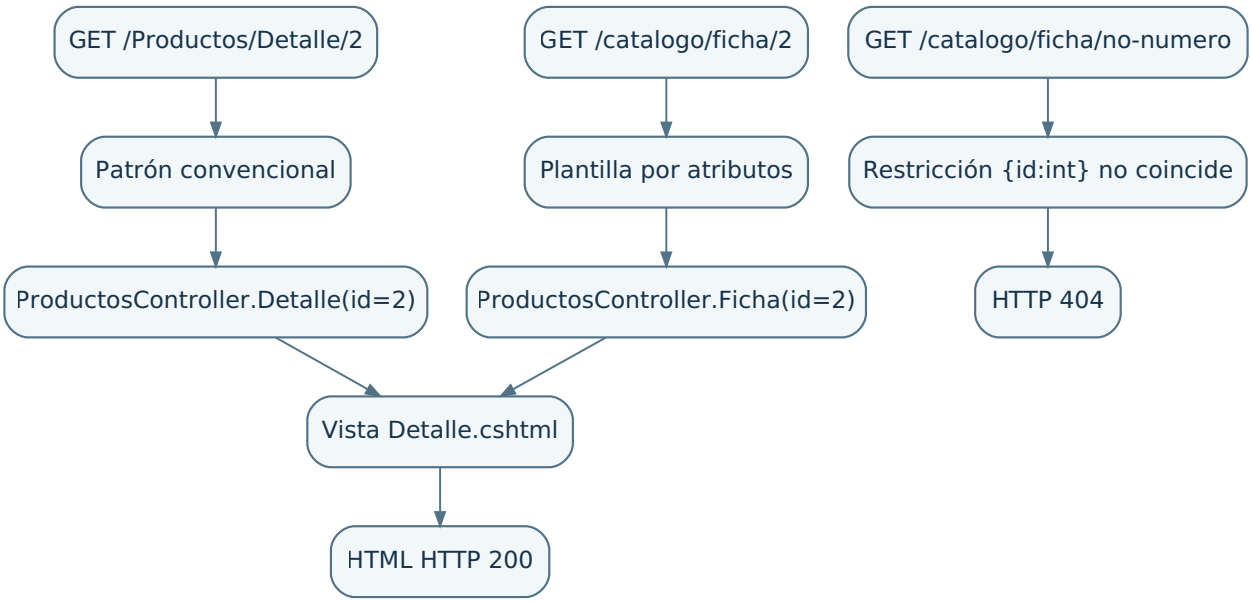
    return View("Detalle", producto);
}
```

La plantilla absoluta `"/catalogo/ficha/{id:int}"` indica una dirección nueva, por ejemplo `GET /catalogo/ficha/2`, aunque el controlador se llame `ProductosController`. El prefijo `/` señala una plantilla absoluta. `{id:int}` exige que el segmento de la ruta pueda interpretarse como entero; una petición `/catalogo/ficha/no-numero` **no cumple la restricción** y no alcanza esta acción, por lo que obtendrá 404.

La acción `Ficha` utiliza la misma colección y retorna `View("Detalle", producto)` para **reutilizar explícitamente** la vista de detalle. Si utilizáramos `return View(producto);` sin especificar nombre, MVC intentaría localizar la vista de nombre `Ficha` por convención. Esta diferencia entre nombre de acción y nombre de vista es uno de los objetivos principales del ejemplo.

Con rutas por atributos, una acción seleccionada exclusivamente por su plantilla no adquiere necesariamente una URL convencional equivalente. En nuestro caso, `/Productos/Ficha/2` no corresponde al endpoint definido; la dirección prevista es `/catalogo/ficha/2`.

Los dos enfoques pueden coexistir en una aplicación. **No significa que una ruta por atributos sea siempre mejor** ni que debamos reemplazar el patrón convencional para todas las páginas. Lo importante es saber qué URL se publica en cada caso y qué restricción acepta.



Razonar sobre los segmentos de una ruta antes de programar.

Una ruta MVC no es el nombre de un archivo. Es una regla utilizada por la infraestructura de endpoints para asociar una solicitud HTTP con código ejecutable. La plantilla convencional de nuestro proyecto:

La plantilla concreta es `{controller=Home}/{action=Index}/{id?}`, tal como se declara en `Program.cs`. `Home` e `Index` son los valores predeterminados; el segmento final, `id`, es opcional.

contiene tres posiciones. Los dos primeros valores tienen nombres y valores predeterminados; el último segmento es opcional. La sintaxis con llaves representa parámetros de ruta, no variables de C# declaradas dentro de `Program.cs`. Su misión es describir **cómo interpretar los segmentos de una URL**.

Para comprenderla, conviene leer las siguientes direcciones sin acudir todavía al navegador:

```
/
 /Home
 /Home/Index
 /Productos
 /Productos/Index
 /Productos/Detalle/1
 /Productos/Detalle/2
 /Productos/Detalle/99
```

Los espacios iniciales que aparecen en esta representación son únicamente parte de la indentación del documento, no caracteres que deban teclearse. En cada caso, identificamos primero el controlador, después la acción y finalmente el eventual parámetro `id`. Una vez comprendida la regla podemos predecir qué método **podría** ejecutarse, pero aún no qué código HTTP devolverá.

Coincidencia de plantilla frente a existencia de acción.

Una URL podría ajustarse formalmente a los segmentos `controller/action/id` y, sin embargo, no existir una acción pública compatible dentro de la aplicación. La regla de ruta no inventa métodos; solo ayuda a seleccionar los endpoints registrados a partir del código de controladores.

También puede ocurrir lo contrario: una acción pública existe, pero su atributo de ruta establece un patrón específico que no coincide con la dirección que suponíamos por su nombre. Esto sucede con `Ficha(int id)`, que tiene una URL explícita `/catalogo/ficha/{id:int}`. La acción no debe interpretarse como si fuera necesariamente accesible por `/Productos/Ficha/2`.

Esta diferencia es una de las razones por las que conviene enseñar enrutamiento después de haber ejecutado acciones sencillas: el alumno ya sabe que una acción puede devolver una vista y, por tanto, puede concentrarse en **por qué fue seleccionada**.

La diferencia entre una dirección, una ruta y un recurso.

Al navegar por Internet utilizamos direcciones completas, pero en el código del curso solemos escribir únicamente la parte que identifica un endpoint dentro de la aplicación. Es importante distinguir estos conceptos porque un error puede producirse en cualquiera de ellos.

La dirección completa de nuestro laboratorio tiene esquema, host, puerto y ruta:

```
http://localhost:5091/Productos/Detalle/2
```

`http` identifica el esquema utilizado; `localhost` es el host; `5091` es el puerto; `/Productos/Detalle/2` es el camino que la aplicación utiliza para seleccionar el endpoint. Cambiar el puerto en el navegador sin cambiar dónde escucha Kestrel puede impedir conectar con el proceso. Cambiar el último segmento, en cambio, puede seleccionar otro producto sin modificar el host.

La ruta convencional definida en `Program.cs` interpreta los segmentos del camino. Por tanto, no es lo mismo preguntar «¿qué servidor atiende la solicitud?» que preguntar «¿qué acción MVC corresponde a esta ruta?». La primera pregunta pertenece al entorno de ejecución y la segunda al enrutamiento de la aplicación.

Un recurso puede no corresponder a un archivo.

La página `/Productos/Detalle/2` no necesita un archivo físico llamado `2.html`. La acción recibe el valor 2, obtiene un objeto y pide al motor Razor que represente una vista. El resultado generado puede contener el nombre Lápiz, pero ese HTML no se obtiene necesariamente de un archivo estático con la misma URL.

En cambio, `/css/catalogo.css` puede resolverse a un recurso estático de `wwwroot`. Ambos casos utilizan HTTP, pero no es correcto deducir por la forma de la URL que uno u otro debe ser obligatoriamente un archivo del sistema operativo.

Esta distinción une los capítulos 1.3 y 1.4. En el primero descubrimos la raíz web y los archivos estáticos; en el segundo añadimos endpoints que ejecutan código. La aplicación contiene ambos tipos de respuesta y una misma URL base no determina que todo se sirva por el mismo mecanismo.

Un ejemplo de error que no se arregla creando otra vista.

Supongamos que un alumno escribe `/catalogo/ficha/2` y recibe 404. Podría interpretar que debe crear `Views/Productos/Ficha.cshtml`. Sin embargo, la acción preparada devuelve `View("Detalle", producto)`, lo que indica explícitamente que desea utilizar la vista existente `Detalle.cshtml`.

Antes de crear archivos, debemos comprobar si la ruta por atributo se ha activado. Mientras el bloque `PASO 1.4.4` esté comentado, el endpoint no existe en la versión que ejecutamos. Crear una vista adicional no resolvería esa ausencia de ruta.

Si la acción está activa y selecciona correctamente el producto, la llamada `View("Detalle", producto)` ya proporciona el nombre de la vista. El aprendizaje consiste en separar **encontrar la acción** de **encontrar el archivo de representación**.

Por qué esa distinción ahorra trabajo.

En una aplicación más grande, múltiples rutas pueden llevar a una misma representación. Si copiamos una vista por cada dirección sin que exista una necesidad de diseño diferente, aumentamos innecesariamente el mantenimiento. El proyecto del curso muestra una alternativa concreta y sencilla: reutilizar una vista y expresar su nombre cuando la convención basada en el nombre de la acción no es apropiada.

No estamos promoviendo que todas las acciones utilicen siempre la misma plantilla. Solo demostramos que la elección de la ruta y la elección de la vista son decisiones independientes que podemos coordinar de forma explícita.

Puente práctico: dos criterios de búsqueda y una sola vista

El detalle del catálogo utiliza un identificador entero: `Detalle(int id)` busca un elemento por `Id` y decide entre un resultado de vista, 400 o 404. Sin embargo, un identificador no es el único dato que puede viajar en un segmento de ruta. Un parámetro de tipo `string` permite expresar una consulta por nombre. Por ejemplo, una plantilla por atributos del estilo `/catalogo/buscar/{nombre}` puede transportar la palabra Cuaderno y enlazarla con un argumento `string nombre`.

Esta selección responde a dos preguntas distintas: **¿la URL coincide con una ruta que existe?** y **¿existe un producto que cumple el criterio recibido?** La primera corresponde a routing y model binding; la segunda, a la lógica de la acción. La coincidencia con la plantilla no garantiza que el catálogo contenga el nombre solicitado, del mismo modo que recibir `id=99` no garantiza que exista el producto 99.

Para comparar nombres sin que cambie el resultado por la capitalización puede emplearse `StringComparison.OrdinalIgnoreCase`. Es una decisión explícita del código de búsqueda; no la aporta automáticamente el atributo `[HttpGet]`. Una vez encontrado un `Producto`, podemos utilizar la misma

vista `Detalle.cshtml` que usábamos en la búsqueda por identificador: la vista necesita un objeto del tipo correcto, no conocer el criterio con que se obtuvo.

En P1.4 el alumno escribirá esa segunda acción en el controlador existente. Al abrir un producto con dos URLs distintas tendrá que explicar por qué el HTML puede ser el mismo y, sin embargo, **los mecanismos de selección del producto son diferentes**. El reto JSON posterior trabaja sobre otro eje: modificar el formato del resultado. No debemos confundir cambiar la ruta con cambiar el tipo de respuesta.

Parámetros, búsqueda y decisiones de la acción

Parámetros de una acción: el detalle de producto.

Vamos a introducir una acción `Detalle(int id)`. La ruta convencional puede capturar el tercer segmento de una URL y la infraestructura de **model binding** convertirlo a un parámetro `int` del método. El binding toma valores de distintas partes de la petición según el caso; durante este capítulo nos centraremos en el valor de ruta `id`.

El código que activaremos en la práctica es:

```
[HttpGet]
public IActionResult Detalle(int id)
{
    if (id <= 0)
    {
        return BadRequest("El identificador debe ser mayor que cero.");
    }

    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);

    if (producto is null)
    {
        return NotFound();
    }

    return View(producto);
}
```

`int id` declara un parámetro numérico. Cuando la solicitud es `GET /Productos/Detalle/2`, recibimos `id = 2`. La acción busca ese identificador dentro de la lista creada por `ObtenerProductosEjemplo()`.

`BadRequest(...)` crea una respuesta de error **HTTP 400** cuando nuestro propio método detecta un valor menor o igual que cero. En este ejemplo, si visitamos `/Productos/Detalle` sin proporcionar id, el parámetro numérico no anulable tiene el valor predeterminado 0 en ausencia del segmento y entra en esa comprobación. No es una regla universal para todos los controladores: depende de la firma y del binding utilizados.

`FirstOrDefault(...)` selecciona un elemento de la lista o devuelve `null` si no existe. Si visitamos `/Productos/Detalle/99`, no existe un producto con id 99 y el código devuelve **HTTP 404** mediante `NotFound()`. Este 404 ocurre **dentro de una acción válida**, porque la ruta y el método sí se encontraron.

`View(producto)` crea un resultado de vista que recibe un objeto `Producto` y produce una respuesta **HTTP 200 con HTML**. Por convención, `ProductosController.Detalle` selecciona `Views/Productos/Detalle.cshtml`.

La vista será fuertemente tipada:

```
@model CursoWeb.Mvc.Models.Producto

<h1 id="titulo-detalle">1.4 | Detalle de producto</h1>
<dl id="detalle-producto">
    <dt>Identificador</dt><dd id="detalle-id">@Model.Id</dd>
    <dt>Nombre</dt><dd id="detalle-nombre">@Model.Nombre</dd>
    <dt>Precio</dt><dd id="detalle-precio">@Model.Precio.ToString("0.00") €</dd>
</dl>
<p><a href="/Productos">Volver al catálogo</a></p>
```

Este ejemplo utiliza `@Model` porque cada solicitud de detalle recibe **un objeto**, no la colección que recibía la vista de `Index`. El listado tiene `@model IReadOnlyList<Producto>?`; la ficha tiene `@model Producto`. Confundir los dos tipos provocaría errores al ejecutar la vista.

La función de `<dl>`, `<dt>` y `<dd>` es representar relaciones nombre/valor en HTML, como identificador, nombre y precio. No cambian el tipo de resultado HTTP. Los atributos `id` del HTML permiten identificar los valores en el navegador y en las pruebas.

El model binding explicado con el parámetro más simple.

Cuando `Detalle(int id)` recibe una petición a `/Productos/Detalle/2`, el valor textual `2` de la ruta se vincula al parámetro `id` de tipo `int`. Ese mecanismo forma parte del enlace de modelos de MVC, conocido como *model binding*.

El código de la acción no necesita buscar manualmente un segmento en una cadena de URL:

```
[HttpGet]
public IActionResult Detalle(int id)
{
    if (id <= 0)
    {
        return BadRequest("El identificador debe ser mayor que cero.");
    }

    var producto = ObtenerProductosEjemplo()
        .FirstOrDefault(p => p.Id == id);

    if (producto is null)
    {
        return NotFound();
    }

    return View(producto);
}
```

En este capítulo nos interesa la ruta como origen del dato. MVC también puede enlazar valores desde parámetros de consulta, formularios y otras fuentes, pero su estudio sistemático llegará en M02 con operaciones GET y POST. Adelantar formularios completos aquí impediría dedicar el tiempo suficiente a distinguir la selección de una acción de su resultado.

Tres clases de ausencia o invalidez.

Podemos distinguir casos que parecen parecidos:

Falta el segmento de id. En esta acción, con `int id` no anulable y la plantilla convencional con `id?`, la ausencia del segmento conduce a un valor predeterminado que nuestra comprobación rechaza, produciendo un 400. El resultado depende de cómo está definido el método y de que no hemos añadido una convención de validación automática propia de un controlador API.

Existe el segmento y contiene 0. El binding produce un valor entero; la acción se ejecuta y nuestra regla `id <= 0` devuelve 400. Es una decisión expresa de esta demostración.

Existe un entero positivo que no corresponde a un producto. La acción puede buscarlo, pero no encuentra ningún objeto, y devuelve `NotFound()`, es decir, 404. El código de estado describe un resultado distinto del 400.

El caso de un segmento no numérico requiere todavía otra explicación. Si utilizamos una ruta con la restricción `{id:int}`, como la acción `Ficha`, la coincidencia se descarta antes de ejecutar el método. Con la ruta convencional sin esa restricción, el comportamiento depende del proceso de enlace del parámetro y de la validación configurada. Para no mezclar situaciones distintas, las pruebas de P1.4 verifican un segmento no entero específicamente sobre la ruta por atributos.

Leer una acción como una sucesión de decisiones.

Un método de controlador puede entenderse como una secuencia de preguntas:

1. ¿La solicitud permite seleccionar este endpoint?
2. ¿Se ha obtenido un valor de entrada que podamos utilizar?
3. ¿Es válido para las reglas simples del ejercicio?
4. ¿Existe el objeto solicitado?
5. ¿Qué resultado HTTP debemos devolver?

La primera pregunta no se resuelve dentro de `Detalle(int id)`, sino durante la selección del endpoint. Las demás sí pueden observarse en el código del método. Este orden evita escribir una acción que intente renderizar una vista antes de comprobar que tiene un objeto válido.

Evitar un resultado incorrecto cuando no existe el producto.

Supongamos que quitáramos `if (producto is null) return NotFound();` y enviáramos un valor nulo a una vista tipada `Producto` que accede a `Model.Nombre`. La representación podría fallar por ausencia del objeto esperado. Sería un error de ejecución de la vista, no una respuesta 404 diseñada deliberadamente.

El tratamiento correcto del ejemplo consiste en reconocer que no existe el producto y devolver un resultado apropiado **antes de ejecutar la vista**. Esto enseña un patrón de decisión que será reutilizable cuando más adelante consultemos registros persistentes.

En una aplicación real, la clasificación de respuestas de error debe seguir las reglas funcionales y el contrato esperado por los clientes. Aquí elegimos casos explícitos para que el alumnado los pueda predecir y verificar sin necesidad de una infraestructura de errores más compleja.

Restricciones de ruta y validación de dominio.

La plantilla `{id:int}` exige que el segmento pueda interpretarse como entero. Esta restricción participa en el enrutamiento: ayuda a decidir si el endpoint puede seleccionarse. No demuestra que el valor sea un identificador válido de un producto.

Un entero como 99 puede satisfacer la restricción de ruta y, sin embargo, no corresponder a ningún registro. Por eso `Ficha(99)` debe seguir buscando el producto y decidir qué hacer al no encontrarlo.

La acción `Detalle(int id)` incorpora además la regla didáctica de que el identificador debe ser mayor que cero. Esa regla no está expresada en la plantilla convencional. Si la dirección contiene `0`, el método se ejecuta y devuelve 400.

En el reto `Resumen` conservamos la restricción `{id:int}` y buscamos el producto. Para un entero inexistente la acción se ejecuta y devuelve 404; para texto no entero no se selecciona el endpoint. Los códigos son iguales, pero el origen técnico no lo es.

Esta distinción es importante al depurar sistemas web. Una comprobación correcta no consiste solo en memorizar que un navegador muestra una página «Not Found». Debemos identificar si no existía la ruta, si no existía una acción compatible o si la acción devolvió deliberadamente ese resultado.

Diagramas de decisiones sin convertir la clase en una API completa.

Podemos describir la acción `Detalle` mediante un esquema narrativo:

```
GET /Productos/Detalle/{id}
           |
      ¿id <= 0?
       /       \
      Sí       No
      |         |
  HTTP 400   Buscar producto
                |
           ¿Existe id?
            /       \
           No       Sí
           |         |
       HTTP 404  View(producto)
                     |
                HTTP 200 HTML
```

La representación es simplificada: primero debe existir un endpoint seleccionado y el modelo de parámetros debe poder enlazarse como corresponde. Su utilidad está en mostrar que una **misma acción** puede producir códigos de estado distintos según el valor de entrada y el resultado de la búsqueda.

Con la acción de texto `Estado()` la decisión es más sencilla: recibe GET y devuelve una respuesta textual. No intenta seleccionar un producto ni renderizar la vista. Esta diferencia ayuda a reconocer cuándo no debemos diseñar una acción más compleja de lo necesario.

Estado HTTP y mensaje visual.

Al abrir una URL 404, el navegador puede mostrar su propio texto de error o una respuesta vacía, según cómo se procese el resultado. El texto visible no forma parte necesariamente de la vista de productos. Por eso el ejercicio verifica el código HTTP con herramientas de red, en lugar de exigir una página de errores con una apariencia determinada.

El tratamiento visual de errores de producción, las páginas personalizadas y los mecanismos globales de excepción existen, pero no forman parte del objetivo acotado de P1.4.

Por qué el orden de las comprobaciones importa.

El método `Detalle(int id)` comienza comprobando que el identificador sea mayor que cero. Solo después busca un objeto dentro de la lista. Finalmente, si existe, representa una vista.

Este orden facilita la lectura del método. La validación sencilla de entrada aparece al principio, la búsqueda de datos ocupa la parte central y la decisión de representación queda al final. Si el identificador no cumple la regla, no necesitamos continuar. Si el producto no existe, tampoco tenemos un objeto válido que entregar a la vista.

La forma de la acción se puede leer como una sucesión de retornos anticipados:

```
if (id <= 0)
{
    return BadRequest("El identificador debe ser mayor que cero.");
}

var producto = ObtenerProductosEjemplo()
```

```
    .FirstOrDefault(p => p.Id == id);

if (producto is null)
{
    return NotFound();
}

return View(producto);
```

Es posible escribir código equivalente con otros estilos, pero esta estructura es especialmente didáctica porque cada condición conduce a un resultado HTTP visible. No tenemos que descifrar un bloque de `if/else` profundamente anidado para identificar las ramas.

Qué demuestra cada prueba del método.

Para el caso de `id=0`, el método debe terminar en `BadRequest`. Para `id=99`, el identificador no es rechazado por el primer `if`, pero la búsqueda devuelve `null` y se utiliza `NotFound`. Para `id=2`, el código supera ambas comprobaciones y crea un resultado de vista.

Estas tres rutas de ejecución cubren las decisiones esenciales del método sin introducir técnicas de pruebas ajenas al objetivo de este módulo. Son además situaciones independientes: cambiar la comprobación de cero no debería hacer desaparecer el producto de id 2.

Al introducir cada identificador en la URL, se observa el recorrido completo: la ruta selecciona la acción, la acción decide el resultado y el servidor devuelve el estado HTTP correspondiente.

Evitar suposiciones sobre el enlace de modelos.

El enlace de modelos de ASP.NET Core puede obtener valores de varias fuentes de la petición. Para este capítulo seleccionamos el caso más pequeño y claro: un identificador que procede de la ruta. La sintaxis `int id` permite al framework preparar un argumento de tipo entero para la acción.

No debemos concluir que toda cadena recibida del navegador es automáticamente válida o que la plataforma convierte cualquier texto arbitrario al valor que la aplicación necesita. Los valores pueden faltar, no cumplir una restricción o no corresponder a un objeto del dominio.

En `Ficha(int id)` utilizamos una plantilla con `{id:int}`. Esta es una **restricción de ruta**: ayuda a descartar direcciones cuyos segmentos no cumplan el formato entero. No es un mecanismo de autorización ni una garantía de existencia del producto.

Distinguir forma y significado.

El valor 99 tiene forma de entero y satisface la restricción. Sin embargo, no tiene significado como identificador de uno de nuestros tres productos, porque la lista contiene únicamente los ids 1, 2 y 3.

El valor 0 también tiene forma de entero, pero la acción `Detalle` lo considera inválido según una regla explícita. Esa regla expresa una decisión de aplicación, no una propiedad intrínseca de todas las rutas con números.

El texto `no-numero` no cumple la restricción `int` del atributo de `Ficha`, por lo que la dirección no coincide con ese endpoint. Este comportamiento se resuelve en la selección de rutas, antes de buscar un producto.

Estos tres ejemplos permiten enseñar que **una entrada puede ser sintácticamente correcta y semánticamente inválida**. Más adelante, al estudiar validación de formularios, necesitaremos separar de nuevo esos niveles. Por ahora basta con reconocerlos en un único parámetro.

El contrato visible de una ficha de detalle.

La vista `Detalle.cshtml` muestra un objeto `Producto` mediante una estructura HTML que incluye identificador, nombre y precio. Se trata de una interfaz de consulta. No dispone todavía de un formulario para modificar los valores ni de operaciones de persistencia.

La URL contiene el identificador solicitado, pero el contenido de la ficha procede del objeto que la acción encontró. Por eso puede ocurrir que una dirección sea correcta y, sin embargo, el objeto no exista. La acción debe distinguir ese caso y no intentar dibujar una ficha inválida.

Por qué la vista es tipada.

La declaración `@model Producto` facilita leer y comprobar expresiones como `@Model.Nombre`. En lugar de pasar una colección genérica de valores sueltos al archivo Razor, entregamos un objeto cuyo tipo expresa qué propiedades contiene.

En el listado el contrato era distinto, porque necesitábamos recorrer varias instancias. En el detalle el contrato se concentra en una. Esta variación es razonable: las vistas no tienen por qué compartir exactamente el mismo tipo de modelo aunque pertenezcan al mismo controlador.

El enlace de vuelta al catálogo.

La ficha dispone de un enlace hacia `/Productos`. Al pulsarlo, el navegador envía una nueva solicitud y el servidor ejecuta de nuevo la acción correspondiente al listado. No se trata de una llamada directa de un método C# de la vista de detalle al método del controlador. Es navegación HTTP y, por eso, participa el enrutamiento.

Esta precisión prepara al alumno para comprender después los formularios y la navegación de aplicaciones Blazor sin confundir eventos de interfaz con llamadas internas a métodos del servidor.

El valor de los casos negativos en una práctica.

Los ejemplos iniciales de programación suelen centrarse en hacer que una operación tenga éxito. Sin embargo, una aplicación web también debe responder de forma predecible cuando la entrada no corresponde a un recurso válido.

El capítulo 1.4 incluye deliberadamente casos negativos:

- El id 99 no existe en la colección y debe producir 404.
- El id 0 se rechaza en `Detalle` y debe producir 400.
- Un texto no entero no coincide con la restricción de ruta de `Ficha`.
- Una dirección convencional no publicada para la acción `Ficha` debe responder 404.

La función de estas comprobaciones no es hacer el ejercicio más largo sin motivo. Cada caso distingue un mecanismo técnico diferente que el alumno deberá reconocer después en aplicaciones reales.

Una batería que probara únicamente `/Productos/Detalle/2` podría pasar aunque el código omitiera la comprobación de identificadores inválidos. Otra que comprobara solamente 404 podría ocultar que ninguna ruta funciona. Necesitamos una combinación de casos positivos y negativos.

La importancia de no falsear el resultado esperado.

Si la acción devuelve 400 para el identificador 0, no debe presentarse esa respuesta como si fuera un 200 correcto: cada resultado HTTP expresa una decisión distinta. Del mismo modo, si se pide devolver JSON, una cadena de texto con el nombre del producto no es equivalente. Explica siempre qué método genera la respuesta y por qué.

Resultados HTTP y representación

Restricciones de verbo HTTP y respuestas distintas de View.

El atributo `[HttpGet]` indica que una acción está destinada a manejar solicitudes GET. En este ejercicio lo utilizamos sin una plantilla de URL propia para permitir que la acción `Estado()` se alcance por la ruta convencional:

```
[HttpGet]
public IActionResult Estado()
{
    return Content("Catalogo MVC disponible", "text/plain");
}
```

Una solicitud a `GET /Productos/Estado` devolverá **HTTP 200**, contenido de tipo texto y la cadena `Catalogo MVC disponible`. No hay vista `Estado.cshtml`: `Content()` crea un tipo de resultado HTTP distinto de `View()`.

Esto demuestra algo esencial para quien ya conoce Web API: **un controlador MVC no está obligado a devolver HTML**. La acción elige un resultado apropiado para la necesidad de la interfaz. Al mismo tiempo, una aplicación que principalmente muestra vistas no tiene por qué convertirse por ello en una API REST completa.

En el punto 2.1 utilizaremos formularios GET/POST. Aquí no introducimos métodos para alterar datos; solo recibimos consultas GET y comparamos sus respuestas.

IActionResult, resultados concretos y códigos de estado.

En una acción hemos utilizado varios métodos de retorno. Es importante no tratar `return View(...)`, `return Content(...)` y `return NotFound()` como si todos significaran «mostrar una página».

Instrucción del controlador	Resultado habitual en este capítulo	Qué recibe el cliente
`return View(producto);`	200, `text/html`	HTML generado desde Razor
`return Content("...", "text/plain");`	200, `text/plain`	Texto sin layout
`return BadRequest("...");`	400	Resultado de error con información explicativa
`return NotFound();`	404	Respuesta de recurso no encontrado
`return Ok(new { ... });`	200, formato JSON negociado habitualmente	Datos JSON del objeto anónimo

El tipo declarado `IActionResult` permite devolver diferentes tipos concretos de resultado desde una misma acción. Una firma `ActionResult<T>` es frecuente en Web API cuando se desea describir un tipo de dato de éxito y posibles errores; aquí utilizamos `IActionResult` para no desviar la práctica hacia el diseño de API.

Dos causas distintas de una respuesta 404.

Si `GET /no-existe` no coincide con ninguna ruta de nuestra aplicación, el sistema de enrutamiento termina sin ejecutar una acción de catálogo. Si `GET /Productos/Detalle/99` coincide con `Detalle(int id)`, **la acción sí se ejecuta**, intenta buscar el producto y decide `NotFound()`. Ambas situaciones pueden producir 404, pero sus causas y soluciones no son iguales.

400 frente a 404: entrada rechazada o recurso inexistente.

Un identificador que nuestra acción considera inválido, como 0, obtiene 400 por una regla explícita del código. Un identificador válido en formato pero inexistente, como 99, obtiene 404. El código de estado

no aparece automáticamente por el hecho de usar un int: lo decidimos deliberadamente en estos métodos. Verificaremos la diferencia mediante HTTP.

Diferencias entre IActionResult y el contenido concreto.

La firma `public IActionResult Detalle(int id)` declara que la acción devolverá un objeto compatible con la interfaz de resultados MVC. Eso no define por sí solo el formato ni el estado de la respuesta. El valor devuelto en cada rama concreta completa la decisión.

```
return View(producto);
```

indica que deseamos representar una vista con datos. En nuestro caso, esa vista es `Views/Productos/Detalle.cshtml` y produce HTML.

```
return Content("Catalogo MVC disponible", "text/plain");
```

indica que queremos devolver un cuerpo de texto plano. No hay una vista ni un layout en esa respuesta.

```
return NotFound();
```

representa un error HTTP 404, útil cuando la acción no encontró el producto solicitado.

```
return BadRequest("El identificador debe ser mayor que cero.");
```

representa un error HTTP 400 y aporta una explicación sencilla.

El alumno que conoce Web API reconocerá varios de estos métodos. La idea importante es que **MVC y Web API utilizan infraestructura compartida de ASP.NET Core** y no dos protocolos HTTP diferentes. Lo que cambia en una aplicación orientada a páginas suele ser qué resultados predominan y cómo se organizan la presentación y la navegación.

¿Por qué no usamos siempre el mismo tipo de resultado?.

Porque las necesidades del cliente son distintas. Un navegador necesita HTML para representar la ficha. Una comprobación simple de estado puede utilizar texto. Una llamada pensada para consumir datos estructurados puede recibir JSON. Los resultados HTTP permiten hacer explícita esa diferencia.

No hay nada que impida devolver JSON desde un controlador `Controller`, como demostraremos en el reto. Esto no significa que toda aplicación MVC deba diseñar sus endpoints como una API pública ni que una vista deba consumir obligatoriamente otra API para mostrar datos. El curso reserva el consumo de API para Blazor en M04.

Cuando una acción reutiliza una vista cuyo nombre es diferente.

En el caso convencional, `ProductosController.Detalle()` devuelve `View(producto)` y MVC busca una vista relacionada con el nombre de la acción, `Detalle.cshtml`. Sin embargo, `Ficha(int id)` no quiere introducir una segunda plantilla visual idéntica.

Podemos escribir:

```
[HttpGet("/catalogo/ficha/{id:int}")]
public IActionResult Ficha(int id)
{
    var producto = ObtenerProductosEjemplo()
        .FirstOrDefault(p => p.Id == id);

    if (producto is null)
```

```
    {
        return NotFound();
    }

    return View("Detalle", producto);
}
```

La cadena `"Detalle"` del resultado indica explícitamente qué vista se selecciona. La plantilla del atributo indica la **ruta externa** del endpoint. Son dos responsabilidades diferentes, aunque ambas aparezcan en el mismo método.

Esto permite comprobar una idea fundamental: dos URL pueden producir el mismo HTML sin que existan dos copias de la plantilla. Además de evitar duplicación, el mecanismo muestra que **la selección de una acción** y **la búsqueda de la vista** son decisiones relacionadas pero independientes.

¿Cuándo habría que utilizar otra vista?.

Si el requisito pidiera una presentación diferente para `Ficha`, podríamos preparar otro archivo Razor y devolverlo explícitamente. Pero mientras la representación requerida sea la misma, resulta más simple reutilizar el archivo existente. El diseño debe responder a necesidades reales; no conviene multiplicar vistas únicamente porque hemos añadido otra URL.

Tabla de predicciones antes de ejecutar una solicitud.

Antes de abrir las direcciones de la práctica, el alumno debe ser capaz de completar una predicción. Con las acciones del capítulo activadas, los siguientes resultados deberían ser razonables:

Solicitud	¿Selecciona una acción?	Respuesta prevista	Motivo
`GET /Productos`	Sí, Index	200 HTML	Lista de productos
`GET /Productos/Estado`	Sí, Estado	200 texto	`Content()`
`GET /Productos/Detalle/1`	Sí, Detalle	200 HTML	Existe Cuaderno
`GET /Productos/Detalle/2`	Sí, Detalle	200 HTML	Existe Lápiz
`GET /Productos/Detalle/99`	Sí, Detalle	404	No existe id 99
`GET /Productos/Detalle/0`	Sí, Detalle	400	La acción rechaza id 0
`GET /catalogo/ficha/2`	Sí, Ficha	200 HTML	Ruta por atributos
`GET /catalogo/ficha/99`	Sí, Ficha	404	Id entero inexistente
`GET /catalogo/ficha/no-numero`	No	404	No coincide `id:int`
`GET /Productos/Ficha/2`	No en este proyecto	404	Ficha usa ruta absoluta
`GET /catalogo/resumen/2`	Sí, Resumen cuando el reto está activo	200 JSON	`Ok(new {...})`
`GET /catalogo/resumen/99`	Sí, Resumen cuando el reto está activo	404	Id no encontrado

La tabla no debe convertirse en un ejercicio de memorizar códigos. Lo importante es poder **explicar la columna de motivos** y relacionarla con una línea de código del proyecto. Si el resultado real difiere de

la predicción, conviene localizar exactamente qué bloque está activado y qué versión de proyecto se ejecuta.

Cómo leer evidencias de una prueba HTTP real.

Al comprobar `/Productos/Detalle/2`, conviene observar tres cosas separadas: el código **200**, el tipo de contenido **HTML** y un fragmento que identifique al producto **Lápiz**. Cada una aporta una evidencia diferente.

El 200 indica un resultado de éxito; el tipo de contenido indica que se devolvió HTML; el nombre demuestra que la vista utilizó el objeto correcto. Si solo verificamos 200, una acción que devolviera una página vacía podría superar la prueba. Si solo verificamos que aparece Lápiz, podríamos estar observando un texto fijo colocado accidentalmente en otra respuesta.

Para `/catalogo/resumen/2` el razonamiento es parecido, pero el contenido esperado es JSON. No basta con encontrar las letras «Lápiz» en el cuerpo: debemos comprobar que puede interpretarse como JSON y que las propiedades correctas contienen los datos previstos.

Para los casos 400 y 404 la evidencia principal es el estado HTTP. En este curso no exigimos una página de error visual personalizada. El aprendizaje se centra en seleccionar resultados adecuados y distinguir enrutamiento de decisiones de la acción.

Comparar tres respuestas del mismo controlador.

Cuando el alumnado ya conoce Web API, resulta útil mostrar en una misma clase las tres formas de salida que el capítulo introduce:

Una respuesta de texto sencillo se obtiene con `return Content("Catalogo MVC disponible", "text/plain");`, como en `ProductosController.Estado()`.

Una respuesta HTML tipada utiliza `return View(producto);`, como en `ProductosController.Detalle(int id)` cuando se ha localizado el objeto.

```
return Ok(new { producto.Id, producto.Nombre });
```

La primera produce texto, la segunda pide HTML de una vista Razor y la tercera produce un resultado con datos que puede serializarse en JSON. Las tres instrucciones pertenecen al ecosistema de resultados HTTP de ASP.NET Core y no necesitan tres servidores diferentes.

No obstante, las tres salidas no contienen información equivalente. El texto de Estado indica que el catálogo está disponible; la vista de detalle puede mostrar identificador, nombre y precio con marcado HTML; el resumen JSON del reto incluye únicamente identificador y nombre.

Esa diferencia de contenido también debe verificarse. Una respuesta JSON con dos campos no es una copia del HTML de la ficha, aunque ambos se refieran al mismo producto. El código de controlador ha decidido qué representación preparar en cada endpoint.

El formato de salida como parte de un contrato.

Un cliente que espera JSON no debería recibir HTML sin que esa diferencia se considere relevante. Aunque ambos cuerpos puedan contener el nombre «Lápiz», sus formatos y significados son distintos. Por eso, al resolver el reto, comprueba tanto el estado 200 como el tipo de contenido JSON y las propiedades del objeto.

Del mismo modo, una solicitud de Estado no exige un layout HTML completo. El resultado de texto no tiene que incluir navegación ni hojas de estilo. Pretender que todas las respuestas de un controlador sean visualmente idénticas equivaldría a ignorar la flexibilidad de `IActionResult`.

Datos compartidos y enlaces de navegación

Enlaces de navegación desde la vista del listado.

Una ficha de detalle resulta mucho más útil cuando se puede abrir desde el listado. La vista de `Index` ya recorre la colección `Model`. Dentro de cada elemento de la lista añadiremos:

```
<a class="enlace-detalle" href="/Productos/Detalle/@producto.Id">Ver detalle</a>
```

Razor inserta el valor de `producto.Id` en el atributo `href`. Para los tres objetos, las URLs serán `/Productos/Detalle/1`, `/Productos/Detalle/2` y `/Productos/Detalle/3`. Cada enlace usa la misma acción pero envía un **valor de parámetro diferente**.

El código está comentado en el proyecto como `PASO 1.4.5` para que el alumno lo active después de crear y verificar la acción `Detalle`. Activar el enlace antes no impide que el listado cargue, pero conduciría a una URL todavía sin acción y provocaría una navegación fallida. La secuencia docente evita ese problema.

En este punto escribimos el atributo `href` directamente para visualizar qué URL se forma. En M02 aprenderemos los Tag Helpers que permiten generar enlaces y formularios usando metadata MVC en lugar de escribir siempre las rutas a mano.

Generar enlaces a partir de datos del modelo.

La vista del listado trabaja con una colección. Cada elemento tiene un identificador que puede utilizarse para crear la dirección de su ficha:

```
@foreach (var producto in Model)
{
    <li data-id="@producto.Id">
        @producto.Nombre
        <a href="/Productos/Detalle/@producto.Id">Ver detalle</a>
    </li>
}
```

No escribimos manualmente tres URLs independientes; el mismo fragmento Razor se ejecuta para cada objeto y utiliza su `Id`. Esto facilita añadir productos sin tener que modificar el marcado de navegación por cada nueva instancia.

Al mismo tiempo, la URL generada no es un dato de dominio del producto: es parte de cómo la aplicación decide navegar a sus acciones. El modelo solo aporta el identificador. La vista decide dónde situarlo dentro de una dirección. En M02 estudiaremos los Tag Helpers de enlaces para evitar depender de cadenas URL escritas a mano, pero en este capítulo la forma explícita facilita seguir el recorrido.

Riesgos de activar un enlace antes que la acción.

Durante la práctica mantenemos el enlace comentado hasta comprobar que `Detalle(int id)` funciona. Un enlace podría estar bien construido visualmente y conducir a una dirección que devuelve 404, porque todavía no existe el endpoint. El hecho de que el HTML contenga un `href` no garantiza que el servidor atienda esa ruta.

Este detalle enseña la importancia de comprobar la funcionalidad extremo a extremo. Comprueba primero que se ha generado la etiqueta `<a>` y después que su destino responde correctamente al navegar hasta él.

Reutilización de datos y coherencia entre acciones.

La función privada `ObtenerProductosEjemplo()` es deliberadamente sencilla. Gracias a ella, `Index`, `Detalle` y `Ficha` consultan la misma definición de datos, sin copiar tres listas en distintos métodos. El reto `Resumen` hace lo mismo para su respuesta JSON.

Esta estructura no es todavía un repositorio de datos. No almacena información entre peticiones ni coordina transacciones. Simplemente evita duplicación en la demostración. Sin embargo, permite entender qué cambiará en M02: cuando exista una forma persistente de obtener productos, las acciones deberán pedir esa información a la capa apropiada en lugar de reconstruir siempre una lista fija.

Si copiáramos la lista completa en cada acción, podríamos cometer un error difícil de detectar: `Index` mostraría Lápiz por 1,20 €, pero `Detalle` lo mostraría por otro importe al haberse editado solo una copia. Compartir una única función de construcción reduce esa incoherencia, incluso en un programa pequeño.

Alcance de static en el ejemplo.

El método es `private static`. El modificador `private` significa que no se expone como acción pública del controlador. `static` indica que puede ejecutarse sin utilizar estado de una instancia concreta de `ProductosController`. La propiedad que nos importa en P1.4 es **que centraliza los datos de ejemplo y no crea una ruta HTTP**.

No se utiliza una lista estática mutable compartida. El método devuelve una nueva colección con objetos de ejemplo. Esto evita introducir problemas de concurrencia y persistencia que no son necesarios para estudiar el enrutamiento.

Comparar comportamiento funcional y organización del código.

Dos implementaciones pueden producir la misma página en el navegador, pero tener una organización distinta. En nuestra versión inicial, `Index()` construía directamente los tres productos. En el proyecto de 1.4, la lista se obtiene mediante un método privado compartido.

La presentación del catálogo puede permanecer idéntica, aunque el código se haya organizado de otra manera. Esto demuestra que una **refactorización** puede mejorar la reutilización sin cambiar el comportamiento externo esperado.

Si los datos se hubieran copiado en `Index`, `Detalle` y `Ficha`, el ejercicio funcionaría en su primera versión, pero sería más difícil mantenerlos coherentes. Una modificación del nombre de Lápiz podría aplicarse en una copia y no en las otras.

Al utilizar `ObtenerProductosEjemplo()`, las acciones consultan la misma definición de los productos. No es todavía una arquitectura empresarial completa ni una implementación de repositorio; es una mejora limitada para reducir duplicación.

Cómo distinguir una refactorización de un requisito nuevo.

Cuando extraemos la lista a un método, buscamos conservar los resultados anteriores. Cuando añadimos `Detalle(int id)`, estamos incorporando una nueva capacidad observable. Ambas tareas pueden estar relacionadas, pero sus criterios de comprobación son diferentes.

La primera exige que el catálogo continúe mostrando los mismos tres productos. La segunda exige que exista una URL de detalle que acepte identificadores y devuelva respuestas apropiadas. Esta separación permite escribir pruebas útiles y evita confundir mejoras internas con nuevas funcionalidades.

Reto JSON, pruebas y diagnóstico

Reto: devolver JSON desde un controlador MVC.

El reto conecta las distintas ideas del capítulo: crear una acción con parámetro, publicarla mediante una plantilla de atributo, seleccionar un producto y decidir entre un resultado **200 JSON** y un **404**.

El método, incluido comentado en el proyecto, es:

```
[HttpGet("/catalogo/resumen/{id:int}")]
public IActionResult Resumen(int id)
{
    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
    {
        return NotFound();
    }

    return Ok(new { producto.Id, producto.Nombre });
}
```

La URL `/catalogo/resumen/2` selecciona el producto Lápiz y produce un objeto JSON semejante a:

```
{"id":2,"nombre":"Lápiz"}
```

La serialización JSON habitual de ASP.NET Core utiliza nombres de propiedades en `camelCase` para este resultado. No ejecuta una vista Razor ni modifica la plantilla de datos. Una consulta `/catalogo/resumen/99` devuelve 404 porque el método no encuentra un producto. Si se solicita un segmento no entero, la ruta con `{id:int}` no coincide y también se obtiene 404, en ese caso sin ejecutar la acción.

Para un desarrollador que conoce Web API, esto confirma que **MVC comparte los tipos fundamentales de resultado HTTP**, aunque nuestro curso utilice principalmente vistas Razor para representar la interfaz del usuario. En M04 construiremos una API auxiliar mínima que será consumida desde Blazor; el propósito de este reto es observar formatos de respuesta, no enseñar toda la disciplina de diseño de API.

Errores habituales y diagnóstico.

«La ruta existe, luego siempre responde 200». Falso. `Detalle(99)` y `Detalle(0)` coinciden con una acción pero responden 404 y 400, respectivamente, por decisiones expresas del controlador.

«Tengo que crear una vista Estado.cshtml». No, porque la acción usa `Content()`. Solo necesitamos una vista si elegimos un resultado de vista.

«La ruta Ficha se construye como /Productos/Ficha/2». No en este ejemplo. El atributo `[HttpGet("/catalogo/ficha/{id:int}")]` define otra URL, que no coincide con el patrón convencional habitual de esa acción.

«No entiendo por qué la ficha Ficha utiliza Detalle.cshtml». Porque la instrucción `View("Detalle", producto)` señala explícitamente el nombre de la vista compartida por ambas acciones.

«La acción no recibe el id». Revisar que la ruta tiene el tercer segmento y que la firma recibe `int id`. En el caso del atributo hay que escribir `{id:int}` y llamar igual al parámetro que la plantilla vincula.

«Al activar el comentario, aparece un error de compilación». Comprobar que se han quitado **los dos** delimitadores del bloque y no se han alterado las llaves, la indentación semántica ni las llamadas a métodos.

«Una respuesta 404 siempre significa que el servidor está apagado». No. Un servidor apagado provoca normalmente un error de conexión. Un 404 es una respuesta HTTP que el servidor ha generado.

«El navegador recibe un objeto Producto cuando abre el detalle». No directamente: `View(producto)` usa el objeto en el servidor para producir HTML. En el reto `Ok(new { ... })` sí se serializa un objeto a JSON.

Aplicación integrada: observar una acción y su resultado

Activación incremental y regresión.

El proyecto del capítulo conserva la funcionalidad de las prácticas anteriores. En cada etapa activamos un bloque de código marcado, compilamos, iniciamos y comprobamos un resultado nuevo. No desactivamos el listado al habilitar `Estado` ni eliminamos la ficha al añadir `Ficha`.

Esto permite enseñar el desarrollo incremental con un criterio verificable: **la nueva capacidad aparece y las anteriores permanecen operativas**. Cuando activamos el reto, además de comprobar JSON 200, debemos confirmar que `/Productos` continúa mostrando sus tres elementos y que `/Productos/Detalle/2` sigue ofreciendo HTML.

Cada estado del ejercicio se puede construir y observar desde su punto de partida. El estudiante comprobará tanto la nueva capacidad como la continuidad de las rutas anteriores.

Con estos conceptos, el punto 1.5 podrá centrarse en enriquecer la presentación Razor sin volver a explicar desde cero cómo se obtiene el producto ni cómo se selecciona una acción.

Un recorrido narrado de extremo a extremo.

Consideremos la solicitud `GET /Productos/Detalle/2` en la versión final del capítulo. El navegador construye una petición HTTP dirigida al proceso local. La infraestructura ASP.NET Core recibe la solicitud y utiliza los endpoints MVC registrados para seleccionar `ProductosController.Detalle(int id)`.

El valor 2 del tercer segmento se enlaza con el parámetro del método. La acción comprueba que es positivo, obtiene la colección de ejemplo y localiza el producto cuyo identificador es 2. El resultado es el objeto Lápiz, con su nombre y precio.

Como el producto existe, el método devuelve `View(producto)`. La infraestructura MVC localiza `Views/Productos/Detalle.cshtml`, entrega el modelo y representa el HTML. El layout incorpora la navegación y las hojas de estilo compartidas. Finalmente, Kestrel envía la respuesta HTTP al navegador, que representa el contenido.

Si repetimos el recorrido para `/Productos/Detalle/99`, los pasos cambian a partir de la búsqueda: no existe el producto, por lo que la acción devuelve `NotFound()`. No hay una ficha de Lápiz ni un modelo válido que representar. La ruta fue correcta, pero el resultado de aplicación es distinto.

Si solicitamos `/catalogo/ficha/2`, la selección inicial del endpoint ocurre mediante un atributo de ruta, no mediante la forma convencional. Después, la acción elige explícitamente la vista `Detalle` para representar el mismo producto. Esto explica por qué un cambio en la ruta no obliga a duplicar el HTML de la ficha.

Reconocer los límites del recorrido.

Este ejemplo no realiza consultas a una base de datos ni autentica al usuario. Tampoco depende de un servicio externo. El flujo es suficientemente completo para enseñar MVC, rutas, resultados y Razor, pero sigue siendo una aplicación didáctica de consulta.

Cuando incorporemos EF Core en M02, la forma de obtener los datos cambiará. La idea general de recibir una solicitud, validar su entrada, localizar el recurso y devolver un resultado apropiado continuará siendo útil.

Criterio de dominio del punto 1.4.

Para considerar aprendido el capítulo, el alumno debería poder responder sin consultar una solución:

¿Por qué `/Productos/Detalle/2` devuelve HTML mientras `/Productos/Estado` devuelve texto? ¿Qué diferencia existe entre una URL que no coincide con ninguna ruta y una acción que devuelve

`NotFound()`? ¿Por qué `Ficha` puede reutilizar `Detalle.cshtml`? ¿Qué significa `{id:int}` y qué no significa? ¿Qué parte del código crea los datos de ejemplo? ¿Por qué el reto JSON no necesita una nueva vista?

Cada respuesta debe hacer referencia a un archivo o línea real del proyecto, no a una definición memorizada. El éxito consiste en relacionar el recorrido HTTP, la acción ejecutada, el dato utilizado y el resultado devuelto.

El capítulo 1.5 reutilizará esos resultados de detalle para profundizar en cómo Razor compone la presentación. Por eso es importante cerrar antes esta distinción: **las rutas seleccionan acciones, las acciones eligen resultados y las vistas representan datos**.

Relación con P1.4 y continuidad.

La práctica comienza inspeccionando el controlador existente y la lista compartida; activa `Estado()` y observa texto plano; activa `Detalle(int id)` y comprueba 200/400/404; incorpora una ruta por atributos; añade enlaces por producto en la vista; y realiza el reto JSON, igualmente comentado en el proyecto.

La práctica permite activar los cambios sucesivamente y observar sus respuestas HTTP: primero la ruta de estado, luego el detalle y sus errores, después la ficha alternativa y finalmente el resultado JSON. Después de cada avance, el alumno debe comprobar también que la portada, el catálogo y los recursos que ya funcionaban continúan accesibles.

El capítulo siguiente, **1.5**, profundizará en la sintaxis Razor, la composición de vistas, los layouts y las vistas parciales usando las páginas y el detalle funcional de este capítulo.

Fuentes y referencias.

- Microsoft Learn, enrutamiento a acciones de controlador: https://learn.microsoft.com/en-us/aspnet/core/mvc/controllers/routing?view=aspnetcore-8.0
- Microsoft Learn, solicitudes mediante controladores: https://learn.microsoft.com/en-us/aspnet/core/mvc/controllers/actions?view=aspnetcore-8.0
- Microsoft Learn, resultados de acciones de controlador: https://learn.microsoft.com/en-us/aspnet/core/web-api/action-return-types?view=aspnetcore-8.0
- Microsoft Learn, vistas Razor y tipado de modelos: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/overview?view=aspnetcore-8.0

1.5. Vistas Razor, modelos tipados, layouts y vistas parciales

Objetivos y contexto.

En el punto 1.4 construimos una acción de detalle de producto, comprobamos distintos resultados HTTP y añadimos enlaces desde el listado. Ahora el foco se traslada a **cómo se produce y se organiza el HTML de una aplicación ASP.NET Core MVC**.

Al terminar, el alumno podrá interpretar las directivas y expresiones básicas de Razor, comparar una vista de colección con una vista de objeto individual, utilizar un layout común, reconocer una sección opcional, incorporar una vista parcial tipada y reutilizarla desde dos vistas diferentes. También podrá explicar por qué la misma regla de presentación cambia el resultado de distintas páginas sin alterar los controladores.

No adelantaremos los formularios y los Tag Helpers de entrada de datos de M02. El dominio sigue siendo un pequeño catálogo de tres productos y no necesita una base de datos.

Razor y modelos fuertemente tipados

Situación de partida: las acciones ya seleccionan correctamente un producto; falta mantener su presentación.

Tras P1.4 tenemos un listado, una ficha, varias formas de llegar a ella y resultados HTTP diferentes. Sin embargo, el HTML que representa un producto puede repetirse. Si cambiamos cómo aparece el precio, no debería ser necesario localizar y corregir cada copia del mismo fragmento.

El capítulo avanza desde el dato hasta la composición de la página. Primero distinguiremos el tipo declarado mediante `@model` y el objeto que recibe `Model`; después utilizaremos una condición Razor para decidir qué contenido aparece. A continuación estudiaremos el layout como marco compartido y las secciones como una aportación específica de una página. Finalmente construiremos una tarjeta en una vista parcial para que **la misma representación de producto** pueda utilizarse en el catálogo y en el detalle.

La práctica P1.5 conserva los datos en memoria y no introduce formularios. Su pregunta central es esta: **si una regla visual de precio debe aplicarse en varias pantallas, ¿en qué archivo conviene escribirla para mantener una sola decisión?** La respuesta se comprobará en el HTML de ambas vistas, no solamente viendo si la aplicación compila.

Qué es Razor y dónde se ejecuta.

Razor es la sintaxis de plantillas de ASP.NET Core que combina HTML con expresiones y bloques de C#. En MVC, la infraestructura de vistas procesa los archivos `.cshtml` **en el servidor**, usando el modelo que la acción entrega, y genera el HTML de la respuesta HTTP.

No es un lenguaje que el navegador interprete. Cuando el navegador muestra «Lápiz», no está ejecutando `@Model.Nombre`: esa expresión ya se ha evaluado en el servidor. En las herramientas de desarrollo veremos el resultado HTML, no el archivo Razor original como código ejecutable.

Ejemplos básicos:

```
@model CursoWeb.Mvc.Models.Producto
```

```
<h2>@Model.Nombre</h2>
<p>Precio: @Model.Precio.ToString("0.00") €</p>
```

La primera línea es una **directiva Razor** y no genera por sí misma un elemento HTML. Declara que la vista recibirá un objeto `Producto` como modelo. Las expresiones `@Model.Nombre` y `@Model.Precio.ToString("0.00")` insertan valores en el marcado. El resultado de una petición de detalle con `id=2` contiene un encabezado con el nombre del producto y un párrafo con el precio.

El símbolo `@` tiene diferentes usos según el contexto. Es importante diferenciarlos antes de comenzar a copiar ejemplos:

Forma	Qué representa	Ejemplo
`@model`	Directiva que declara el tipo de modelo	`@model Producto`
`@{ ... }`	Bloque de código C#	`@{ ViewData["Title"] = "Catálogo"; }`
`@Model.Nombre`	Expresión cuyo valor se escribe en HTML	`<h3>@Model.Nombre</h3>`
`@if (...) { ... }`	Condición Razor que decide qué fragmento HTML se genera	Mensaje de catálogo amplio
`@foreach (...) { ... }`	Iteración que puede generar elementos HTML repetidos	Tarjetas de productos
`@* ... *@`	Comentario Razor que no se representa en el HTML final	Bloques que activamos en prácticas

Como en cualquier plantilla, **la estructura importa**. El C# dentro de `@if` o `@foreach` puede utilizarse para controlar el marcado generado, pero no conviene trasladar a las vistas las consultas a bases de datos o las reglas empresariales del sistema. Las vistas deben centrarse en la presentación.

HTML codificado de forma segura.

Cuando Razor genera contenido desde una expresión ordinaria, codifica caracteres que podrían interpretarse como HTML. Esta protección es relevante para mostrar nombres escritos por usuarios, aunque todavía utilicemos datos fijos. Evitaremos utilizar `Html.Raw` sobre texto no confiable. El hecho de que en la respuesta aparezca una entidad HTML para una letra acentuada no implica que se haya perdido el texto: el navegador la representa correctamente.

Una vista de lista y una vista de detalle no reciben el mismo modelo.

El proyecto utiliza dos vistas tipadas. La vista del catálogo declara:

```
@model IReadOnlyList<CursoWeb.Mvc.Models.Producto>?
```

Esto indica que espera **una colección de productos** y que el modelo podría ser nulo. Su estructura actual comienza con una condición para ese caso y, cuando hay lista, muestra la cantidad y los productos:

```
@if (Model is null)
{
    <p id="sin-modelo">El controlador no ha proporcionado productos.</p>
}
else
{
    <p id="numero-productos">
        El controlador ha enviado <strong>@Model.Count</strong> productos.
```

```
    </p>
}
```

La página de detalle, por su parte, declara:

```
@model CursoWeb.Mvc.Models.Producto
```

y consulta `@Model.Id`, `@Model.Nombre` y `@Model.Precio`. Para una acción `Detalle(int id)` que entrega un único objeto, la vista no necesita un `foreach` sobre una colección.

No debemos intercambiar los contratos: una vista que declara `Producto` no puede tratar automáticamente su modelo como una lista. El contrato tipado facilita comprender el código y detecta errores de acceso a propiedades. Una vista también puede utilizar un *view model* distinto del tipo de dominio; por ahora emplearemos `Producto` directamente, porque la práctica se concentra en la sintaxis Razor y no en arquitecturas de capas.

Una condición de presentación adicional.

En el proyecto 1.5 existe un bloque comentado del listado:

```
@if (Model.Count >= 3)
{
    <p id="catalogo-completo">El catálogo contiene al menos tres productos.</p>
}
```

Se activa **dentro del `else` que garantiza que `Model` no es nulo**. La condición consulta los datos existentes y decide si mostrar un mensaje: no altera el catálogo ni recalcula precios, y tampoco introduce persistencia.

Cuando el alumnado lo habilita en P1.5.2, comprobará que el contador sigue mostrando tres y se añade un texto nuevo. Esta separación entre datos y formato será la base de la vista parcial que aparecerá después.

El proceso de representación Razor visto desde dentro.

Razor permite escribir una plantilla legible que combina HTML y C#. Sin embargo, para utilizar correctamente las vistas MVC debemos distinguir el archivo de plantilla, el modelo que recibe y la respuesta finalmente enviada al navegador. La vista no es una copia de HTML estático. Durante la representación, la plataforma ejecuta expresiones y bloques Razor para generar el documento resultante.

En nuestro catálogo, el controlador `ProductosController.Index()` devuelve una colección de objetos `Producto`. La vista `Views/Productos/Index.cshtml` recibe esa colección como `Model` y produce marcado a partir de los objetos. El layout proporciona la estructura HTML exterior. Si activamos una vista parcial, su contenido se incorpora como otro fragmento dentro de la misma respuesta.

Este proceso puede observarse sin necesidad de conocer los detalles internos del compilador de Razor. Abrimos una vista `.cshtml` y vemos expresiones como `@Model.Count`. Después abrimos el navegador y, mediante las herramientas de desarrollo, examinamos el HTML resultante. Allí aparece un número y no la expresión Razor que lo produjo.

Separar el código fuente del contenido entregado.

Por ejemplo, la plantilla contiene:

```
<h3>@Model.Nombre</h3>
```

Para un producto cuyo nombre es Lápiz, el resultado conceptual es:

```
<h3>Lápiz</h3>
```

El contenido de la respuesta puede codificar algunos caracteres de manera equivalente mediante entidades HTML; eso no cambia la propiedad de que el navegador presenta el texto del producto. La transformación se realiza en el servidor antes de enviar el resultado.

De la misma manera, un bucle:

```
@foreach (var producto in Model)
{
    <li>@producto.Nombre</li>
}
```

no crea una etiqueta `<li>` que contenga código C# visible en el cliente. Produce varios elementos HTML, tantos como objetos recorra la colección. En nuestro caso hay tres productos, de modo que pueden generarse tres elementos.

Esta observación constituye una de las diferencias más importantes entre aprender sintaxis Razor y limitarse a copiar una plantilla. El alumnado debe poder explicar no solo **qué se escribe**, sino **qué fragmento de respuesta genera cada instrucción**.

La directiva @model como contrato fuerte.

En el catálogo hemos utilizado:

En `Views/Productos/Index.cshtml`, el contrato es `@model IReadOnlyList<CursoWeb.Mvc.Models.Producto>?`: se recibe una colección que puede ser nula.

En la ficha de detalle y en la parcial utilizamos:

En `Views/Productos/Detalle.cshtml` y en `Views/Shared/_TarjetaProducto.cshtml`, el contrato es `@model CursoWeb.Mvc.Models.Producto`: ambas representaciones trabajan con un único producto.

La diferencia responde a la estructura de los datos. El listado recorre una colección; la ficha y la tarjeta trabajan con un único producto.

Cuando una vista declara el tipo de su modelo, las expresiones C# pueden acceder a las propiedades conocidas para ese tipo. La lista tiene `Count` y permite iterar elementos, mientras que el objeto `Producto` contiene `Id`, `Nombre` y `Precio`. No debemos intentar consultar `Model.Nombre` cuando `Model` representa una lista ni llamar a `Model.Count` cuando es un producto individual.

Contrato de la vista parcial frente al contrato de la vista principal.

En `Index.cshtml`, durante el `foreach`, la variable `producto` tiene el tipo `Producto`. Al pasarla a la parcial con `model="producto"`, el archivo `_TarjetaProducto.cshtml` puede declarar `@model Producto`. No necesita recibir toda la colección porque su objetivo es representar únicamente una tarjeta.

En `Detalle.cshtml`, `Model` ya es un `Producto`. Podemos enviarlo a la misma parcial utilizando `model="Model"`. La parcial no necesita conocer si su dato procede de una lista, una ficha o cualquier otro origen compatible: recibe el contrato que necesita y genera su HTML.

Esta separación hace que una misma representación pueda reutilizarse en diferentes contextos sin introducir nuevas clases de controlador ni copiar el mismo marcado.

Por qué el código de presentación puede contener condiciones.

Una vista Razor no es un archivo de HTML puro. La directiva `@if` permite decidir si aparece una sección, y `@foreach` permite repetir estructuras. Este comportamiento es necesario para construir páginas basadas en datos, pero exige criterio para no trasladar al archivo de vista decisiones que pertenecen al dominio.

En el capítulo preparamos una condición de presentación:

El bloque de la vista del catálogo empieza con `@if (Model.Count >= 3)` dentro de la rama en la que el modelo no es nulo; únicamente decide si mostrar un mensaje descriptivo.

Su única responsabilidad es mostrar un mensaje cuando la lista contiene al menos tres elementos. No cambia la colección ni aplica ninguna regla comercial. Además, la situamos dentro del `else` de una comprobación anterior, de forma que sabemos que `Model` no es nulo cuando accedemos a `Count`.

En el reto, una condición de precio aparece dentro de la parcial:

```
@if (Model.Precio < 3m)
{
    <strong class="etiqueta-economico">Precio económico</strong>
}
```

El precio es un valor decimal del modelo y `3m` es un literal decimal de C#. La etiqueta se añade únicamente a la representación de productos que cumplen el criterio. En nuestro conjunto de datos, Lápiz cuesta 1,20 €, mientras que Cuaderno y Carpeta superan el umbral.

Cuándo convendría trasladar una decisión fuera de Razor.

Si la condición pasara a determinar quién puede comprar un producto, qué impuestos se aplican o qué precio final debe persistirse, estaríamos en un ámbito de lógica empresarial. Sería peligroso mantener esa regla solo en el HTML de una parcial porque podría necesitarse también en otras interfaces y servicios.

La decisión `Precio < 3m` del reto tiene un alcance explícitamente didáctico y visual. No ofrece descuentos reales ni determina la disponibilidad del producto. La mantenemos en Razor porque el ejercicio consiste en demostrar **la reutilización de una regla simple de presentación**.

Composición mediante layout y secciones

El layout: la estructura compartida de la página.

Si cada vista copiara su propio `<html>`, `<head>`, navegación, hojas CSS y pie, un cambio de diseño tendría que repetirse en todas las páginas. En MVC, el **layout** permite reunir esta estructura común.

Nuestro archivo `Views/Shared/_Layout.cshtml` contiene elementos como:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8" />
    <title>@ViewData["Title"] - Curso Web .NET 8</title>
</head>
<body>
    <nav><a href="/">Inicio</a><a href="/Productos">Productos</a></nav>
    <main>
        @RenderBody()
    </main>
    <footer>Curso ASP.NET Core MVC · .NET 8</footer>
</body>
</html>
```

Esta es una representación abreviada para explicar la idea; el archivo real incluye sus hojas de estilo, clases y otros enlaces. La instrucción `@RenderBody()` es especialmente importante: indica dónde se insertará el contenido de la vista concreta que devuelve la acción MVC.

El archivo `Views/_ViewStart.cshtml` establece:

```
@{
    Layout = "_Layout";
}
```

Por eso las vistas completas comparten el layout sin repetir en cada archivo la instrucción de asignación. El archivo `_ViewImports.cshtml`, en cambio, aporta directivas compartidas y el registro de Tag Helpers:

```
@using CursoWeb.Mvc
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Conviene evitar una confusión habitual: `_ViewStart.cshtml` interviene en las **vistas completas**, pero **no se ejecuta al representar una vista parcial**. Una parcial incluida dentro de otra página no introduce por sí sola un segundo layout. Esta distinción se volverá visible cuando añadamos tarjetas en 1.5.4.

Títulos que dependen de cada vista.

Cada página puede indicar un valor mediante `ViewData["Title"]`:

```
@{
    ViewData["Title"] = "Detalle del producto";
}
```

La vista proporciona un valor y el layout lo utiliza dentro de `<title>`. Así el título del navegador cambia sin tener que crear varios layouts. Para datos más complejos, el modelo tipado es la opción principal; aquí `ViewData` se usa únicamente como mecanismo sencillo de comunicación de presentación con la estructura común.

Secciones de un layout: contenido opcional que depende de la página.

Un layout también puede reservar una posición para contenido adicional que solo algunas vistas definen, mediante una **sección Razor**. Nuestro proyecto prepara este bloque comentado en el layout:

```
<aside id="notas-del-layout" aria-label="Notas de la página">
    @await RenderSectionAsync("NotasPagina", required: false)
</aside>
```

La instrucción `RenderSectionAsync("NotasPagina", required: false)` indica una posición para una sección de ese nombre y declara que **es opcional**. Como no es obligatoria, la portada y la lista no tienen que definirla.

En `Views/Productos/Detalle.cshtml`, otro bloque comentado permite definir:

```
@section NotasPagina
{
    <p id="nota-pagina-detalle">Detalle construido con una vista Razor tipada.</p>
}
```

Al activar ambos bloques, la página de detalle escribe su mensaje en el lugar dispuesto por el layout, y las demás vistas siguen funcionando. Esta es una forma de mantener la estructura compartida y permitir a cada página completar determinadas zonas.

Regla importante: declarar una sección no equivale a renderizarla en el mismo archivo. El layout debe renderizarla o ignorarla. En este ejercicio siempre activaremos **los dos bloques del paso 1.5.3**, de manera que la sección declarada en el detalle tiene un destino en el layout. El propósito es enseñar una combinación válida y reproducible; no se pide al alumno que active solo una de las dos mitades.

Una sección no debe confundirse con una vista parcial. La sección es contenido definido por la vista para ser colocado en una zona prevista del layout. Una parcial, en cambio, es un **archivo de marcado reutilizable** que una vista puede representar pasándole un modelo.

Qué aporta el layout y qué no aporta.

El layout define la estructura común de una página MVC: documento HTML, cabecera, navegación, hojas de estilo, zona principal y pie. Esto evita repetir un mismo bloque de marcado en todas las vistas.

En el proyecto, `Views/_ViewStart.cshtml` selecciona `_Layout`:

En `Views/_ViewStart.cshtml` se indica `Layout = "_Layout";` ; así las vistas usan de manera predeterminada `Views/Shared/_Layout.cshtml`.

Después, cada vista concreta define el contenido que se insertará en `@RenderBody()`. El título del documento también puede variar mediante `ViewData["Title"]`, permitiendo que la pestaña del navegador refleje si estamos en el catálogo o en una ficha individual.

Si un alumno añadiera manualmente un segundo `<html>` completo dentro de `Index.cshtml`, estaría duplicando responsabilidades del layout. La composición correcta consiste en dejar que la vista se concentre en su contenido y que el layout aporte la estructura exterior.

El layout no sirve para recuperar los datos del catálogo ni sustituye al controlador. Puede presentar elementos compartidos y utilizar ciertos valores de presentación, pero no debemos hacer depender la lógica de negocio de cómo esté estructurada la cabecera del sitio.

El ciclo de vida de una sección opcional.

Una sección Razor permite que una vista defina contenido destinado a una región específica del layout. Nuestro ejercicio incorpora dos bloques que deben entenderse conjuntamente.

En el layout:

En el layout compartido se reserva el lugar mediante `@RenderSection("Notas", required: false)`, una sección opcional que no obliga a cada vista a definir contenido adicional.

En la vista de detalle:

La vista `Detalle.cshtml` aporta su propio bloque `@section Notas { ... }`. La vista del catálogo no lo aporta, y por eso ambas funcionan con el mismo layout sin crear contenido vacío.

La vista declara la sección y el layout define dónde se representa. Como el parámetro `required` es `false`, otras vistas que no declaran esa sección pueden seguir utilizándolo sin tener que añadir texto vacío.

Qué demuestra la comparación con el catálogo.

La ficha de Lápiz incluye una sección de notas y el catálogo no la declara. Si el layout está correctamente preparado, ambos recorridos siguen funcionando. Eso es más informativo que comprobar únicamente que aparece una etiqueta `<aside>`: confirma que la nueva posibilidad de personalizar una página no obliga a alterar todas las demás.

El alumno también debe comprender por qué el código se activa en ambos archivos en el mismo paso. Si se declara una sección en una vista sin que exista un mecanismo apropiado para representarla o

ignorarla, el motor de vistas puede detectar una sección sin consumir. La práctica evita deliberadamente dejar el proyecto en ese estado intermedio.

Las secciones pertenecen a la composición de una vista completa con su layout. No constituyen un mecanismo general para enviar contenido desde cualquier vista parcial. Por eso el mensaje `NotasPagina` se define en `Detalle.cshtml` y no en `_TarjetaProducto.cshtml`.

Puente práctico: centralizar una decisión que aparece dos veces

En `Views/Productos/Index.cshtml` recorremos una colección; en `Views/Productos/Detalle.cshtml` recibimos un producto individual. Si ambas páginas necesitan la misma tarjeta, escribir su marcado dos veces puede parecer rápido, pero convierte cada cambio visual en dos tareas de mantenimiento. Un cambio aplicado solo al listado dejaría la ficha de detalle con una presentación distinta.

La vista parcial `_TarjetaProducto.cshtml` recibe un `Producto` por su propio contrato `@model`. Esto significa que puede leer `Model.Nombre` y `Model.Precio` **sin saber de qué página fue invocada**. El listado entrega cada elemento de la colección; el detalle entrega su propio objeto. La parcial solo transforma ese producto en un fragmento de HTML.

Lo mismo ocurre con las decisiones de presentación. Si queremos señalar los productos de precio elevado o económico, podemos evaluar una condición `@if` dentro de la tarjeta. Colocarla en la parcial evita repetir una condición en ambas páginas; colocarla solamente en el layout sería inadecuado porque ese marco compartido no recibe de forma natural el producto de cada tarjeta.

En P1.5 el alumno escribirá una regla nueva con un umbral de 4 € para comprobar el efecto tanto en el catálogo como en el detalle. El reto del mismo punto usa un umbral distinto para el precio económico. Ambas actividades tienen sentido porque obligan a **razonar la ubicación de la regla** y contrastar el resultado en más de un contexto, no porque sea complejo escribir una comparación decimal.

Vistas parciales y reutilización

Vistas parciales: evitar repetir marcado.

Supongamos que el mismo producto debe presentarse como tarjeta tanto en el catálogo como dentro de su propia ficha. Podríamos copiar el mismo `<article>` en ambos archivos, pero entonces cualquier cambio de presentación tendría que repetirse. Para evitarlo utilizaremos una **vista parcial**.

Una vista parcial es un archivo Razor `.cshtml` que genera un fragmento HTML para ser incluido dentro de otra vista. No requiere tener su propia acción ni dispone automáticamente de una página o layout independiente. Puede declarar su propio `@model` para trabajar con un tipo concreto.

Nuestro proyecto incorpora `Views/Shared/_TarjetaProducto.cshtml`, inicialmente así:

```
@model CursoWeb.Mvc.Models.Producto

<article class="tarjeta-producto" data-producto-id="@Model.Id">
    <h3>@Model.Nombre</h3>
    <p class="tarjeta-precio">Precio: @Model.Precio.ToString("0.00") €</p>
    <a href="/Productos/Detalle/@Model.Id">Ver detalle de @Model.Nombre</a>
</article>
```

El archivo real añade comentarios docentes y el bloque opcional del reto. La parcial recibe un solo `Producto`, presenta su nombre y precio y produce un enlace de detalle. Como se encuentra en `Views/Shared`, puede localizarse desde las vistas del catálogo y de la ficha por las convenciones de búsqueda de vistas.

Llamar a una parcial con el Partial Tag Helper.

Para representar la tarjeta dentro de la página de listado utilizamos:

```
@foreach (var producto in Model)
{
    <partial name="_TarjetaProducto" model="producto" />
}
```

`<partial>` es un **Tag Helper**, habilitado por `_ViewImports.cshtml`. Su atributo `name` indica el nombre de la parcial; `model="producto"` le entrega el objeto actual de la iteración. El código Razor del archivo parcial accede a ese objeto como `Model`.

El Tag Helper construye el fragmento de HTML de forma asíncrona, sin que el alumnado tenga que gestionar ese detalle aquí. No es un componente Blazor ni crea interactividad por sí mismo. En M03 compararemos los componentes Razor de Blazor con las vistas de MVC, para no confundir conceptos que utilizan nombres y archivos parecidos.

Resultado: en el catálogo aparecen **tres tarjetas**, una por producto. La página ya mostraba un listado sencillo; activamos otra representación deliberadamente para poder comparar ambas y observar el uso de la parcial sin borrar el ejercicio anterior.

Reutilizar la misma parcial en el detalle.

En `Views/Productos/Detalle.cshtml` se activa más adelante:

```
<section id="tarjeta-detalle" aria-label="Ficha presentada como tarjeta">
    <h2>Tarjeta reutilizada en el detalle</h2>
    <partial name="_TarjetaProducto" model="Model" />
</section>
```

Aquí `Model` es un único objeto `Producto` y se pasa directamente. La misma vista parcial produce una tarjeta, sin duplicar el código en el archivo de detalle.

La diferencia entre las dos invocaciones es intencionada:

- `model="producto"` recibe el objeto actual de un `foreach` del catálogo.
- `model="Model"` recibe el objeto completo de la vista de detalle.

El atributo `model` no debe confundirse con la directiva `@model`. El primero es un atributo del Tag Helper que pasa un objeto; el segundo declara el tipo esperado por el archivo Razor que va a representarlo.

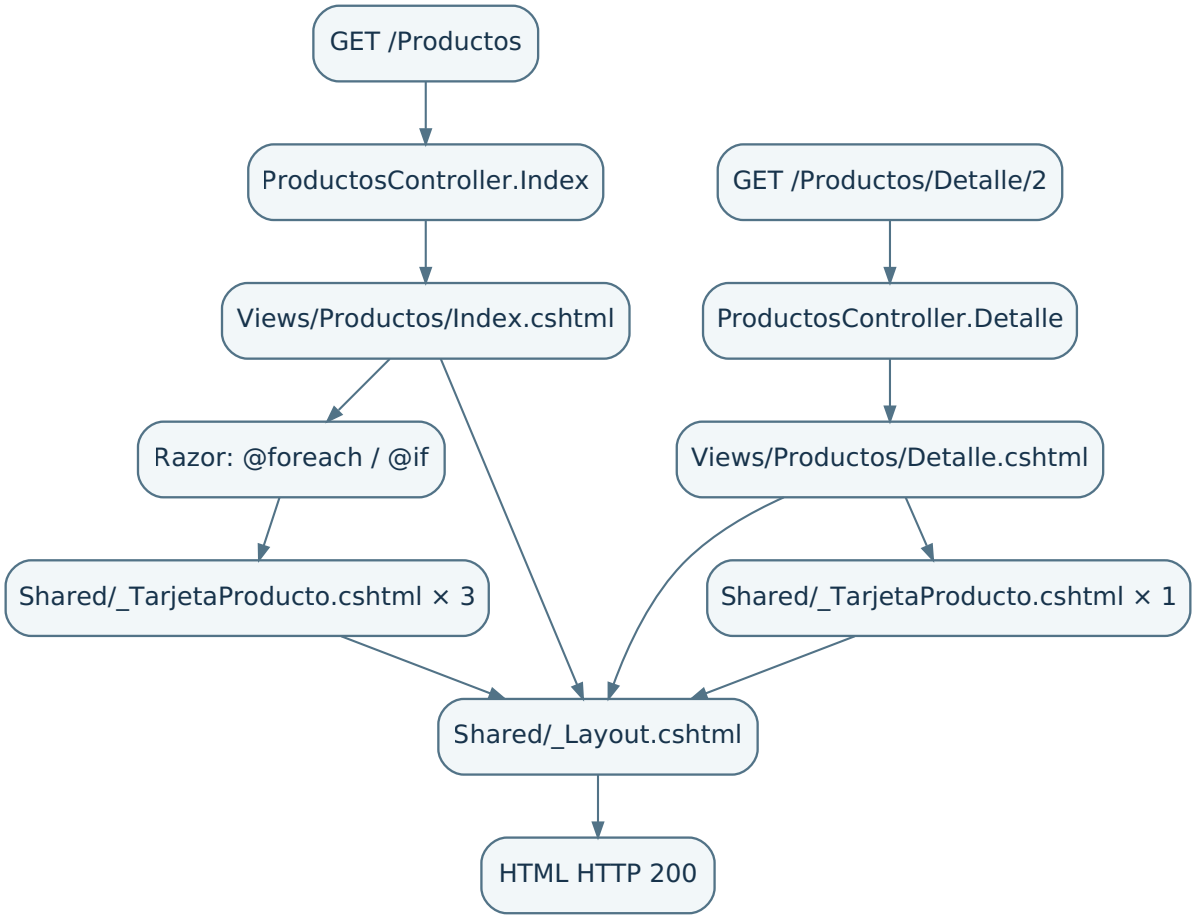
Qué no debe hacer una parcial.

Una vista parcial resulta apropiada para repetir marcado y lógica **simple de presentación**. No conviene ocultar dentro de ella acceso a una base de datos ni usarla como solución universal para construir páginas complejas. En esta práctica, la parcial **recibe datos**, y el controlador sigue siendo el que obtiene los productos.

Si se necesita lógica propia compleja para preparar un fragmento reutilizable, ASP.NET Core dispone también de otros mecanismos, como View Components. No forman parte del alcance de este capítulo.

Flujo de composición de una respuesta HTML.

Los elementos del ejercicio se relacionan de la siguiente manera:



El diagrama representa la **composición conceptual** del HTML. El motor Razor coordina la ejecución de vistas y parciales; no son servicios HTTP independientes ni se ejecuta un controlador distinto por cada tarjeta. La petición al navegador produce una respuesta HTML que contiene todos los fragmentos.

Cuando la página de detalle define la sección `NotasPagina`, el layout también coloca su contenido en la zona `RenderSectionAsync`. La parcial no necesita conocer esa sección ni declararla.

Extraer una estructura repetida a una parcial.

Supongamos que cada producto debe mostrar su nombre, precio y enlace. Podríamos copiar ese HTML en `Index.cshtml` y en `Detalle.cshtml`, pero la duplicación introduciría un coste de mantenimiento: cualquier modificación del formato debería reproducirse en ambos archivos.

La parcial `_TarjetaProducto.cshtml` concentra ese marcado:

La tarjeta se encuentra en `Views/Shared/_TarjetaProducto.cshtml`: concentra el nombre, el precio y el enlace de un único `Producto` para ser representada tanto en el listado como en la ficha.

Ahora la vista de listado necesita solamente iterar los productos y pasar uno a uno a la parcial. La vista de detalle puede pasar directamente su único `Model`.

La ventaja no es únicamente escribir menos líneas. Al existir **una sola implementación del marcado de tarjeta**, corregir una etiqueta o modificar un texto cambia todos los lugares que utilizan esa parcial. El reto de precio económico ofrece una demostración concreta: añadimos la condición en un archivo y la observamos en dos páginas.

Por qué la parcial está en Shared.

`Views/Shared` es una ubicación convencional para componentes de presentación reutilizables entre diferentes vistas. Si la tarjeta perteneciera exclusivamente a una pantalla del catálogo, podríamos plantearnos una ubicación más cercana a esa vista. En nuestro ejemplo es compartida por dos vistas, así que el nombre `_TarjetaProducto.cshtml` y su ubicación en `Shared` comunican su propósito.

El prefijo de guion bajo es una convención habitual de archivos de infraestructura o fragmentos Razor. No convierte la parcial en una acción privada de controlador ni proporciona por sí solo mecanismos de seguridad. La diferencia relevante es que se procesa como fragmento dentro de otra vista.

Un paso adicional: Anchor Tag Helper y la relación entre vistas y routing.

Una vez que hemos construido una tarjeta reutilizable, surge otra cuestión de mantenimiento: ¿debe la tarjeta ensamblar siempre manualmente la URL del producto? En el proyecto inicial el enlace se escribe con un `href` que incorpora `@Model.Id`. Es sencillo y suficiente para el primer ejercicio, pero ASP.NET Core MVC también ofrece el *Anchor Tag Helper*, que puede generar la URL a partir del controlador, la acción y el valor del parámetro de ruta.

Podemos describir esa intención directamente en el HTML fuente de la tarjeta:

```
<a asp-controller="Productos"
   asp-action="Detalle"
   asp-route-id="@Model.Id">Ver detalle de @Model.Nombre</a>
```

Cuando la aplicación representa la vista y el Tag Helper está habilitado mediante `@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers`, el servidor procesa los atributos `asp-controller`, `asp-action` y `asp-route-id`. El resultado enviado al navegador contiene un enlace HTML ordinario con `href`; esos atributos especiales no son un protocolo de navegación que tenga que entender el navegador.

Este ejemplo enlaza dos conocimientos previos. Primero, **la parcial recibe un solo `Producto`**: por tanto, `Model.Id` se evalúa según el elemento actual y no necesita repetir valores como 1 o 2. Segundo, **las rutas seleccionan acciones**: el enlace generado conducirá a una nueva solicitud `GET`, que MVC encaminará hacia el controlador y la acción indicados. No hay acceso a datos dentro del Tag Helper ni se ejecuta `Detalle` cuando se genera el enlace; eso sucederá cuando alguien lo siga.

La versión inicial de la parcial mantiene un `href` convencional para que el alumno pueda observar la transición entre ambas formas. En la integración final de P1.5 se le pide sustituirlo, comprobar el HTML resultante y razonar por qué el catálogo y la ficha siguen utilizando un único bloque de presentación. Esta introducción no pretende adelantar la configuración de formularios del M02; solamente muestra que el framework puede relacionar la navegación con las acciones MVC a partir de atributos declarativos.

El Partial Tag Helper: qué significa cada atributo.

La invocación:

```
<partial name="_TarjetaProducto" model="producto" />
```

contiene dos atributos con funciones distintas. `name` indica qué archivo de parcial se busca; `model` suministra el objeto que el archivo recibirá como su `Model`.

El Tag Helper se habilita mediante una directiva de `_ViewImports.cshtml`:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Esto significa que la etiqueta `<partial>` no debe interpretarse como una etiqueta HTML arbitraria que el navegador tenga que entender. MVC reconoce el Tag Helper durante la representación de la vista y sustituye su uso por el contenido HTML de la parcial.

El elemento produce un **fragmento dentro de una respuesta**. No abre una conexión HTTP adicional por cada tarjeta, no genera un controlador separado y no crea automáticamente un layout por tarjeta. En el listado hay tres objetos; por tanto, el `foreach` hace tres invocaciones de la parcial, incorporando tres fragmentos en el HTML que recibe el navegador.

Diferenciar Partial Tag Helper, sección y componente Blazor.

Los tres conceptos pueden aparecer en archivos con sintaxis Razor, pero cumplen funciones distintas.

Una **vista parcial MVC** es un fragmento de presentación reutilizable que se incluye dentro de otra vista. Una **sección de layout** permite que una vista concreta aporte contenido a una posición definida en el layout. Un **componente Blazor** pertenece a otro modelo de construcción de interfaces y puede manejar parámetros, eventos y estado según su modo de renderizado.

En M03 conoceremos el modelo de componentes de Blazor. En M01 evitamos llamar «componente interactivo» a la vista parcial porque eso crearía expectativas incorrectas: nuestra tarjeta se genera del lado del servidor como HTML, y hacer clic en su enlace provoca una navegación HTTP convencional.

Diseño HTML, clases CSS e identificadores de prueba.

La parcial utiliza elementos semánticos como `<article>` y `<h3>`, y la vista del detalle utiliza una lista de definiciones con `<dl>`. La finalidad es aportar una estructura razonable para los datos, no enseñar un curso completo de CSS o accesibilidad en este apartado.

Las clases `tarjeta-producto` y `tarjeta-precio` permiten aplicar estilos coherentes a las tarjetas, tanto en el catálogo como en una ficha. La clase `etiqueta-economico` pertenece al fragmento que se activa en el reto y dispone de un estilo mínimo en la hoja del proyecto.

Los atributos como `data-producto-id` son útiles para inspeccionar qué producto se está representando. También permiten escribir verificaciones que distingan tres tarjetas diferentes en lugar de comprobar únicamente que aparece tres veces la palabra «producto».

El alumno debe evitar una solución frágil que identifique cada tarjeta solo por su posición visual. La representación está basada en un modelo que tiene `Id`, de modo que podemos comprobar su correspondencia con los objetos de la lista.

Representación visual frente a corrección funcional.

Que el HTML contenga tres elementos `article` no demuestra por sí solo que estén bien distribuidos en la pantalla o que el contraste de colores sea adecuado. Inspeccionar el HTML permite comprobar su estructura y sus datos, mientras que abrirlo en un navegador permite evaluar su disposición visual.

También ocurre lo contrario: tres tarjetas pueden parecer correctas a simple vista y esconder enlaces de detalle incorrectos. Por eso combinamos observación visual con comprobaciones de URLs, modelos y contenido HTML.

Problemas de mantenimiento que la parcial previene.

Imaginemos que el requisito cambia de «Precio» a «Importe» en la tarjeta. Con la parcial basta con modificar un archivo. El catálogo y la ficha utilizarían la nueva presentación, siempre que ambos sigan representando la misma parcial.

Si en lugar de ello tuviéramos dos copias del HTML, una podría seguir mostrando «Precio» y la otra «Importe». No sería un error de compilación, y una prueba que solo visitara el listado podría no detectar la incoherencia.

Otra situación similar aparece con los enlaces. Si decidimos cambiar la dirección de detalle de producto, conviene revisar todos los lugares que generan URLs. En nuestro curso el enlace de la tarjeta está centralizado en la parcial, mientras que el listado clásico conserva otro enlace para comparar formas de presentación. En un proyecto mayor habría que valorar si esas dos formas deben también compartir un mecanismo de generación de enlaces.

La reutilización es beneficiosa cuando varios lugares necesitan realmente el mismo marcado. No significa que deba extraerse absolutamente cada elemento HTML a su propio archivo parcial. Una separación excesiva puede hacer más difícil leer el conjunto de la vista.

Método de diagnóstico cuando una parcial falla.

Si al activar el bloque 1.5.4 no aparecen tarjetas, podemos seguir un diagnóstico ordenado.

Primero comprobamos que la página `/Productos` devuelve 200 y muestra el listado clásico. Si no funciona, el problema puede estar fuera de la nueva parcial y conviene recuperar el estado del capítulo anterior.

Segundo, verificamos que el bloque `PASO 1.5.4` ya no está dentro del comentario Razor. Los comentarios `@* ... *@` no producen HTML, así que una parcial correctamente escrita pero nunca invocada no aparecerá en la respuesta.

Tercero, comprobamos el nombre `_TarjetaProducto` y su ubicación bajo `Views/Shared`. Si el motor no encuentra la parcial, no puede sustituir la etiqueta por el fragmento correspondiente.

Cuarto, revisamos la directiva `@model Producto` y el objeto suministrado mediante `model="producto"`. Un error de tipado puede impedir representar correctamente el fragmento.

Quinto, observamos el HTML resultante y el CSS. Si las tarjetas están en el HTML pero no se aprecian visualmente, probablemente no se trata de una falta de datos ni de una ruta MVC ausente, sino de un problema de presentación.

Esta secuencia es más útil que cambiar aleatoriamente el controlador, el layout y los estilos sin una hipótesis clara.

Reto de presentación, pruebas y cierre

El reto: una condición de precio centralizada en la parcial.

El reto utiliza lo que acabamos de aprender: al modificar **una única vista parcial** debemos poder cambiar el HTML de la tarjeta en **dos páginas**.

Dentro de `Views/Shared/_TarjetaProducto.cshtml` existe un bloque de código comentado:

El bloque opcional de la parcial utiliza `@if (Model.Precio < 3m)` para mostrar «Precio económico». Al estar en la parcial, la condición se aplica en todas las vistas que la utilizan.

La condición compara el precio decimal del producto con tres unidades monetarias. El código del curso contiene estos valores: Cuaderno 4,50, Lápiz 1,20 y Carpeta 3,75. Por tanto, el mensaje aparece únicamente para **Lápiz**.

Esta expresión es apropiada en la parcial porque la usamos como una decisión **visual y local del ejercicio**; no estamos implementando una política real de descuentos ni alterando los precios. El umbral es ficticio y está explicado en el Markdown de prácticas.

Al activar el reto, la vista parcial añade la etiqueta a la tarjeta de Lápiz dentro del catálogo y dentro de `/Productos/Detalle/2`. La ficha de Cuaderno (`id=1`) no debe mostrarla. El archivo `Index.cshtml`, el archivo `Detalle.cshtml` y el controlador permanecen sin cambios adicionales.

Con ello se demuestra directamente el valor de la reutilización: **una modificación localizada produce un comportamiento coherente dondequiera que se represente la misma parcial**.

Errores habituales.

«Una parcial es una página que el navegador solicita por separado». En este caso, no. Se representa dentro de otra vista del servidor. No necesitamos crear una acción ni una ruta HTTP para `_TarjetaProducto.cshtml`.

«Si llamo a `<partial>` desde un `foreach` se ejecuta el layout por cada elemento». No. El layout es el envoltorio de la vista completa; la parcial no ejecuta `_ViewStart.cshtml` ni inserta otro documento HTML completo.

«Puedo usar cualquier propiedad en `@Model` sin importar el tipo». No. La directiva `@model` declara qué tipo de datos recibirá el archivo. El listado y el detalle tienen modelos diferentes; la parcial espera un `Producto` individual.

«La sección del detalle aparece automáticamente con solo definir `@section`». No. El layout debe contener un marcador de representación con el nombre correspondiente, o la vista debe atender de otra forma el contrato de secciones. Activaremos ambos bloques juntos.

«Razor permite escribir C# en las vistas, por tanto allí va toda la lógica». No. Razor facilita decisiones de presentación; las operaciones de dominio o persistencia deben separarse de esas decisiones visuales.

«Si la parcial tiene código comentado, sus etiquetas pueden aparecer en HTML». Los comentarios Razor `@* ... *@` no se envían en el cuerpo HTML. Al retirarlos correctamente, el bloque comienza a generar contenido.

«El Tag Helper `<partial>` es un componente Blazor». No. Ambos pueden compartir sintaxis Razor, pero las vistas parciales MVC se renderizan como fragmentos de una respuesta HTML; los componentes Blazor tienen otro modelo de ejecución que veremos en M03.

Aplicación integrada: una única decisión visual

El reto como prueba de reutilización real.

El reto pide que la parcial incorpore una etiqueta cuando `Precio < 3m`. No se trata simplemente de añadir una condición cualquiera: la localización de esa condición es la parte central del ejercicio.

Si el alumno escribiera una condición en `Index.cshtml` para mostrar el texto en el catálogo y otra en `Detalle.cshtml` para mostrarlo en la ficha, conseguiría un resultado parecido, pero perdería el objetivo didáctico: **centralizar una decisión de presentación en el marcado reutilizable**.

El proyecto ya contiene el código comentado dentro de la parcial, así que no es necesario inventar un archivo de solución ni escribir un nuevo controlador. Basta con retirar los delimitadores del comentario Razor correspondiente. Al hacerlo, el resultado se refleja en todos los lugares que utilizan la parcial.

Para los tres objetos de ejemplo debemos observar:

Contexto	Modelo suministrado	Etiquetas «Precio económico» esperadas
Catálogo de tres tarjetas	Cuaderno, Lápiz y Carpeta	1
Detalle de Lápiz	Solo Lápiz	1
Detalle de Cuaderno	Solo Cuaderno	0
Detalle de Carpeta	Solo Carpeta	0

Estas predicciones se obtienen comparando los precios del modelo con el umbral. La prueba debe comprobar también la ausencia de la etiqueta donde no corresponde, no únicamente su presencia en la ficha de Lápiz.

Estrategia de pruebas para una vista parcial.

Las pruebas del capítulo representan seis estados de la misma aplicación y verifican que cada nueva responsabilidad produce el resultado previsto. El punto clave es que no se sustituye la comprobación de capítulos anteriores.

Cuando activamos la condición de catálogo completo, los tres enlaces de detalle deben seguir funcionando. Cuando activamos la sección del layout, el listado y la portada deben seguir cargando aunque no la declaren. Cuando incorporamos las tarjetas, debe haber exactamente tres, con los identificadores correctos. Cuando reutilizamos la parcial en el detalle, debe haber una tarjeta para el producto solicitado.

Finalmente, el reto necesita verificar tanto casos positivos como negativos. Encontrar la etiqueta de precio económico una vez no basta: también hay que asegurarse de que no aparece en las tarjetas de Cuaderno y Carpeta. Este tipo de comprobación enseña a razonar sobre todas las ramas de una condición.

Para comprobar el resultado, observa los fragmentos HTML y los códigos HTTP con las herramientas del navegador. Después examina visualmente la página: una respuesta correcta también debe presentar los elementos en el lugar esperado.

Una lectura integrada del resultado de M01.

Tras los cinco puntos, el alumno puede recorrer una cadena completa sin introducir tecnologías nuevas. El navegador solicita `/Productos`; el enrutamiento selecciona `Index()`; la acción entrega los objetos `Producto`; la vista Razor tipada los recorre; la parcial representa tarjetas; el layout incorpora navegación y estructura; el servidor envía HTML.

Si el usuario selecciona un enlace de detalle, se produce otra solicitud HTTP, se selecciona `Detalle(int id)` y la vista recibe un solo objeto. El layout y la parcial pueden reutilizarse sin reconstruir por completo la interfaz.

Este recorrido prepara el siguiente módulo. Cuando M02 incorpore formularios y persistencia, no será necesario reaprender qué son un controlador, una vista, un resultado HTTP o un modelo tipado. La arquitectura y las convenciones seguirán siendo reconocibles.

El aprendizaje central no es memorizar sintaxis aislada de Razor, sino comprender **cómo los datos llegan a una representación coherente y reutilizable**, y cómo una modificación localizada debe conservar el comportamiento previo de la aplicación.

Continuidad y alcance del módulo M01.

La práctica 1.5 reproduce los conceptos en estados pequeños y observables: vistas tipadas ya existentes, condición Razor, layout con sección opcional, tarjeta parcial en el listado, reutilización de la tarjeta en el detalle y reto de etiqueta según precio. Cada estado se compila y se prueba contra Kestrel mediante HTTP real.

Con esta práctica se completa el **contenido funcional previsto de M01**: desde la primera petición hasta un catálogo con listados y fichas Razor reutilizables. Los **formularios, model binding de POST, validación con DataAnnotations y EF Core** se incorporarán en el M02, no en este módulo.

Para afianzar lo aprendido, recorre las cinco etapas del módulo con los proyectos incluidos y comprueba que cada cambio de código corresponde a un comportamiento observable. La práctica ofrece los procedimientos necesarios para continuar el desarrollo del catálogo.

Referencias técnicas.

- Microsoft Learn, Razor syntax: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/razor?view=aspnetcore-8.0
- Microsoft Learn, layouts: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/layout?view=aspnetcore-8.0
- Microsoft Learn, partial views: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/partial?view=aspnetcore-8.0
- Microsoft Learn, Partial Tag Helper: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/tag-helpers/built-in/partial-tag-helper?view=aspnetcore-8.0